

# Liste Linkate o Concatenate

---

Principio del Privilegio Minimo, `struct` Autoreferenziali,  
Liste Linkate

Questi lucidi sono basati sui lucidi di Programmazione 2 del Prof. Damiani e della Prof.ssa Baroglio



# Il Principio del Privilegio Minimo

# Il Principio del Privilegio Minimo



## Definizione

Nel contesto di una applicazione, al codice deve essere garantita solo la quantità di privilegi necessaria per eseguire il compito per cui è stato progettato, non di più.

# Il Qualificatore `const` I

- `const` informa il compilatore che una variabile non deve essere modificata
  - Supporta il principio del privilegio minimo
  - Riduce i tempi di debug e previene effetti collaterali non intenzionali
-

# Il Qualificatore `const` I

- `const` informa il compilatore che una variabile non deve essere modificata
  - Supporta il principio del privilegio minimo
  - Riduce i tempi di debug e previene effetti collaterali non intenzionali
- In C ci sono quattro modi per specificare i privilegi di accesso:

|      |          | Puntatore                       |                                       |
|------|----------|---------------------------------|---------------------------------------|
|      | Costante | No                              | Si                                    |
| Dati | No       | <code>char *strPtr</code>       | <code>const char *strPtr</code>       |
|      | Si       | <code>char *const strPtr</code> | <code>const char* const strPtr</code> |

# Il Qualificatore `const` I

- `const` informa il compilatore che una variabile non deve essere modificata
  - Supporta il principio del privilegio minimo
  - Riduce i tempi di debug e previene effetti collaterali non intenzionali
- In C ci sono quattro modi per specificare i privilegi di accesso:

|      |          | Puntatore                       |                                       |
|------|----------|---------------------------------|---------------------------------------|
|      | Costante | No                              | Si                                    |
| Dati | No       | <code>char *strPtr</code>       | <code>const char *strPtr</code>       |
|      | Si       | <code>char *const strPtr</code> | <code>const char* const strPtr</code> |

`const` prima del tipo `T*`

- si riferisce al puntatore

`const` dopo il tipo `T*`

- si riferisce al dato

# Il Qualificatore `const` II



- Come decidere quale tipo di privilegio usare?
  - Facendosi guidare dal principio del privilegio minimo
  - Ovvero, assegnando sempre a una funzione un accesso sufficiente per eseguire l'attività specificata
    - Ma assolutamente non di più



# Strutture Dati Autoreferenziali



# Strutture Dati Autoreferenziali



- Struttura Autoreferenziale

- `struct` che contiene un membro puntatore che punta a una struttura dello stesso tipo.

```
struct node {  
    int data;  
    struct node *nextPtr;  
};
```

- Esempio:

- Due oggetti di tipo `struct node` collegati a formare una lista
- Il secondo elemento ha puntatore `NULL` che indica la fine della struttura dati





# Lista Linkata

# Lista Linkata

- **Lista Linkata**
  - È una collezione lineare di strutture autoreferenziali dette elementi o nodi



# Lista Linkata



- **Lista Linkata**
  - È una collezione lineare di strutture autoreferenziali dette elementi o nodi
- Definizione astratta:
  - Una lista linkata **ls** (di elementi di tipo **T**) è:
    - **Q** la lista vuota, indicata con **/**
    - **Q** una coppia **<e, t1>** dove:
      - **e** è un elemento (di tipo **T**)
        - Detto **testa (head)** di **ls**
      - **t1** è una lista (di elementi di tipo **T**)
        - Detta **coda (tail)** di **ls**

# Lista Linkata: Notazione

- Una lista di  $n$  elementi:
  - $\langle e_0, \langle e_1, \dots \langle e_i, \dots \langle e_{n-1}, / \rangle \dots \rangle \dots \rangle \rangle$
  - per leggibilità viene anche indicata come:
    - $[e_0, e_1, \dots, e_i, \dots, e_{n-1}]$
  - e la lista vuota come:
    - $[]$
- Per ogni  $i \in 0 \dots n - 2$ :
  - L'elemento  $e_i$  è detto il predecessore di  $e_{i+1}$
  - L'elemento  $e_{i+1}$  è detto il successore di  $e_i$

# Lista Linkata: In C



- Da definizione astratta:
  - Una lista linkata **ls** (di elementi di tipo **T**) è:
    - **Q** la lista vuota, indicata con **/**
    - **Q** una coppia **<e, t1>** dove:
      - **e** è un elemento (di tipo **T**)
      - **t1** è una lista (di elementi di tipo **T**)
- ... a implementazione concreta in C:
  - Una variabile **ls** (di tipo **List**, id est **struct listNode\***) rappresenta una lista (di elementi di tipo **T**) se contiene:
    - **Q** la lista vuota, indicata con il valore **NULL**
    - **Q** un puntatore ad una **struct listNode** dove:
      - **data** contiene un valore (di tipo **T**)
      - **next** rappresenta una lista (di elementi di tipo **T**)

```
// Tipo per liste di (elementi di tipo) T
typedef ... T; // Il tipo degli elementi
typedef struct listNode *List; // Il tipo delle liste
struct listNode { // Il tipo dei nodi della lista
    T data;
    List next;
};
```

Usando typedef

```
// Tipo per liste
(senza typedef)

struct listNode {
    ... data;
    struct listNode *next;
};
```

Senza typedef

11-0

# Lista Linkata: Un Esempio con Creazione Lista I

- Definiamo una lista di `int` e simuliamo il programma:

```
// Definiamo lista di interi
struct listIntNode {
    int data;
    struct listIntNode *next;
};
```

```
void main(void) {

}
```

\*Puntatore dangling: puntatore che non punta ad un oggetto valido. Caso speciale di memory safety violation.

# Lista Linkata: Un Esempio con Creazione Lista I

- Definiamo una lista di `int` e simuliamo il programma:

```
// Definiamo lista di interi
struct listIntNode {
    int data;
    struct listIntNode *next;
};
```

```
void main(void) {
    struct listIntNode *ls;

}
```



\*Puntatore dangling: puntatore che non punta ad un oggetto valido. Caso speciale di memory safety violation.



# Lista Linkata: Un Esempio con Creazione Lista I

- Definiamo una lista di `int` e simuliamo il programma:

```
// Definiamo lista di interi
struct listIntNode {
    int data;
    struct listIntNode *next;
};
```

```
void main(void) {
    struct listIntNode *ls;
}
```



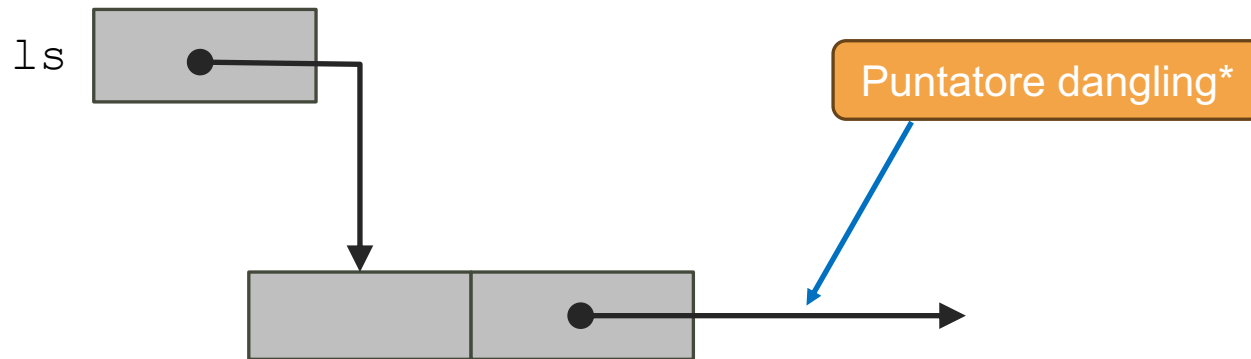
\*Puntatore dangling: puntatore che non punta ad un oggetto valido. Caso speciale di memory safety violation.

# Lista Linkata: Un Esempio con Creazione Lista I

- Definiamo una lista di `int` e simuliamo il programma:

```
// Definiamo lista di interi
struct listIntNode {
    int data;
    struct listIntNode *next;
};
```

```
void main(void) {
    struct listIntNode *ls;
    ls = malloc(sizeof(struct listIntNode));
}
```



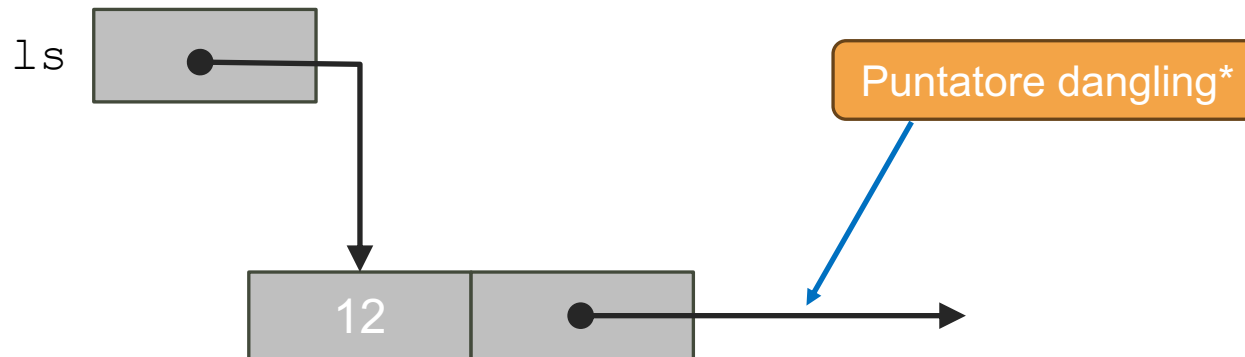
\*Puntatore dangling: puntatore che non punta ad un oggetto valido. Caso speciale di memory safety violation.

# Lista Linkata: Un Esempio con Creazione Lista I

- Definiamo una lista di `int` e simuliamo il programma:

```
// Definiamo lista di interi
struct listIntNode {
    int data;
    struct listIntNode *next;
};
```

```
void main(void) {
    struct listIntNode *ls;
    ls = malloc(sizeof(struct listIntNode));
    ls->data = 12;
}
```



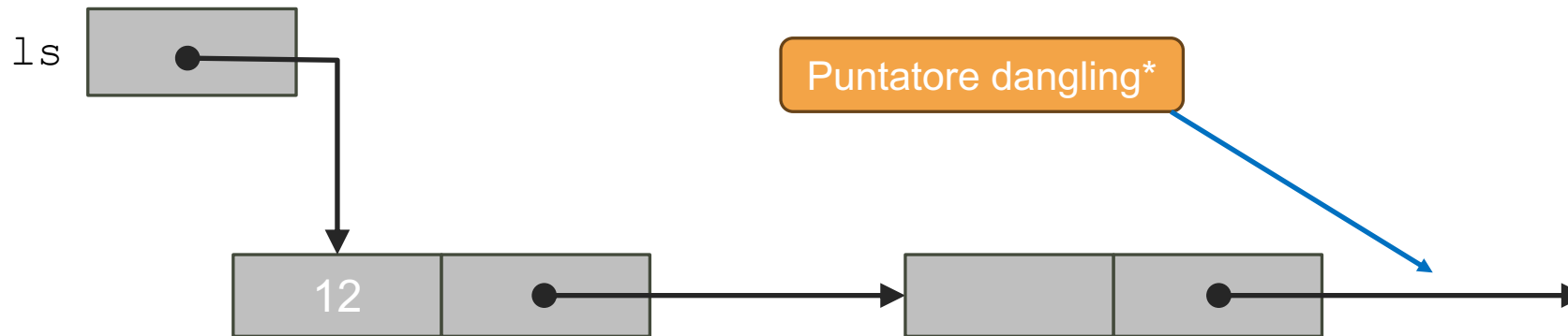
\*Puntatore dangling: puntatore che non punta ad un oggetto valido. Caso speciale di memory safety violation.

# Lista Linkata: Un Esempio con Creazione Lista I

- Definiamo una lista di `int` e simuliamo il programma:

```
// Definiamo lista di interi
struct listIntNode {
    int data;
    struct listIntNode *next;
};
```

```
void main(void) {
    struct listIntNode *ls;
    ls = malloc(sizeof(struct listIntNode));
    ls->data = 12;
    ls->next = malloc(sizeof(struct listIntNode));
}
```



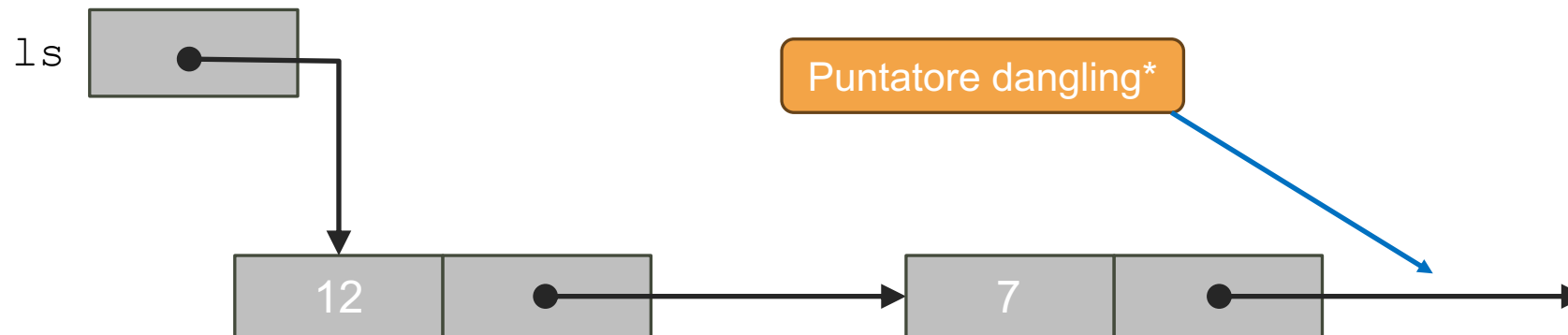
\*Puntatore dangling: puntatore che non punta ad un oggetto valido. Caso speciale di memory safety violation.

# Lista Linkata: Un Esempio con Creazione Lista I

- Definiamo una lista di `int` e simuliamo il programma:

```
// Definiamo lista di interi
struct listIntNode {
    int data;
    struct listIntNode *next;
};
```

```
void main(void) {
    struct listIntNode *ls;
    ls = malloc(sizeof(struct listIntNode));
    ls->data = 12;
    ls->next = malloc(sizeof(struct listIntNode));
    ls->next->data = 7;
}
```



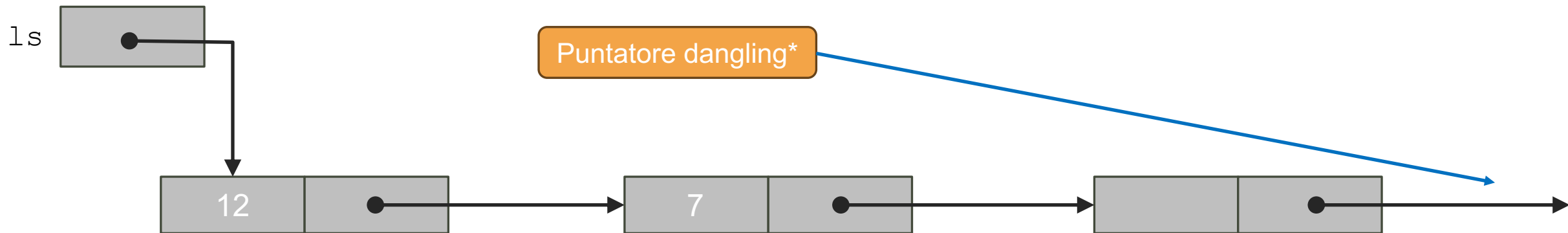
\*Puntatore dangling: puntatore che non punta ad un oggetto valido. Caso speciale di memory safety violation.

# Lista Linkata: Un Esempio con Creazione Lista I

- Definiamo una lista di `int` e simuliamo il programma:

```
// Definiamo lista di interi
struct listIntNode {
    int data;
    struct listIntNode *next;
};
```

```
void main(void) {
    struct listIntNode *ls;
    ls = malloc(sizeof(struct listIntNode));
    ls->data = 12;
    ls->next = malloc(sizeof(struct listIntNode));
    ls->next->data = 7;
    ls->next->next = malloc(sizeof(struct listIntNode));
}
```



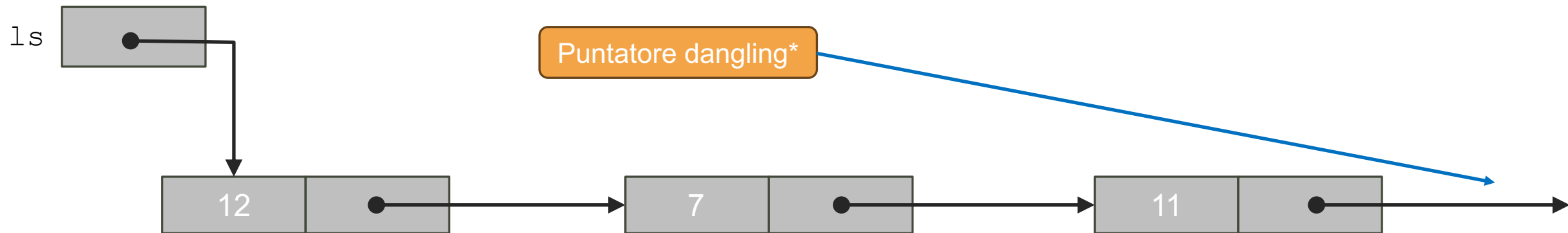
\*Puntatore dangling: puntatore che non punta ad un oggetto valido. Caso speciale di memory safety violation.

# Lista Linkata: Un Esempio con Creazione Lista I

- Definiamo una lista di `int` e simuliamo il programma:

```
// Definiamo lista di interi
struct listIntNode {
    int data;
    struct listIntNode *next;
};
```

```
void main(void) {
    struct listIntNode *ls;
    ls = malloc(sizeof(struct listIntNode));
    ls->data = 12;
    ls->next = malloc(sizeof(struct listIntNode));
    ls->next->data = 7;
    ls->next->next = malloc(sizeof(struct listIntNode));
    ls->next->next->data = 11;
}
```



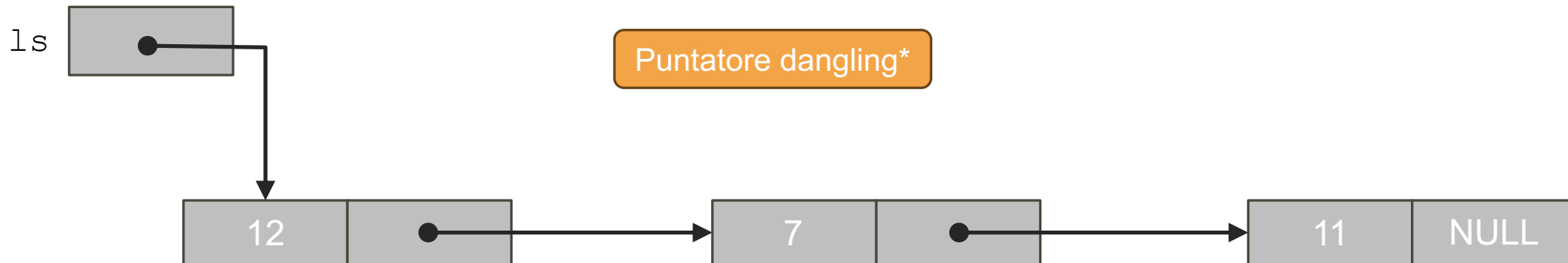
\*Puntatore dangling: puntatore che non punta ad un oggetto valido. Caso speciale di memory safety violation.

# Lista Linkata: Un Esempio con Creazione Lista I

- Definiamo una lista di `int` e simuliamo il programma:

```
// Definiamo lista di interi
struct listIntNode {
    int data;
    struct listIntNode *next;
};
```

```
void main(void) {
    struct listIntNode *ls;
    ls = malloc(sizeof(struct listIntNode));
    ls->data = 12;
    ls->next = malloc(sizeof(struct listIntNode));
    ls->next->data = 7;
    ls->next->next = malloc(sizeof(struct listIntNode));
    ls->next->next->data = 11;
    ls->next->next->next = NULL;
}
```



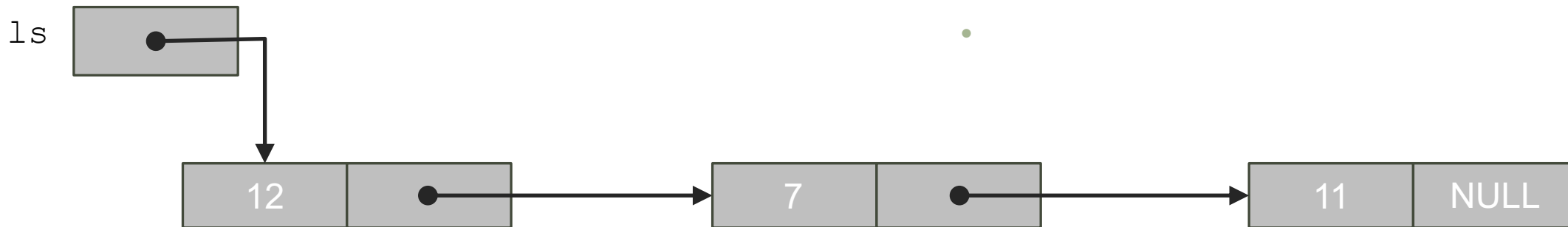
\*Puntatore dangling: puntatore che non punta ad un oggetto valido. Caso speciale di memory safety violation.



# Lista Linkata: Un Esempio con Creazione Lista II

- Stato della memoria a fine esecuzione:

- `ls` dove è memorizzato?



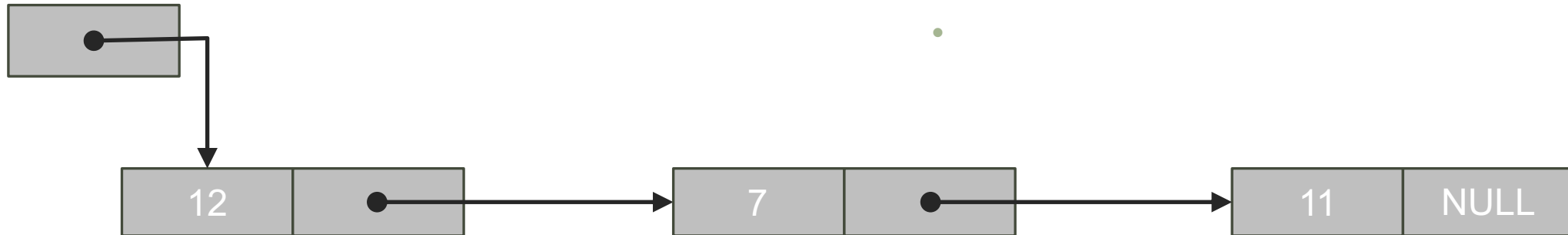
# Lista Linkata: Un Esempio con Creazione Lista II

- Stato della memoria a fine esecuzione:

- `ls` dove è memorizzato?
  - **Stack** (allocato automaticamente)

**Stack**

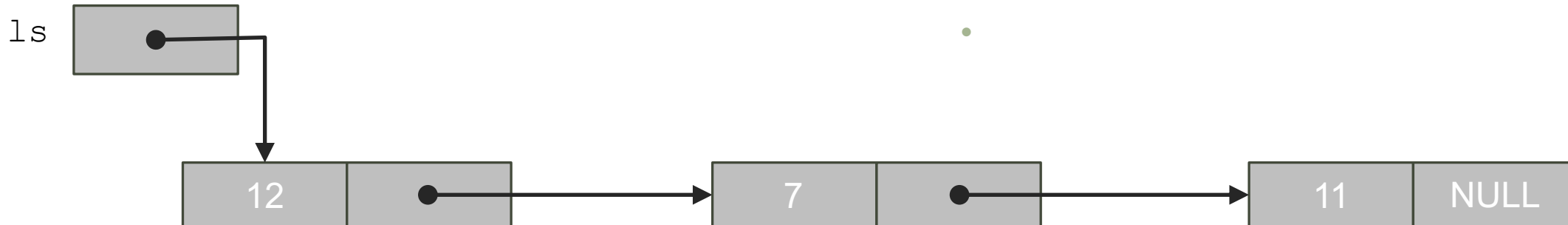
`ls`



# Lista Linkata: Un Esempio con Creazione Lista II

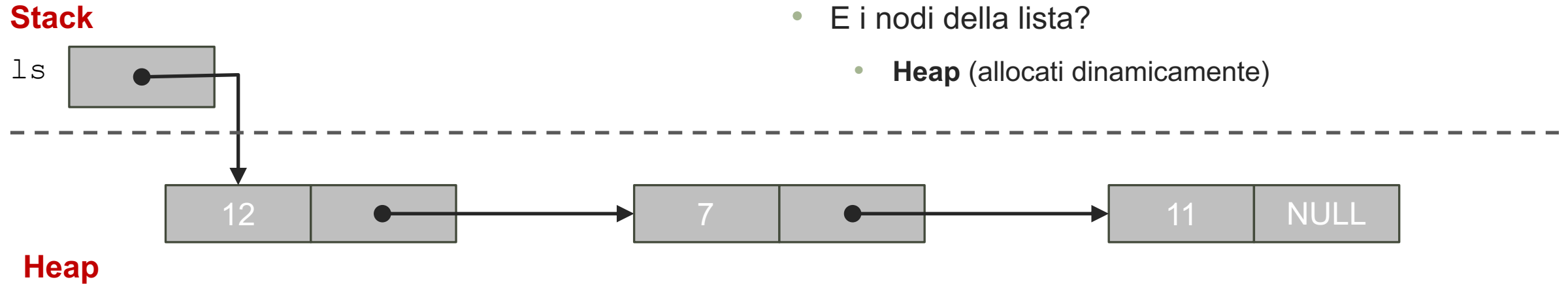
- Stato della memoria a fine esecuzione:
  - `ls` dove è memorizzato?
    - **Stack** (allocato automaticamente)
  - E i nodi della lista?
    -

**Stack**

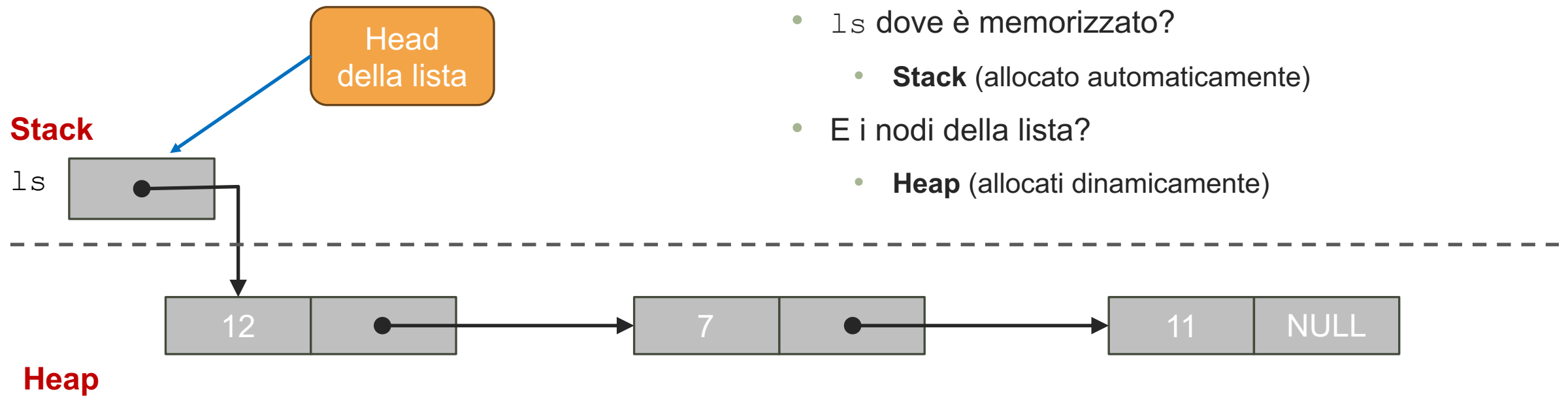


# Lista Linkata: Un Esempio con Creazione Lista II

- Stato della memoria a fine esecuzione:
  - `ls` dove è memorizzato?
    - **Stack** (allocato automaticamente)
  - E i nodi della lista?
    - **Heap** (allocati dinamicamente)

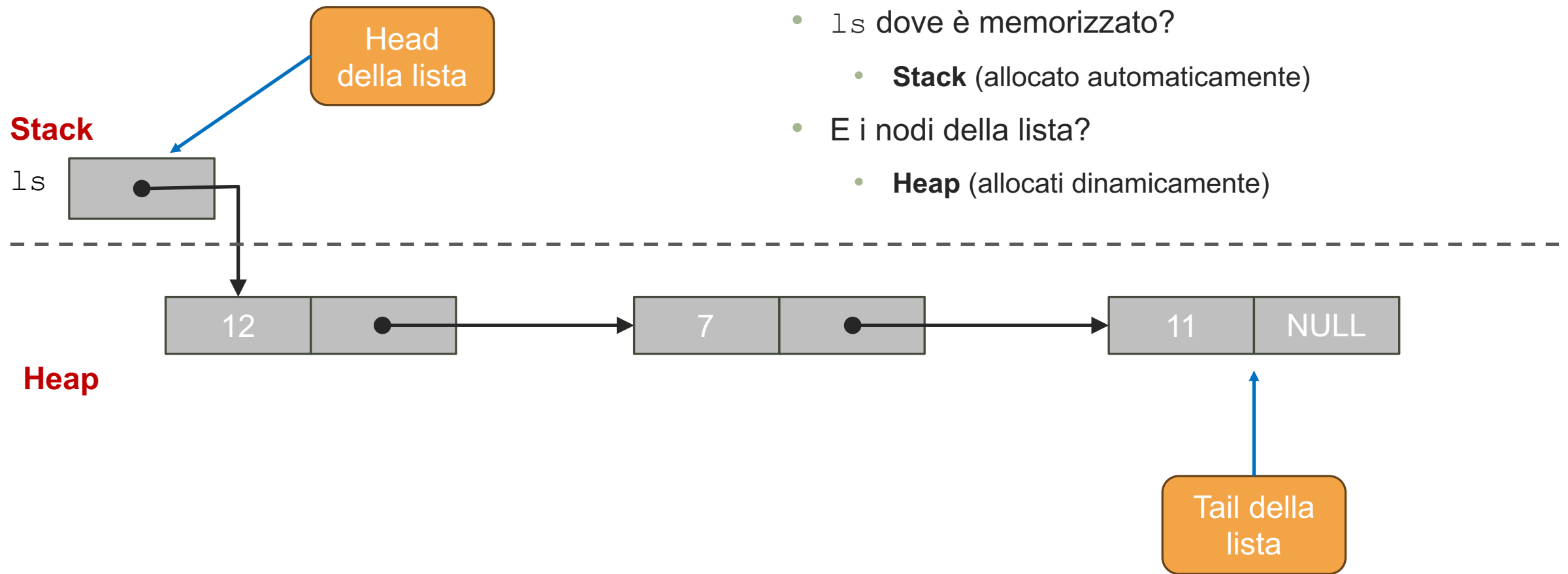


# Lista Linkata: Un Esempio con Creazione Lista II



- Stato della memoria a fine esecuzione:
  - `ls` dove è memorizzato?
    - **Stack** (allocato automaticamente)
  - E i nodi della lista?
    - **Heap** (allocati dinamicamente)

# Lista Linkata: Un Esempio con Creazione Lista II



- Stato della memoria a fine esecuzione:
  - `ls` dove è memorizzato?
    - **Stack** (allocato automaticamente)
  - E i nodi della lista?
    - **Heap** (allocati dinamicamente)

# Nodo Mono- e Multi-Dato

- La parte dati degli elementi (nodi) della lista può essere:
  - Un unico dato (membro di struct)

- Esempio:

```
// Definiamo lista di interi
struct listIntNode {
    int data;
    struct listIntNode *next;
};
```

- Oppure dati (membri) multipli anche di tipi diversi

- Esempio:

```
// Definiamo lista di interi e caratteri
struct listMixedNode {
    int    data1;
    char   data2;
    ...
    double dataN;

    struct listMixedNode *next;
};
```

# Testa della Lista

- `ls` dichiarato come `struct listMixedNode*`
  - Perché non `struct listMixedNode`?
  - 
  - 
  -

```
struct listMixedNode {  
    int    data1;  
    char   data2;  
    ...  
    double dataN;  
  
    struct listMixedNode *next;  
};  
  
struct listMixedNode *ls;
```



# Testa della Lista

- `ls` dichiarato come `struct listMixedNode*`
  - Perché non `struct listMixedNode`?
  - Due problemi:
    - 
    -

```
struct listMixedNode {  
    int    data1;  
    char   data2;  
    ...  
    double dataN;  
  
    struct listMixedNode *next;  
};  
  
struct listMixedNode *ls;
```

# Testa della Lista

- `ls` dichiarato come `struct listMixedNode*`
  - Perché non `struct listMixedNode`?
  - Due problemi:
    - Come rappresentare la lista vuota?
    -

```
struct listMixedNode {  
    int    data1;  
    char   data2;  
    ...  
    double dataN;  
  
    struct listMixedNode *next;  
};  
  
struct listMixedNode *ls;
```

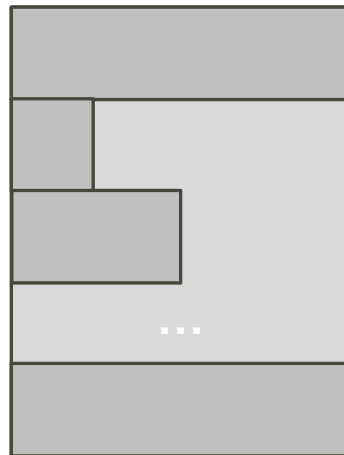
# Testa della Lista

- `ls` dichiarato come `struct listMixedNode*`
  - Perché non `struct listMixedNode`?
  - Due problemi:
    - Come rappresentare la lista vuota?
    - Spreco spazio (soprattutto se nodi sono grossi)

`struct listMixedNode*`



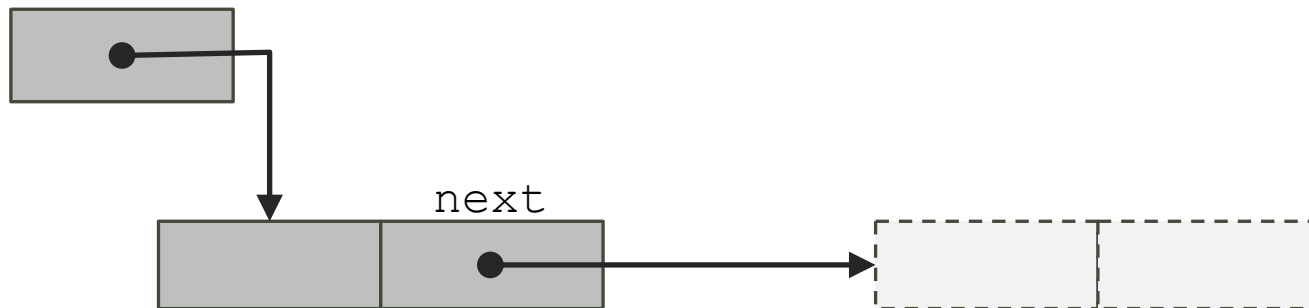
`struct listMixedNode`



```
struct listMixedNode {  
    int    data1;  
    char   data2;  
    ...  
    double dataN;  
  
    struct listMixedNode *next;  
};  
  
struct listMixedNode *ls;
```

# Coda della Lista

- Qual è il rischio se si dimentica di impostare `next` a `NULL`?
  - Una locazione di memoria contenente bit casuali viene interpretata come un indirizzo
    - Denominato anche «dangling pointer»
    - Usando `next` si cerca di accedere a un campo di un nodo non allocato
      - Tipicamente il processo viene terminato (segmentation fault)



# Lista linkata



- Una lista linkata:

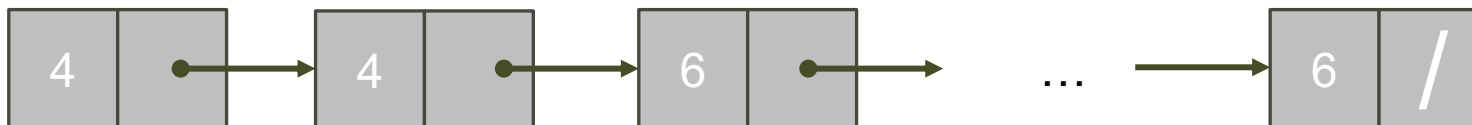
- Può essere non ordinata



- Può essere ordinata (proprietà extra che impatta sulla collocazione degli elementi)



- Può contenere lo stesso dato più volte



# Array vs Linked List



| Array | Linked List |
|-------|-------------|
|-------|-------------|

# Array vs Linked List



| Array                     | Linked List                                                 |
|---------------------------|-------------------------------------------------------------|
| – Hanno dimensioni fisse. | + La lunghezza può aumentare o diminuire secondo necessità. |

# Array vs Linked List



| Array                                                                     | Linked List                                                                                                                                      |
|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| – Hanno dimensioni fisse.                                                 | + La lunghezza può aumentare o diminuire secondo necessità.                                                                                      |
| – Un array può contenere più celle del necessario, ma ciò spreca memoria. | + – Possono far risparmiare memoria, ma, i puntatori nei nodi richiedono memoria aggiuntiva e l'allocazione di memoria comporta un sovraccarico. |



# Array vs Linked List



| Array                                                                                                                                                                                  | Linked List                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| – Hanno dimensioni fisse.                                                                                                                                                              | + La lunghezza può aumentare o diminuire secondo necessità.                                                                                      |
| – Un array può contenere più celle del necessario, ma ciò spreca memoria.                                                                                                              | + – Possono far risparmiare memoria, ma, i puntatori nei nodi richiedono memoria aggiuntiva e l'allocazione di memoria comporta un sovraccarico. |
| – L'inserimento e l'eliminazione da un array ordinato possono essere lenti: tutti gli elementi che seguono l'elemento inserito o eliminato devono essere spostati in modo appropriato. | + Possono essere mantenute in ordine inserendo ogni nuovo elemento nel punto appropriato della lista.                                            |

# Array vs Linked List



| Array                                                                                                                                                                                  | Linked List                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| – Hanno dimensioni fisse.                                                                                                                                                              | + La lunghezza può aumentare o diminuire secondo necessità.                                                                                      |
| – Un array può contenere più celle del necessario, ma ciò spreca memoria.                                                                                                              | + – Possono far risparmiare memoria, ma, i puntatori nei nodi richiedono memoria aggiuntiva e l'allocazione di memoria comporta un sovraccarico. |
| – L'inserimento e l'eliminazione da un array ordinato possono essere lenti: tutti gli elementi che seguono l'elemento inserito o eliminato devono essere spostati in modo appropriato. | + Possono essere mantenute in ordine inserendo ogni nuovo elemento nel punto appropriato della lista.                                            |
| + Gli elementi sono contigui in memoria e accessibili direttamente tramite indice.                                                                                                     | – Non consentono un accesso così immediato agli elementi (i nodi generalmente non sono contigui in memoria).                                     |

# Operazioni sulle Liste Linkate

- Visita della lista:
  - La lista può essere percorsa per calcolare/estrarre informazioni dalla lista
    - Esempi: Quanti sono gli elementi? Somma dei valori contenuti negli elementi in posizione dispari?
- Inserzione e rimozione di elementi:
  - Gli elementi possono essere inseriti o rimossi:
    - All'inizio (in testa o head)
    - Alla fine (in coda o tail)
    - Fra due elementi qualsiasi
  - Se la lista è ordinata (presenta una certa proprietà), le operazioni che modificano la lista dovranno preservare il suo ordinamento (preservare la proprietà)

# Visita degli Elementi di una Lista

# Visita I

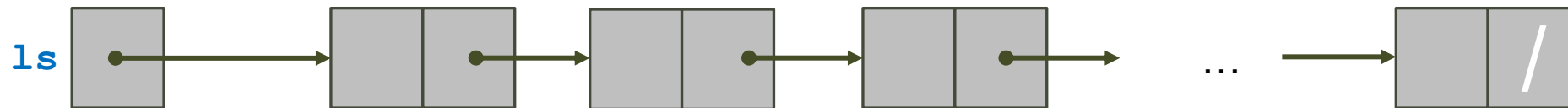


- La visita (degli elementi) di una lista:
  - È un'operazione che percorre tutti i nodi di una lista, eventualmente effettuando un'operazione sugli elementi.
- Esempi:
  - `void printCharList(CharList ls);`
    - Stampa gli elementi della lista
  - `size_t length(List ls);`
    - Restituisce il numero degli elementi della lista
  - `int sumIntList(IntList ls);`
    - Restituisce la somma degli elementi della lista
  - `size_t count(List ls, Bool (*predicate)(T e));`
    - Restituisce il numero degli elementi della lista che soddisfa il predicato 'predicate'

# Visita II

```
/** @brief Assume: ls è una lista. Esegue: visita di tutti i nodi di ls.  
*/  
... visit(List ls) {
```

```
    return ...;  
}
```



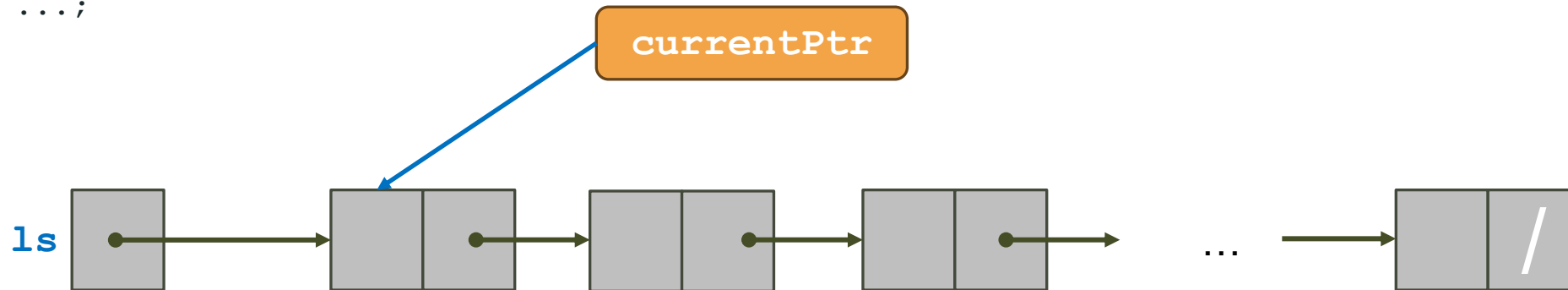
- Complessità computazionale delle operazioni di visita precedenti, rispetto alla lunghezza  $n$  della lista
  - Tempo:  $O(n)$ , Spazio:  $O(1)$

# Visita II

```
/** @brief Assume: ls è una lista. Esegue: visita di tutti i nodi di ls.
*/
```

```
... visit(List ls) {
    // La lista currentPtr è il successore di ls ancora da visitare
    for (List currentPtr = ls;

        /* INSERIRE QUI IL CODICE CHE VISITA IL NODO currentPtr */
    )
    return ...;
}
```

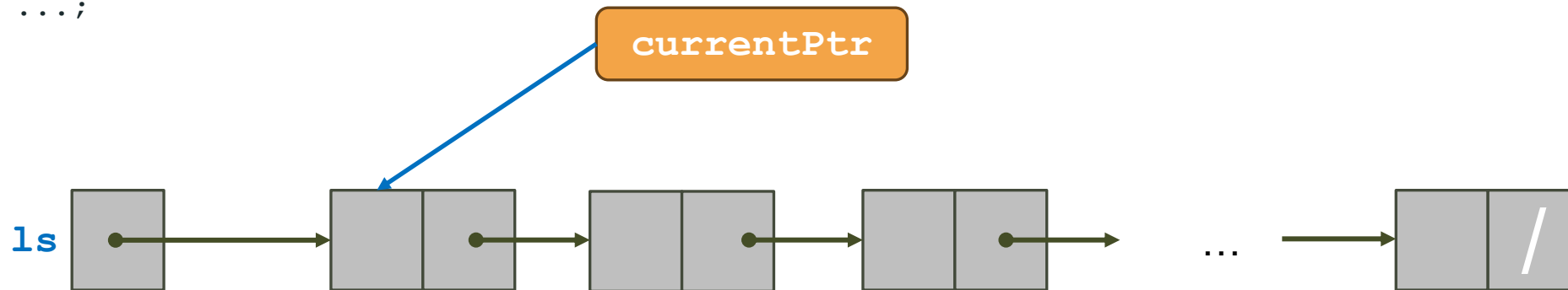


- Complessità computazionale delle operazioni di visita precedenti, rispetto alla lunghezza  $n$  della lista
  - Tempo:  $O(n)$ , Spazio:  $O(1)$

# Visita II

```
/** @brief Assume: ls è una lista. Esegue: visita di tutti i nodi di ls.
*/
... visit(List ls) {
    // La lista currentPtr è il successore di ls ancora da visitare
    for (List currentPtr = ls;

        currentPtr = currentPtr->next) {
        /* INSERIRE QUI IL CODICE CHE VISITA IL NODO currentPtr */
    }
    return ...;
}
```



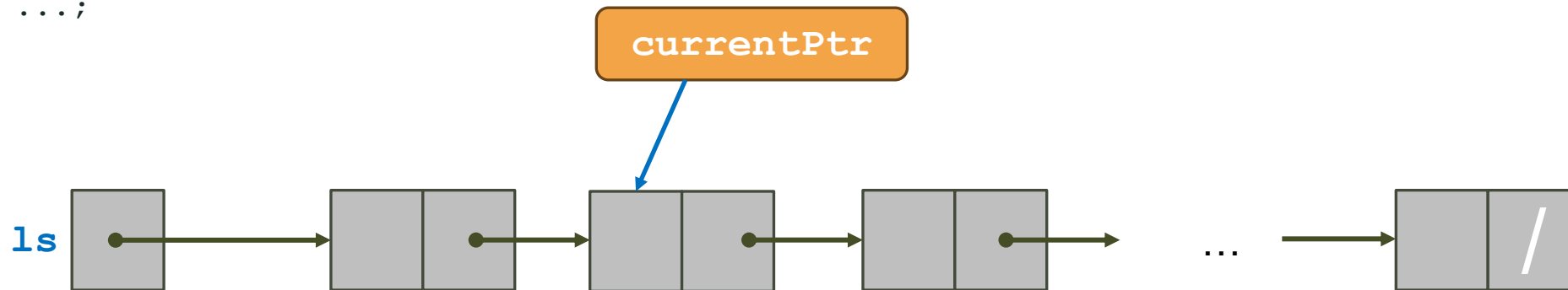
- Complessità computazionale delle operazioni di visita precedenti, rispetto alla lunghezza  $n$  della lista
  - Tempo:  $O(n)$ , Spazio:  $O(1)$



# Visita II

```
/** @brief Assume: ls è una lista. Esegue: visita di tutti i nodi di ls.
*/
... visit(List ls) {
    // La lista currentPtr è il successore di ls ancora da visitare
    for (List currentPtr = ls;

        currentPtr = currentPtr->next) {
        /* INSERIRE QUI IL CODICE CHE VISITA IL NODO currentPtr */
    }
    return ...;
}
```

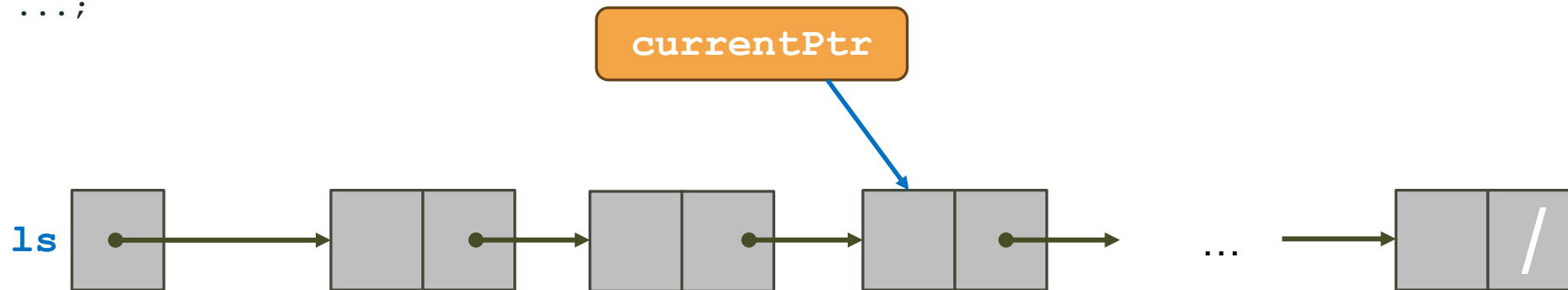


- Complessità computazionale delle operazioni di visita precedenti, rispetto alla lunghezza  $n$  della lista
  - Tempo:  $O(n)$ , Spazio:  $O(1)$

# Visita II

```
/** @brief Assume: ls è una lista. Esegue: visita di tutti i nodi di ls.
*/
... visit(List ls) {
    // La lista currentPtr è il successore di ls ancora da visitare
    for (List currentPtr = ls;

        currentPtr = currentPtr->next) {
        /* INSERIRE QUI IL CODICE CHE VISITA IL NODO currentPtr */
    }
    return ...;
}
```

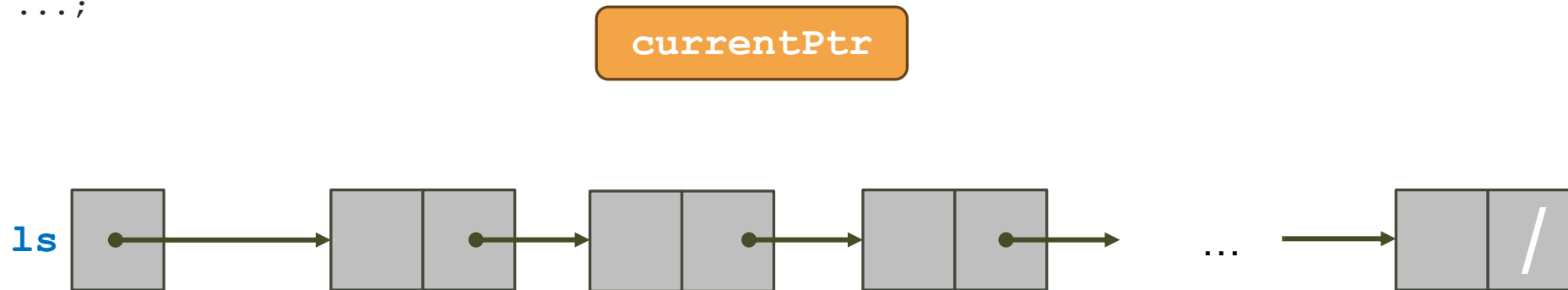


- Complessità computazionale delle operazioni di visita precedenti, rispetto alla lunghezza  $n$  della lista
  - Tempo:  $O(n)$ , Spazio:  $O(1)$

# Visita II

```
/** @brief Assume: ls è una lista. Esegue: visita di tutti i nodi di ls.
*/
... visit(List ls) {
    // La lista currentPtr è il successore di ls ancora da visitare
    for (List currentPtr = ls;

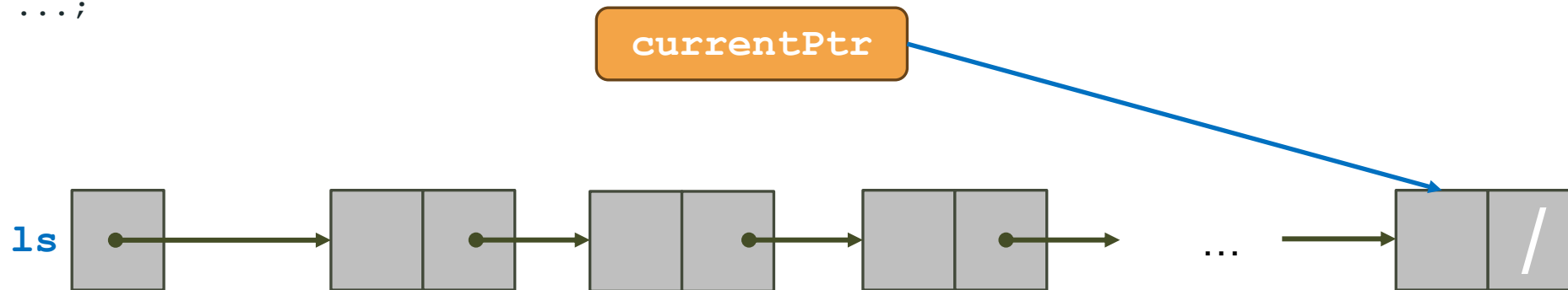
        currentPtr = currentPtr->next) {
        /* INSERIRE QUI IL CODICE CHE VISITA IL NODO currentPtr */
    }
    return ...;
}
```



- Complessità computazionale delle operazioni di visita precedenti, rispetto alla lunghezza  $n$  della lista
  - Tempo:  $O(n)$ , Spazio:  $O(1)$

# Visita II

```
/** @brief Assume: ls è una lista. Esegue: visita di tutti i nodi di ls.
 */
... visit(List ls) {
    // La lista currentPtr è il successore di ls ancora da visitare
    for (List currentPtr = ls;
        currentPtr != NULL;
        currentPtr = currentPtr->next) {
        /* INSERIRE QUI IL CODICE CHE VISITA IL NODO currentPtr */
    }
    return ...;
}
```

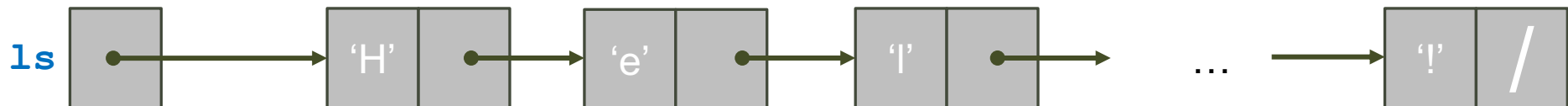


- Complessità computazionale delle operazioni di visita precedenti, rispetto alla lunghezza  $n$  della lista
  - Tempo:  $O(n)$ , Spazio:  $O(1)$

# Visita: Esempio Stampa degli Elementi

```
/** @brief Assume: ls è una lista.  
    Esegue: stampa gli elementi di ls.  
*/  
void printCharList(CharList ls) {  
    // La lista currenPtr è il successore di ls ancora da stampare  
    for (CharList currentPtr = ls;  
        currentPtr != NULL;  
        currentPtr = currentPtr->next) {  
        printf("%c\n", currentPtr->data);  
    }  
}
```

```
typedef struct  
charListNode *CharList;  
struct charListNode {  
    char data;  
    CharList next;  
};
```



# Visita: Esempio Lunghezza Lista (# Elementi)

```

/** @brief Assume: ls è una lista.
    Esegue: conta i nodi della lista.
 */
size_t length(CharList ls) {
    size_t noe = 0;
    // noe contiene il numero degli elementi di ls già visitati.
    // La lista currentPtr è il successore di ls ancora da visitare
    for (List currentPtr = ls;
        currentPtr != NULL;
        currentPtr = currentPtr->next) {
        noe++;
    }
    return noe;
}

```

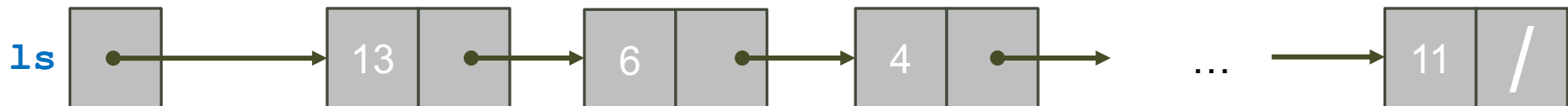


# Visita: Esempio Somma degli Elementi



```
/** @brief Assume: ls è una lista.  
    Restituisce: il numero di elementi di ls.  
*/  
int sumIntList(IntList ls) {  
    int sum = 0;  
    // sum contiene la somma degli elementi di ls già visitati,  
    // currentPtr è il successore di ls ancora da visitare  
    for (IntList currentPtr = ls;  
         currentPtr != NULL;  
         currentPtr = currentPtr->next) {  
        sum += currentPtr->data;  
    }  
    return sum;  
}
```

```
typedef struct  
intListNode *IntList;  
struct intListNode {  
    int data;  
    IntList next;  
};
```



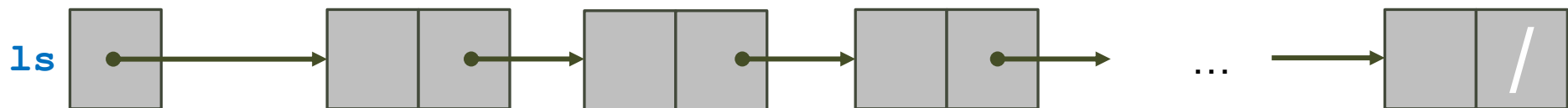
# Visita: Esempio Conta Elementi che Soddisfa Condizione

```

/** @brief Assume: ls è una lista.
    Conta i nodi che soddisfano la condizione predicate
*/
size_t count(CharList ls, int (*predicate)(T e)) {
    size_t noe = 0;
    // noe contiene il numero di elementi che soddisfano 'predicate'
    // currentPtr è il successore di ls ancora da visitare
    for (List currentPtr = ls;
        currentPtr != NULL;
        currentPtr = currentPtr->next) {
        if (predicate(currentPtr->data)) {
            noe++;
        }
    }
    return noe;
}

```

puntatore a funzione





# Inserimento in una Lista

# Inserimento



- Illustriamo tre diverse operazioni di inserimento:
  - `_Bool insertInSortedCharList(CharList *sPtr, char value);`
    - Inserisce al posto giusto in una lista ordinata
  - `_Bool insertBefore(List *lsPtr, T e, List positionPtr);`
    - Inserisce prima del nodo puntato da `positionPtr` (se `positionPtr == NULL`, inserisce in coda)
  - `_Bool insertAfter(List *lsPtr, T e, List positionPtr);`
    - Inserisce dopo il nodo puntato da `positionPtr` (se `positionPtr == NULL`, inserisce in testa)
- Tutte e tre le operazioni restituiscono l'esito:
  - Quando l'inserimento può fallire?
    - 
    -

# Inserimento



- Illustriamo tre diverse operazioni di inserimento:
  - `_Bool insertInSortedCharList(CharList *sPtr, char value);`
    - Inserisce al posto giusto in una lista ordinata
  - `_Bool insertBefore(List *lsPtr, T e, List positionPtr);`
    - Inserisce prima del nodo puntato da `positionPtr` (se `positionPtr == NULL`, inserisce in coda)
  - `_Bool insertAfter(List *lsPtr, T e, List positionPtr);`
    - Inserisce dopo il nodo puntato da `positionPtr` (se `positionPtr == NULL`, inserisce in testa)
- Tutte e tre le operazioni restituiscono l'esito:
  - Quando l'inserimento può fallire?
    - Se `malloc` restituisce 0, allora non è possibile allocare nuova memoria e l'inserimento fallisce.
    -

# Inserimento



- Illustriamo tre diverse operazioni di inserimento:
  - `_Bool insertInSortedCharList(CharList *sPtr, char value);`
    - Inserisce al posto giusto in una lista ordinata
  - `_Bool insertBefore(List *lsPtr, T e, List positionPtr);`
    - Inserisce prima del nodo puntato da `positionPtr` (se `positionPtr == NULL`, inserisce in coda)
  - `_Bool insertAfter(List *lsPtr, T e, List positionPtr);`
    - Inserisce dopo il nodo puntato da `positionPtr` (se `positionPtr == NULL`, inserisce in testa)
- Tutte e tre le operazioni restituiscono l'esito:
  - Quando l'inserimento può fallire?
    - Se `malloc` restituisce 0, allora non è possibile allocare nuova memoria e l'inserimento fallisce.
    - Se l'elemento cercato non è presente

# Inserimento



Passaggio per riferimento

- Illustriamo tre diverse operazioni di inserimento:
  - `_Bool insertInSortedCharList(CharList *sPtr, char value);`
    - Inserisce al posto giusto in una lista ordinata
  - `_Bool insertBefore(List *lsPtr, T e, List positionPtr);`
    - Inserisce prima del nodo puntato da `positionPtr` (se `positionPtr == NULL`, inserisce in coda)
  - `_Bool insertAfter(List *lsPtr, T e, List positionPtr);`
    - Inserisce dopo il nodo puntato da `positionPtr` (se `positionPtr == NULL`, inserisce in testa)
- Tutte e tre le operazioni restituiscono l'esito:
  - Quando l'inserimento può fallire?
    - Se `malloc` restituisce 0, allora non è possibile allocare nuova memoria e l'inserimento fallisce.
    - Se l'elemento cercato non è presente

# Inserimento in una Lista Ordinata I

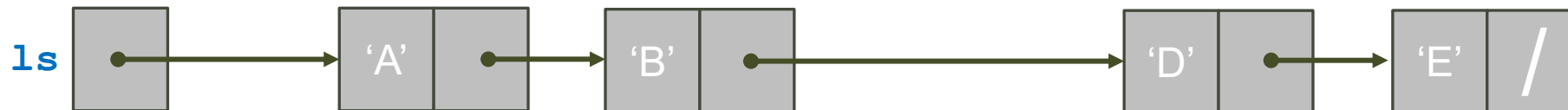
- Supponiamo di voler inserire 'C' in una lista di caratteri `ls`
  - `ls` contiene già ['A', 'B', 'D', 'E']

- 

1.

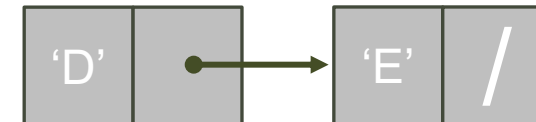
2.

3.



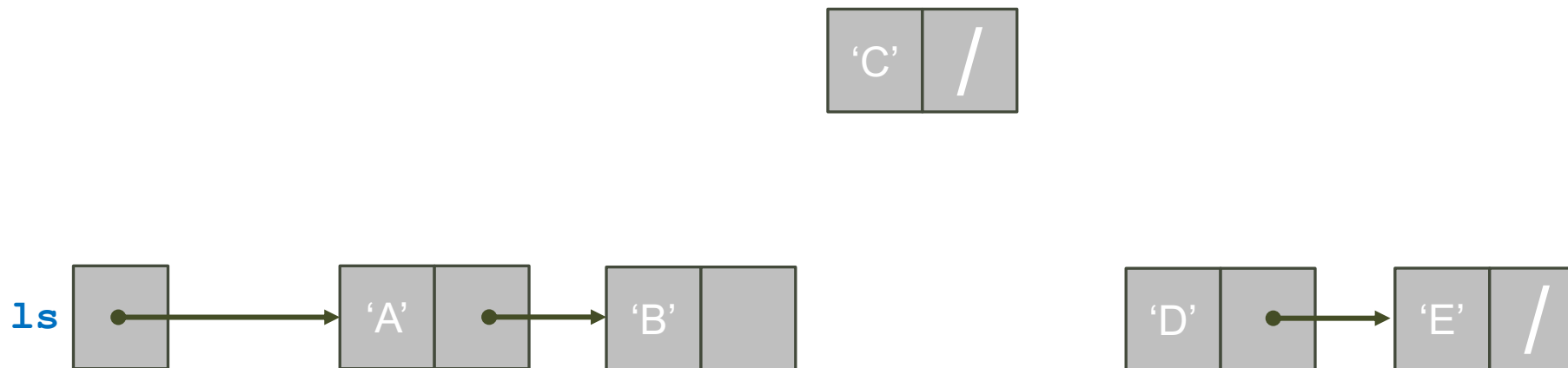
# Inserimento in una Lista Ordinata I

- Supponiamo di voler inserire 'C' in una lista di caratteri `ls`
  - `ls` contiene già ['A', 'B', 'D', 'E']
- L'inserimento consiste in 3 passi:
  - 1.
  - 2.
  - 3.



# Inserimento in una Lista Ordinata I

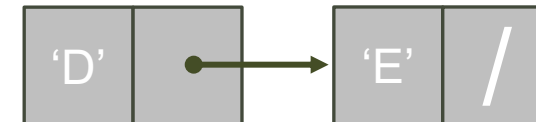
- Supponiamo di voler inserire 'C' in una lista di caratteri `ls`
  - `ls` contiene già ['A', 'B', 'D', 'E']
- L'inserimento consiste in 3 passi:
  1. Creare un nuovo nodo per 'C'
  - 2.
  - 3.





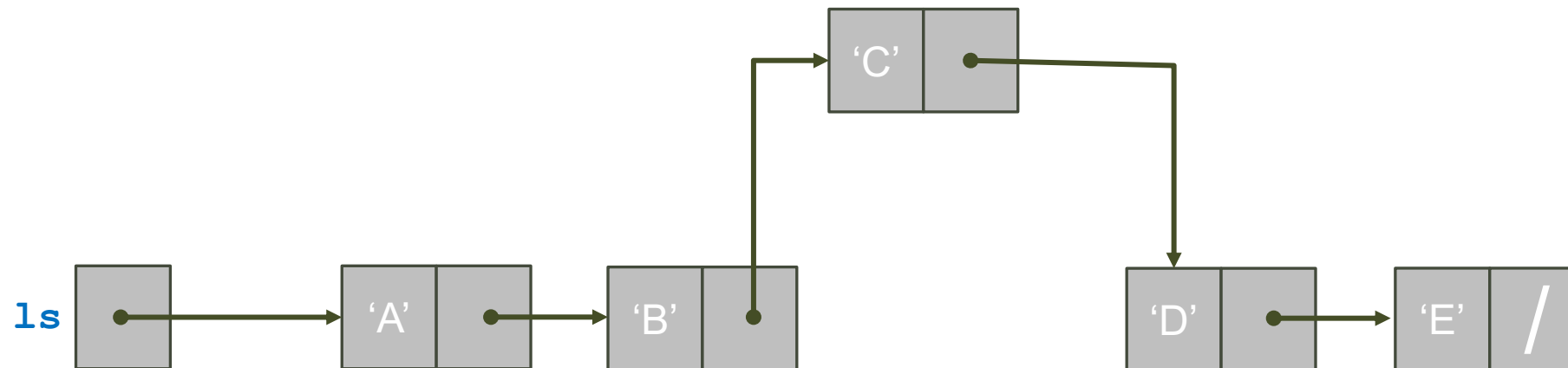
# Inserimento in una Lista Ordinata I

- Supponiamo di voler inserire 'C' in una lista di caratteri `ls`
  - `ls` contiene già ['A', 'B', 'D', 'E']
- L'inserimento consiste in 3 passi:
  1. Creare un nuovo nodo per 'C'
  2. Cercare la posizione giusta di inserzione
  - 3.



# Inserimento in una Lista Ordinata I

- Supponiamo di voler inserire 'C' in una lista di caratteri `ls`
  - `ls` contiene già ['A', 'B', 'D', 'E']
- L'inserimento consiste in 3 passi:
  1. Creare un nuovo nodo per 'C'
  2. Cercare la posizione giusta di inserzione
  3. Inserire il nodo nella lista



# Inserimento in una Lista Ordinata II

## 1. Creazione del nuovo nodo

```
// Inserisce value al primo posto giusto (potrebbero esserci elementi ripetuti)
_Bool insertInSortedCharList(CharList *lsPtr, char value) {
```

...

```
}
```

# Inserimento in una Lista Ordinata II

## 1. Creazione del nuovo nodo

```
// Inserisce value al primo posto giusto (potrebbero esserci elementi ripetuti)
_Bool insertInSortedCharList(CharList *lsPtr, char value) {
    CharList newPtr = malloc(sizeof(CharListNode));
```

...

```
}
```

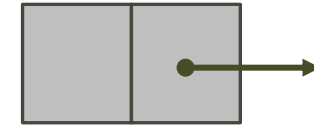
# Inserimento in una Lista Ordinata II

## 1. Creazione del nuovo nodo

```
// Inserisce value al primo posto giusto (potrebbero esserci elementi ripetuti)
_Bool insertInSortedCharList(CharList *lsPtr, char value) {
    CharList newPtr = malloc(sizeof(CharListNode));

    if (newPtr == NULL) {
        return 0; // Se non c'è spazio, usciamo subito
    }

    ...
}
```



# Inserimento in una Lista Ordinata II

## 1. Creazione del nuovo nodo

```
// Inserisce value al primo posto giusto (potrebbero esserci elementi ripetuti)
_Bool insertInSortedCharList(CharList *lsPtr, char value) {
    CharList newPtr = malloc(sizeof(CharListNode));

    if (newPtr == NULL) {
        return 0; // Se non c'è spazio, usciamo subito
    }

    newPtr->data = value; // Salviamo value

    ...
}
```



# Inserimento in una Lista Ordinata II

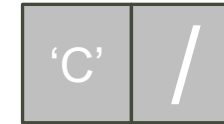
## 1. Creazione del nuovo nodo

```
// Inserisce value al primo posto giusto (potrebbero esserci elementi ripetuti)
_Bool insertInSortedCharList(CharList *lsPtr, char value) {
    CharList newPtr = malloc(sizeof(CharListNode));

    if (newPtr == NULL) {
        return 0; // Se non c'è spazio, usciamo subito
    }

    newPtr->data = value; // Salviamo value
    newPtr->next = NULL; // Il nuovo nodo non è collegato

    ...
}
```



# Inserimento in una Lista Ordinata III

## 2. Cercare la posizione di inserzione

- Scorriamo la lista per trovare il primo elemento con data > value e inseriamo prima

```
...  
// Scorriamo la lista tenendo traccia del predecessore
```

...





# Inserimento in una Lista Ordinata III

## 2. Cercare la posizione di inserzione

- Scorriamo la lista per trovare il primo elemento con data > value e inseriamo prima

```
...
// Scorriamo la lista tenendo traccia del predecessore
CharList previousPtr = NULL;
```

previousPtr



...



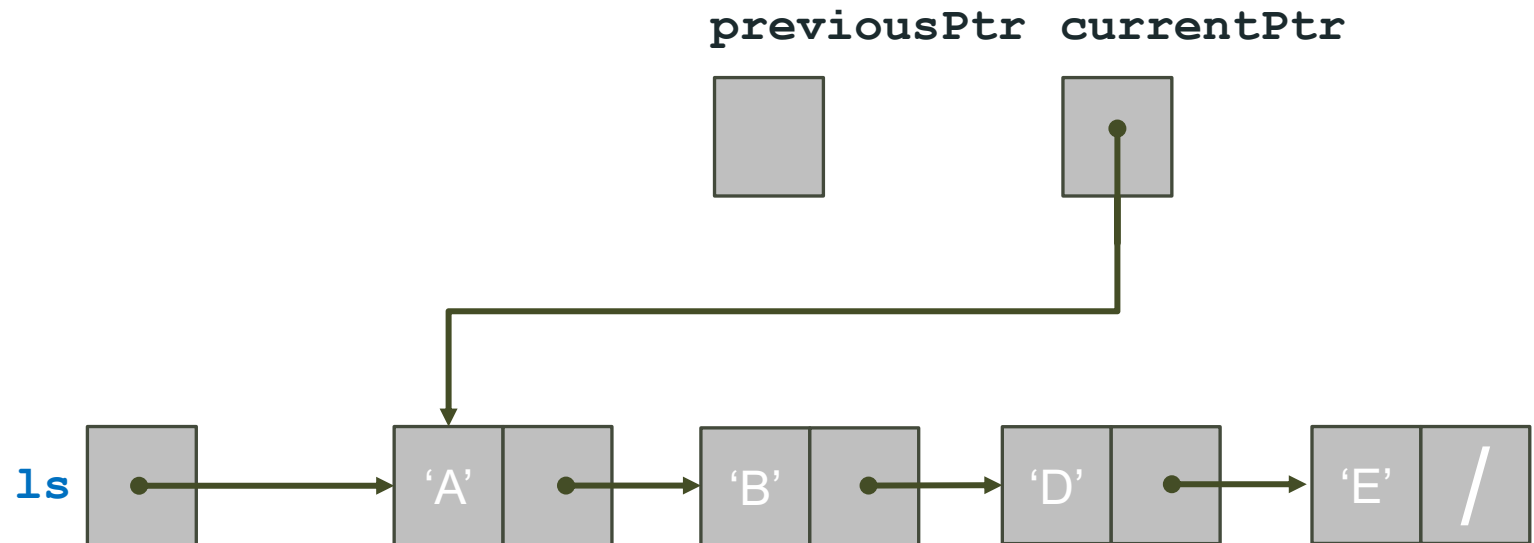
# Inserimento in una Lista Ordinata III



## 2. Cercare la posizione di inserzione

- Scorriamo la lista per trovare il primo elemento con data > value e inseriamo prima

```
...  
// Scorriamo la lista tenendo traccia del predecessore  
CharList previousPtr = NULL;  
CharList currentPtr = *lsPtr;  
...
```



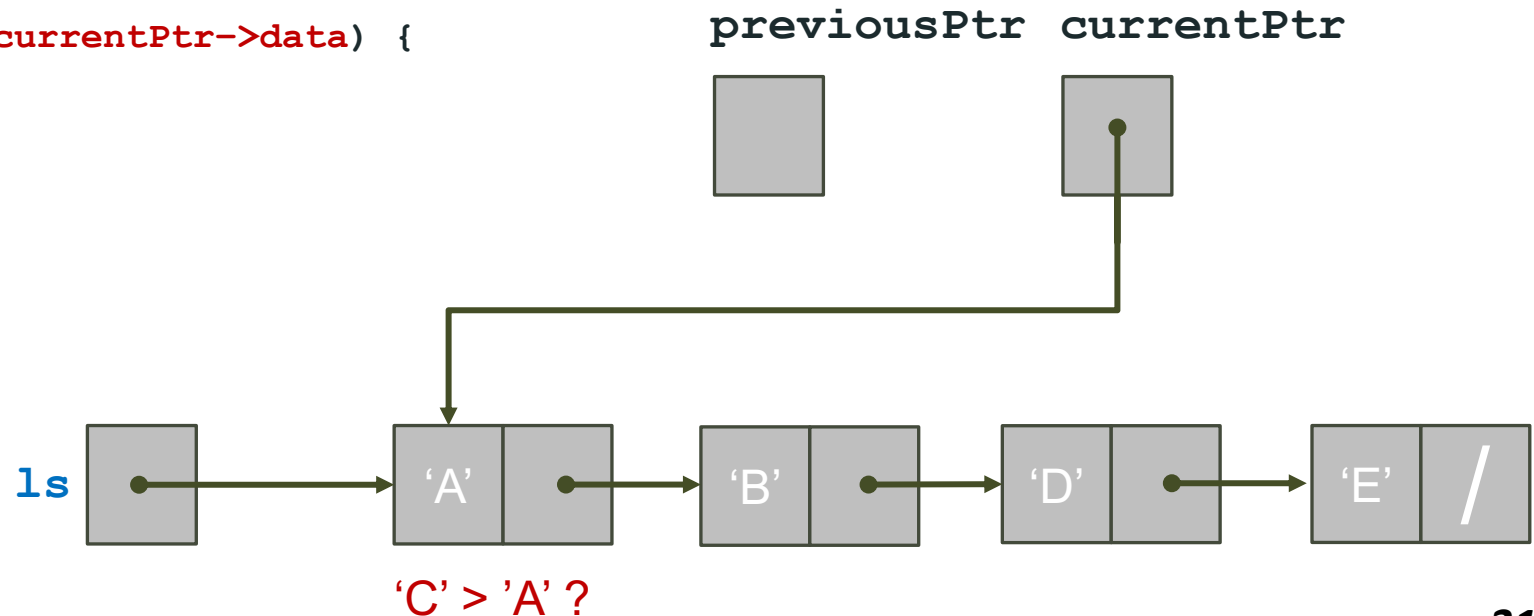
# Inserimento in una Lista Ordinata III



## 2. Cercare la posizione di inserzione

- Scorriamo la lista per trovare il primo elemento con data > value e inseriamo prima

```
...  
// Scorriamo la lista tenendo traccia del predecessore  
CharList previousPtr = NULL;  
CharList currentPtr = *lsPtr;  
  
while (currentPtr != NULL && value > currentPtr->data) {  
  
}  
...
```



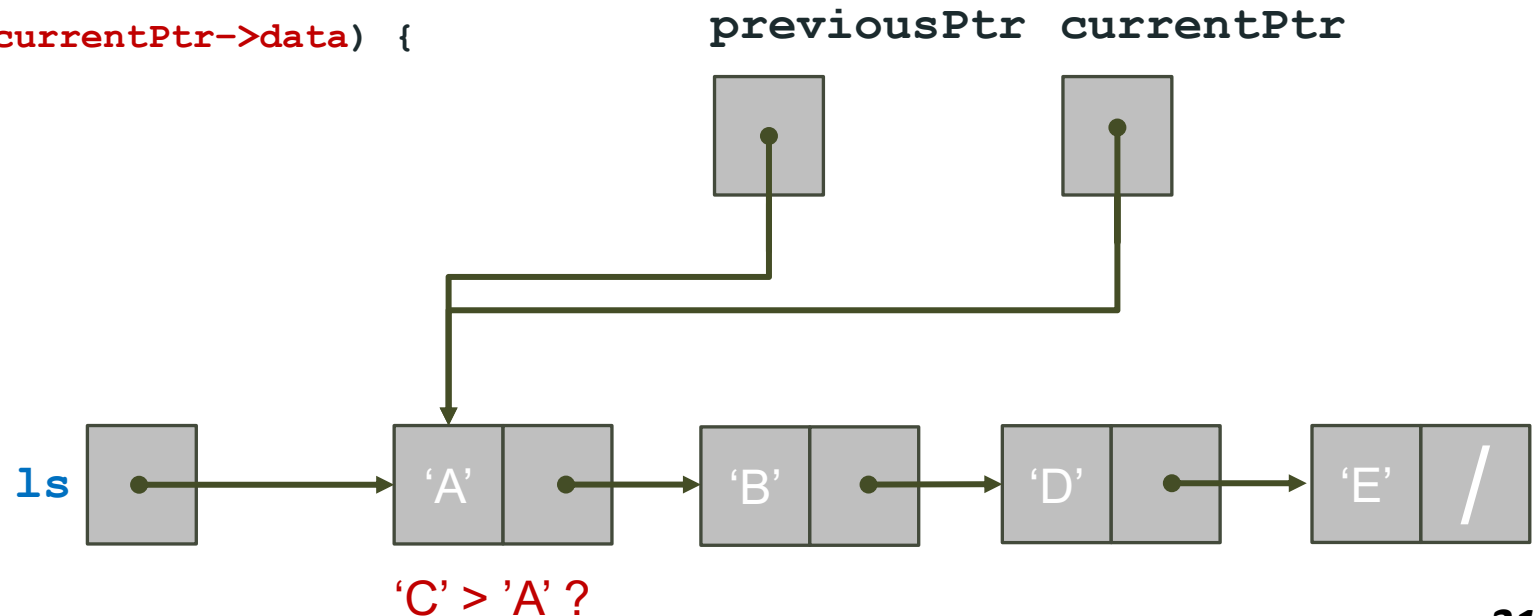
# Inserimento in una Lista Ordinata III



## 2. Cercare la posizione di inserzione

- Scorriamo la lista per trovare il primo elemento con data > value e inseriamo prima

```
...  
// Scorriamo la lista tenendo traccia del predecessore  
CharList previousPtr = NULL;  
CharList currentPtr = *lsPtr;  
  
while (currentPtr != NULL && value > currentPtr->data) {  
    previousPtr = currentPtr;  
}  
...
```



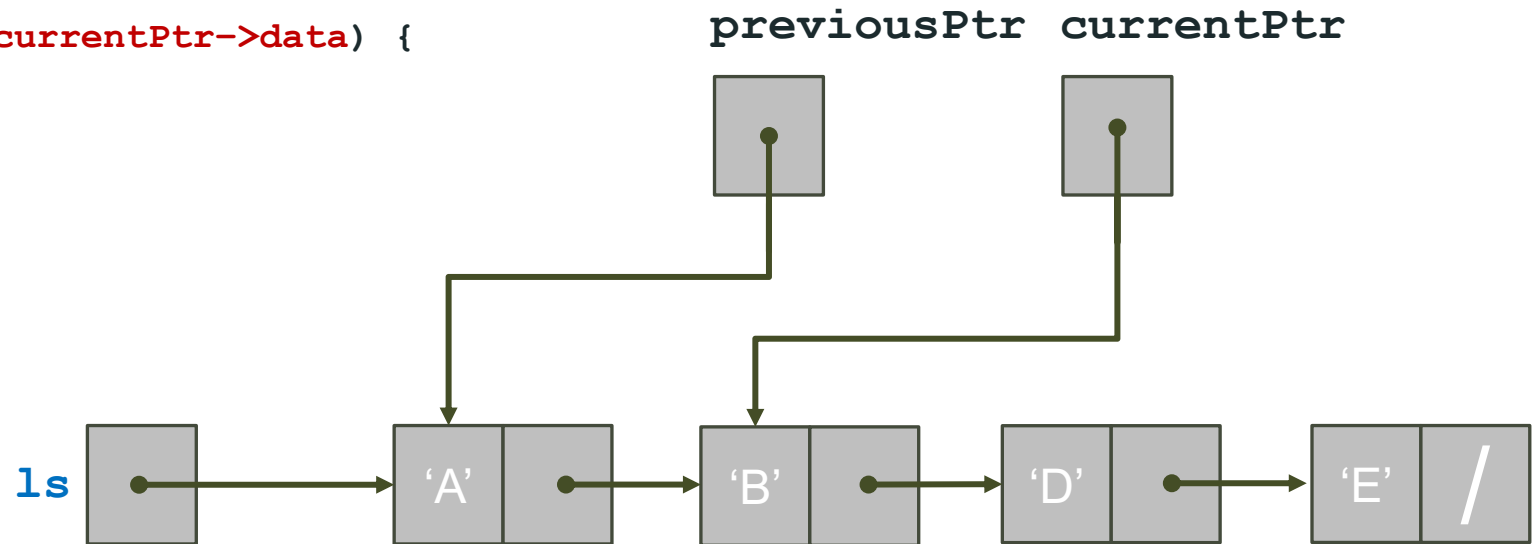
# Inserimento in una Lista Ordinata III



## 2. Cercare la posizione di inserzione

- Scorriamo la lista per trovare il primo elemento con data > value e inseriamo prima

```
...  
// Scorriamo la lista tenendo traccia del predecessore  
CharList previousPtr = NULL;  
CharList currentPtr = *lsPtr;  
  
while (currentPtr != NULL && value > currentPtr->data) {  
    previousPtr = currentPtr;  
    currentPtr = currentPtr->next;  
}  
...
```



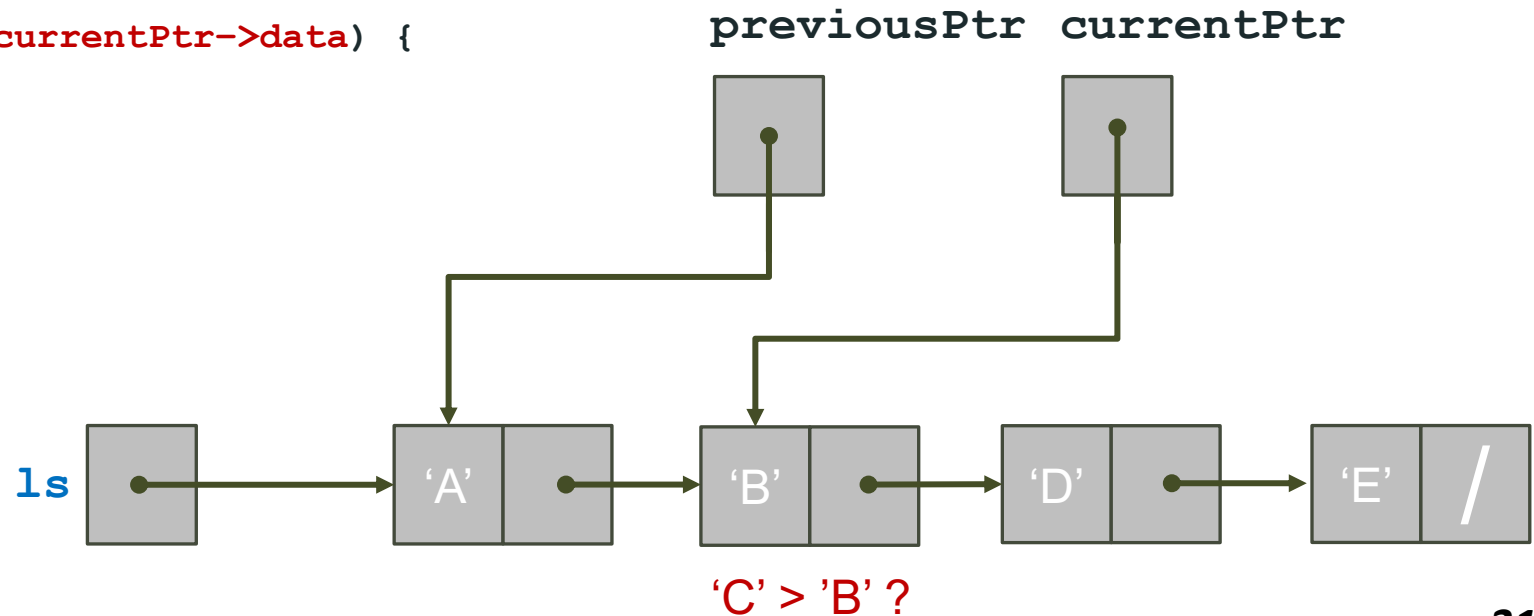
# Inserimento in una Lista Ordinata III



## 2. Cercare la posizione di inserzione

- Scorriamo la lista per trovare il primo elemento con data > value e inseriamo prima

```
...  
// Scorriamo la lista tenendo traccia del predecessore  
CharList previousPtr = NULL;  
CharList currentPtr = *lsPtr;  
  
while (currentPtr != NULL && value > currentPtr->data) {  
    previousPtr = currentPtr;  
    currentPtr = currentPtr->next;  
}  
...
```



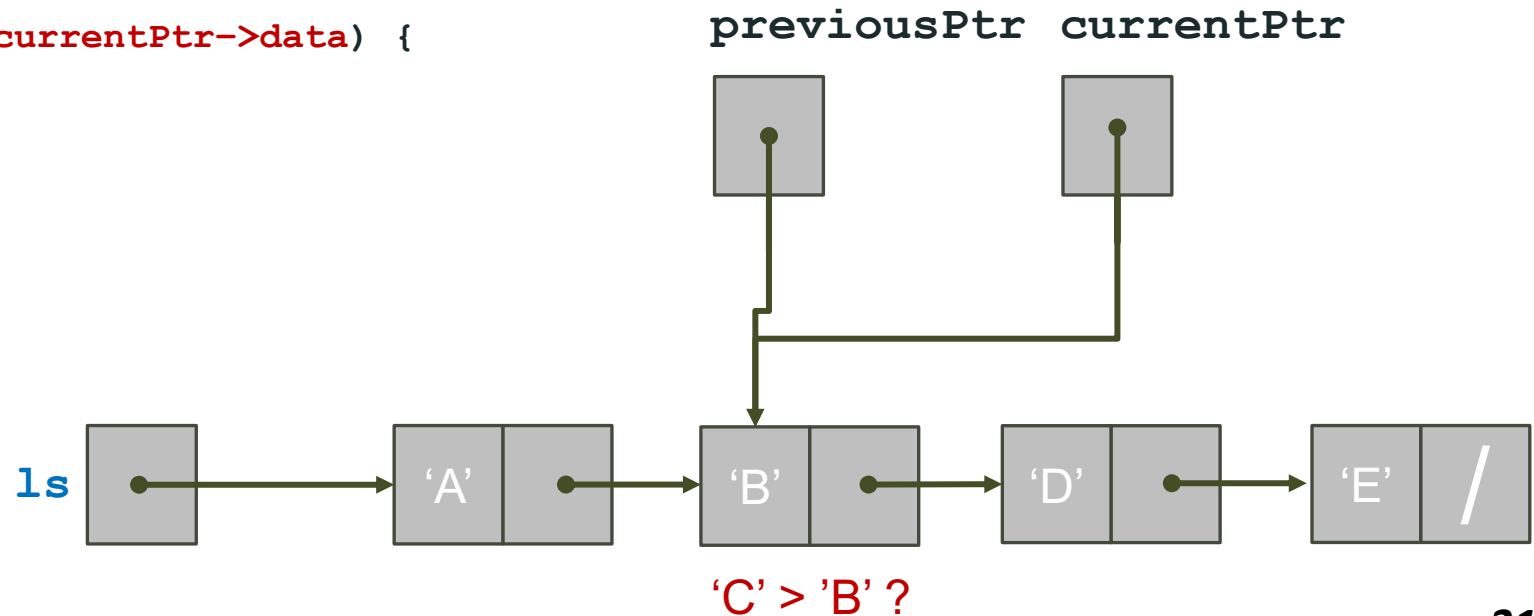
# Inserimento in una Lista Ordinata III



## 2. Cercare la posizione di inserzione

- Scorriamo la lista per trovare il primo elemento con data > value e inseriamo prima

```
...  
// Scorriamo la lista tenendo traccia del predecessore  
CharList previousPtr = NULL;  
CharList currentPtr = *lsPtr;  
  
while (currentPtr != NULL && value > currentPtr->data) {  
    previousPtr = currentPtr;  
    currentPtr = currentPtr->next;  
}  
...
```



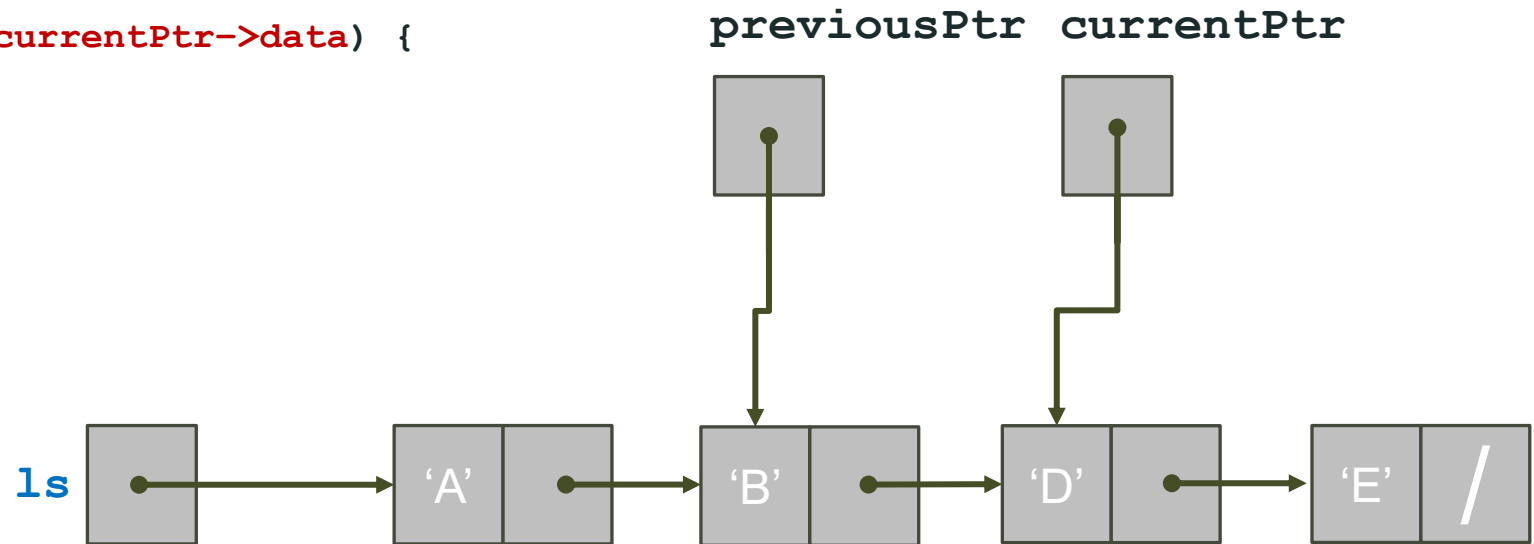
# Inserimento in una Lista Ordinata III



## 2. Cercare la posizione di inserzione

- Scorriamo la lista per trovare il primo elemento con data > value e inseriamo prima

```
...  
// Scorriamo la lista tenendo traccia del predecessore  
CharList previousPtr = NULL;  
CharList currentPtr = *lsPtr;  
  
while (currentPtr != NULL && value > currentPtr->data) {  
    previousPtr = currentPtr;  
    currentPtr = currentPtr->next;  
}  
...
```





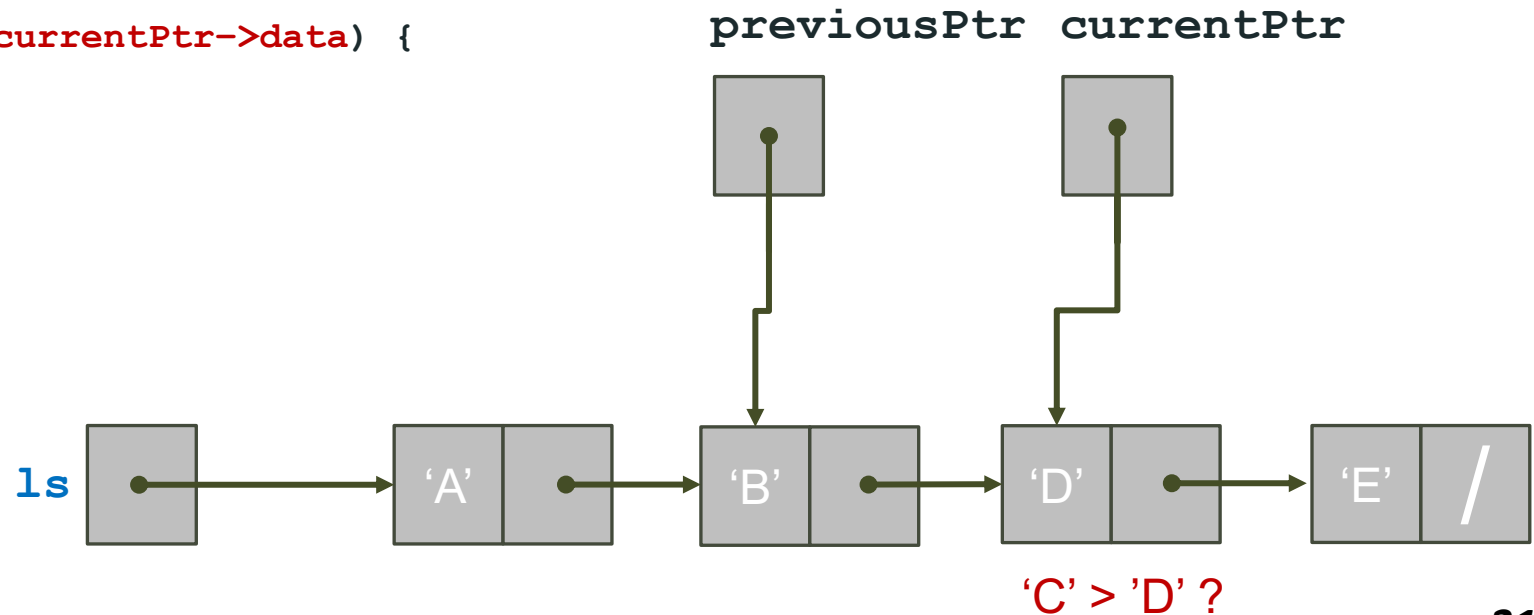
# Inserimento in una Lista Ordinata III



## 2. Cercare la posizione di inserzione

- Scorriamo la lista per trovare il primo elemento con data > value e inseriamo prima

```
...  
// Scorriamo la lista tenendo traccia del predecessore  
CharList previousPtr = NULL;  
CharList currentPtr = *lsPtr;  
  
while (currentPtr != NULL && value > currentPtr->data) {  
    previousPtr = currentPtr;  
    currentPtr = currentPtr->next;  
}  
...
```



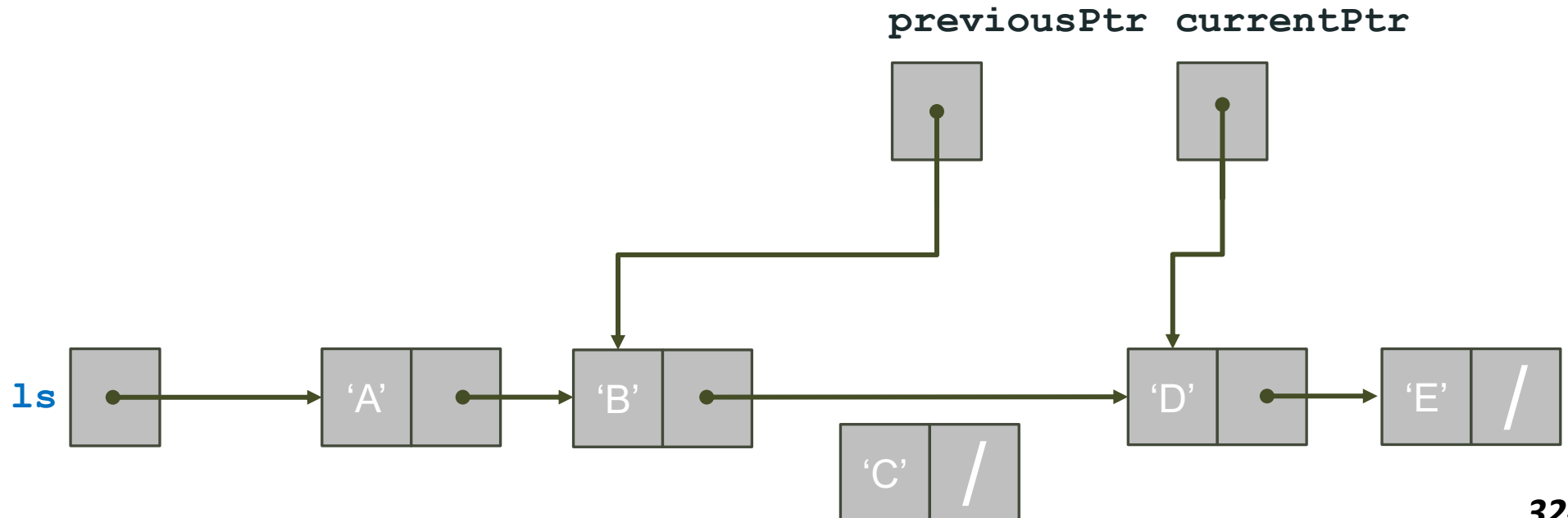
# Inserimento in una Lista Ordinata IV



## 3. Inserire il nodo nella lista

- Dobbiamo inserire il nuovo nodo come successore di **previousPtr** e predecessore di **currentPtr**

```
...  
if (previousPtr != NULL) {  
    // Il nuovo nodo deve essere inserito in tra previousPtr e currentPtr (che potrebbe essere NULL)  
}  
...
```



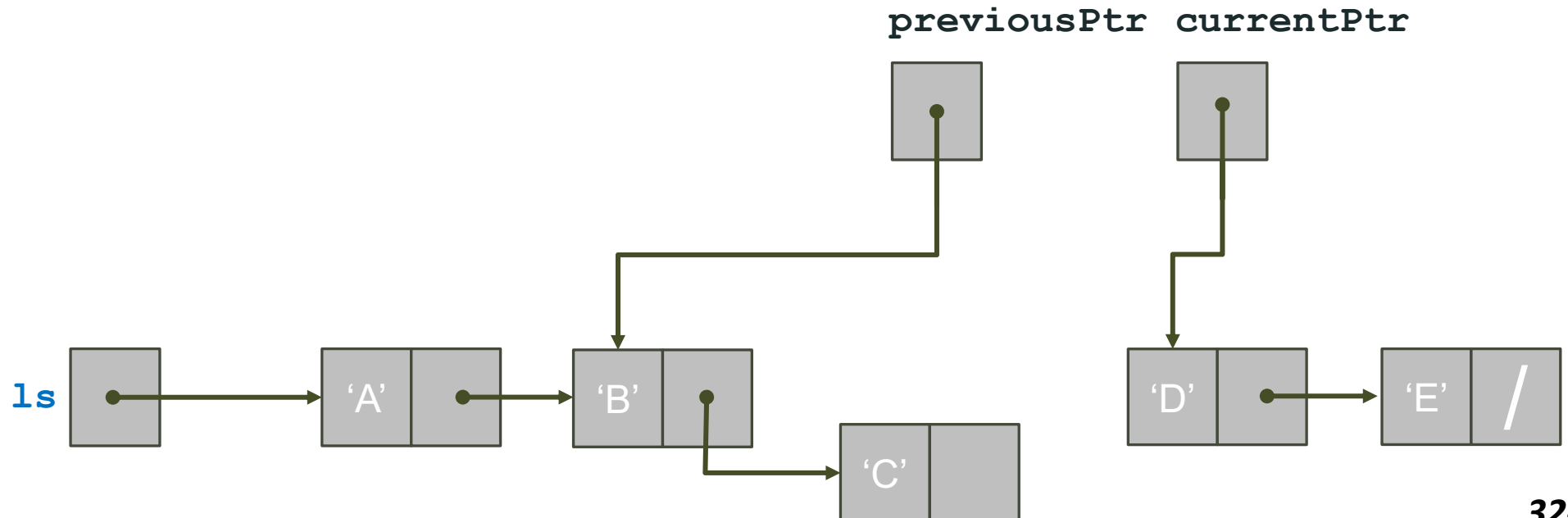
# Inserimento in una Lista Ordinata IV



## 3. Inserire il nodo nella lista

- Dobbiamo inserire il nuovo nodo come successore di **previousPtr** e predecessore di **currentPtr**

```
...  
if (previousPtr != NULL) {  
    // Il nuovo nodo deve essere inserito in tra previousPtr e currentPtr (che potrebbe essere NULL)  
    previousPtr->next = newPtr;  
}  
...
```



# Inserimento in una Lista Ordinata IV

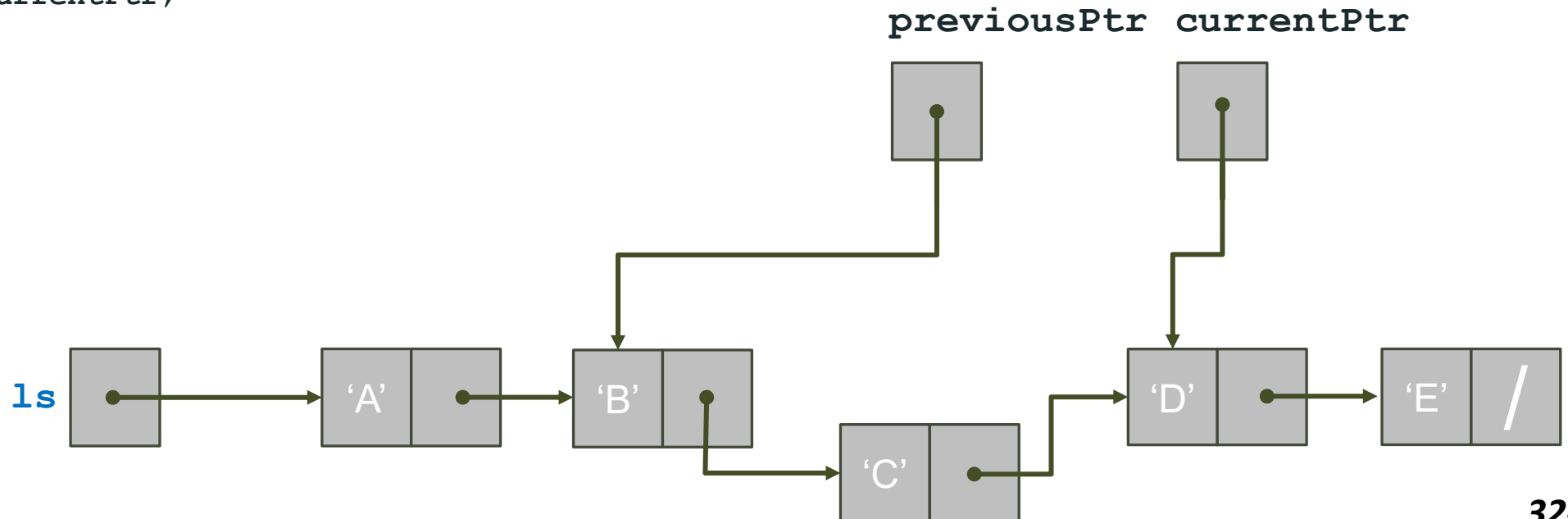


## 3. Inserire il nodo nella lista

- Dobbiamo inserire il nuovo nodo come successore di **previousPtr** e predecessore di **currentPtr**

```
...  
if (previousPtr != NULL) {  
    // Il nuovo nodo deve essere inserito in tra previousPtr e currentPtr (che potrebbe essere NULL)  
    previousPtr->next = newPtr;  
    newPtr->next = currentPtr;  
}
```

...



# Inserimento in una Lista Ordinata IV

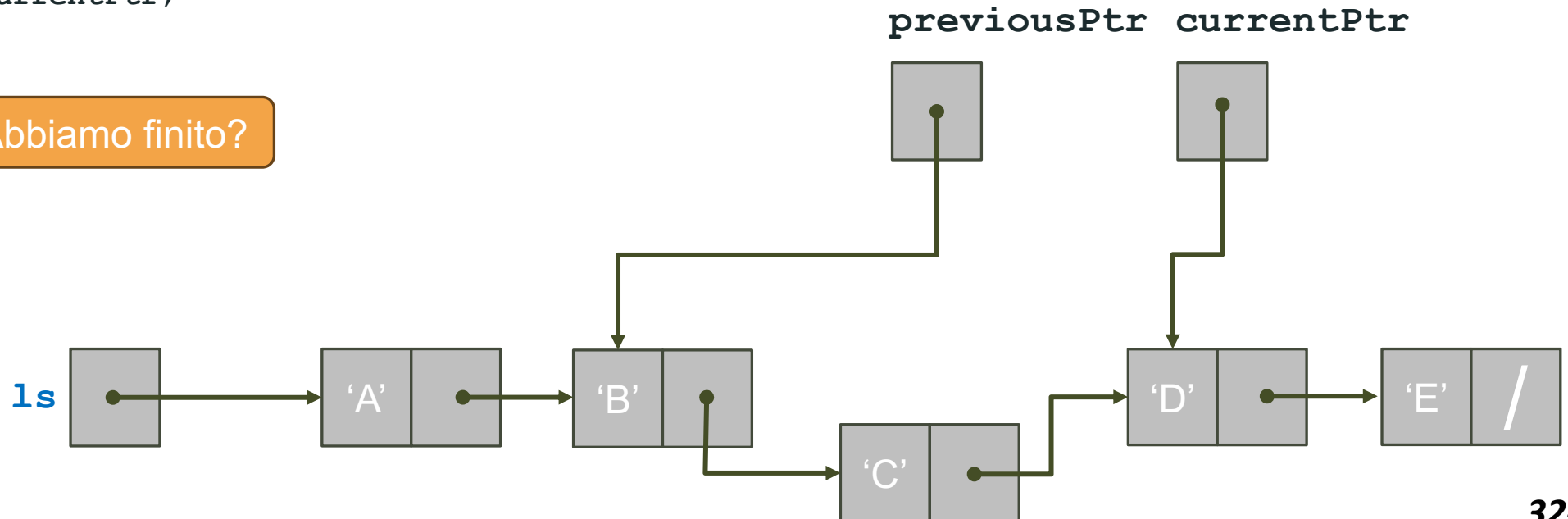


## 3. Inserire il nodo nella lista

- Dobbiamo inserire il nuovo nodo come successore di **previousPtr** e predecessore di **currentPtr**

```
...  
if (previousPtr != NULL) {  
    // Il nuovo nodo deve essere inserito in tra previousPtr e currentPtr (che potrebbe essere NULL)  
    previousPtr->next = newPtr;  
    newPtr->next = currentPtr;  
}
```

Abbiamo finito?



# Inserimento in una Lista Ordinata IV

## 3. Inserire il nodo nella lista

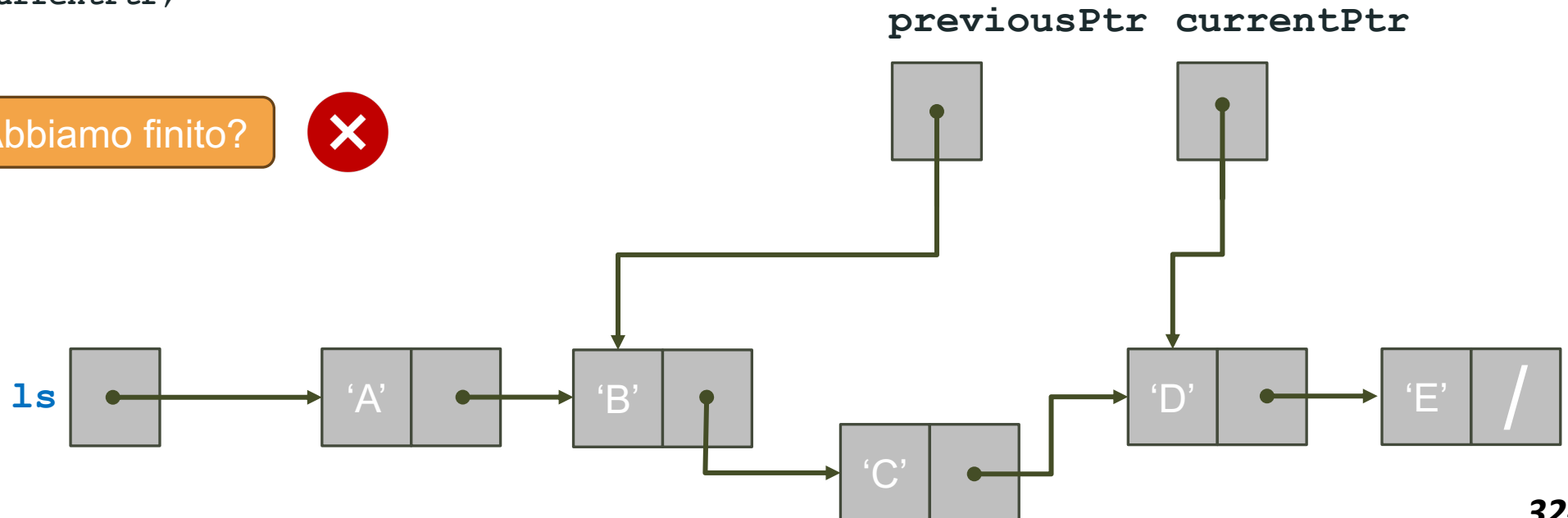
- Dobbiamo inserire il nuovo nodo come successore di **previousPtr** e predecessore di **currentPtr**

```

...
if (previousPtr != NULL) {
    // Il nuovo nodo deve essere inserito in tra previousPtr e currentPtr (che potrebbe essere NULL)
    previousPtr->next = newPtr;
    newPtr->next = currentPtr;
}
else {
    ...
}
...

```

Abbiamo finito? 



# Inserimento in una Lista Ordinata V

## 3. Inserire il nodo nella lista

- `previousPtr == NULL` significa che il nuovo elemento è il primo elemento

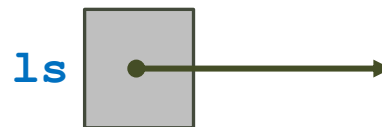
```

...
if (previousPtr != NULL) {
    ...
} else {

}
return 1;

```

`previousPtr`



# Inserimento in una Lista Ordinata V

## 3. Inserire il nodo nella lista

- `previousPtr == NULL` significa che il nuovo elemento è il primo elemento

```
...
if (previousPtr != NULL) {
    ...
} else {
    // Nuovo elemento inserito in testa

```

**previousPtr**



**ls**





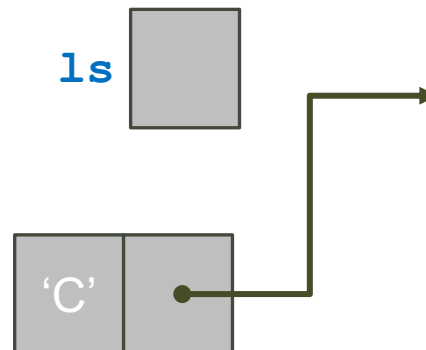
# Inserimento in una Lista Ordinata V

## 3. Inserire il nodo nella lista

- `previousPtr == NULL` significa che il nuovo elemento è il primo elemento

```
...
if (previousPtr != NULL) {
    ...
} else {
    // Nuovo elemento inserito in testa
    newPtr->next = *lsPtr;
}
return 1;
}
```

**previousPtr**



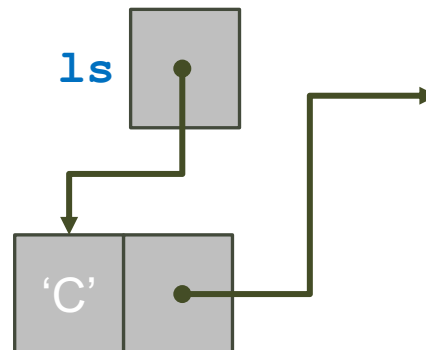
# Inserimento in una Lista Ordinata V

## 3. Inserire il nodo nella lista

- `previousPtr == NULL` significa che il nuovo elemento è il primo elemento

```
...
if (previousPtr != NULL) {
    ...
} else {
    // Nuovo elemento inserito in testa
    newPtr->next = *lsPtr;
    *lsPtr = newPtr;
}
return 1;
}
```

`previousPtr`



# L'Ordine è Importante

- Inserimento in testa

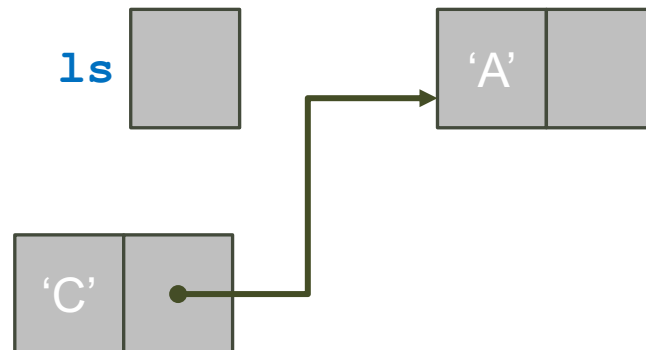
```
// Nuovo elemento inserito in testa
```



# L'Ordine è Importante

- Inserimento in testa

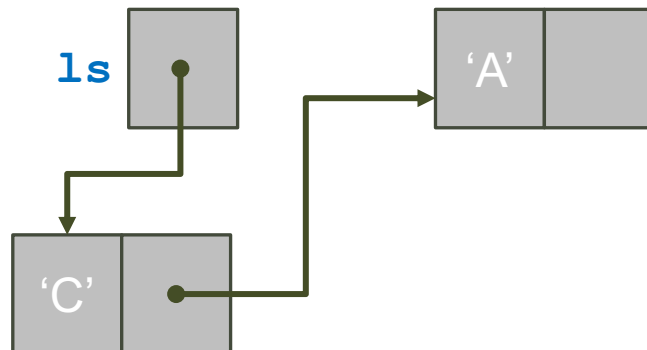
```
// Nuovo elemento inserito in testa  
newPtr->next = *lsPtr;
```



# L'Ordine è Importante

- Inserimento in testa

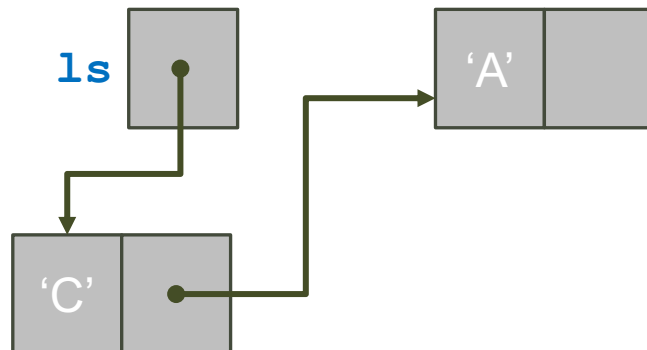
```
// Nuovo elemento inserito in testa  
newPtr->next = *lsPtr;  
*lsPtr = newPtr;
```



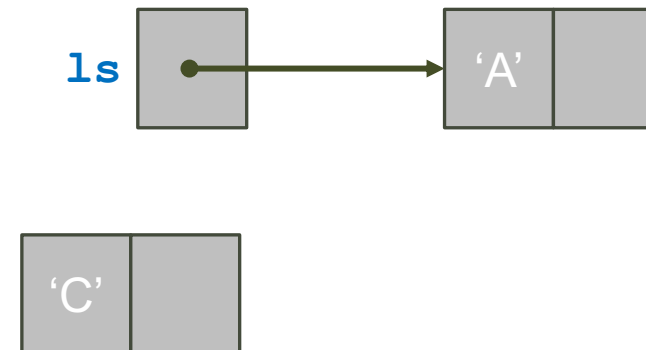
# L'Ordine è Importante

- Inserimento in testa

```
// Nuovo elemento inserito in testa  
newPtr->next = *lsPtr;  
*lsPtr = newPtr;
```



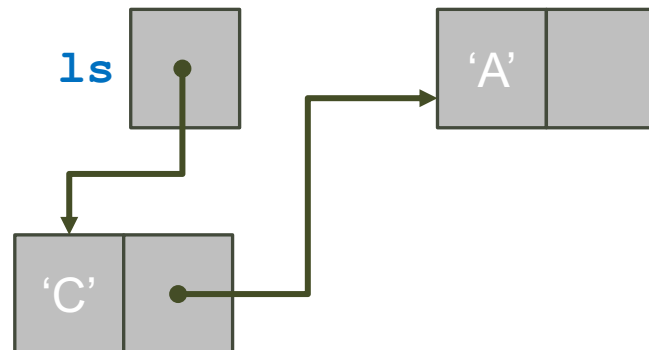
```
// Nuovo elemento inserito in testa
```



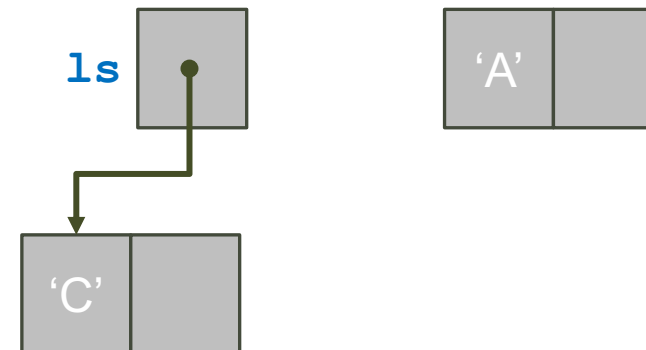
# L'Ordine è Importante

- Inserimento in testa

```
// Nuovo elemento inserito in testa  
newPtr->next = *lsPtr;  
*lsPtr = newPtr;
```



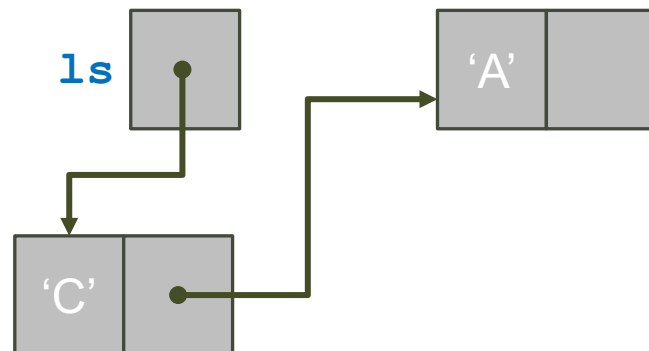
```
// Nuovo elemento inserito in testa  
*lsPtr = newPtr;
```



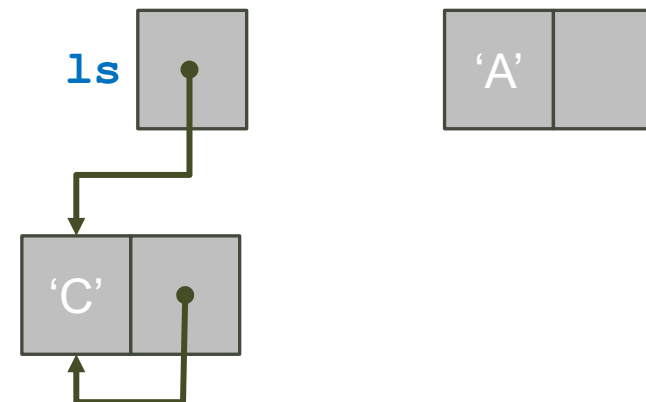
# L'Ordine è Importante

- Inserimento in testa

```
// Nuovo elemento inserito in testa  
newPtr->next = *lsPtr;  
*lsPtr = newPtr;
```



```
// Nuovo elemento inserito in testa  
*lsPtr = newPtr;  
newPtr->next = *lsPtr;
```





# Inserimento Prima di Nodo Dato I



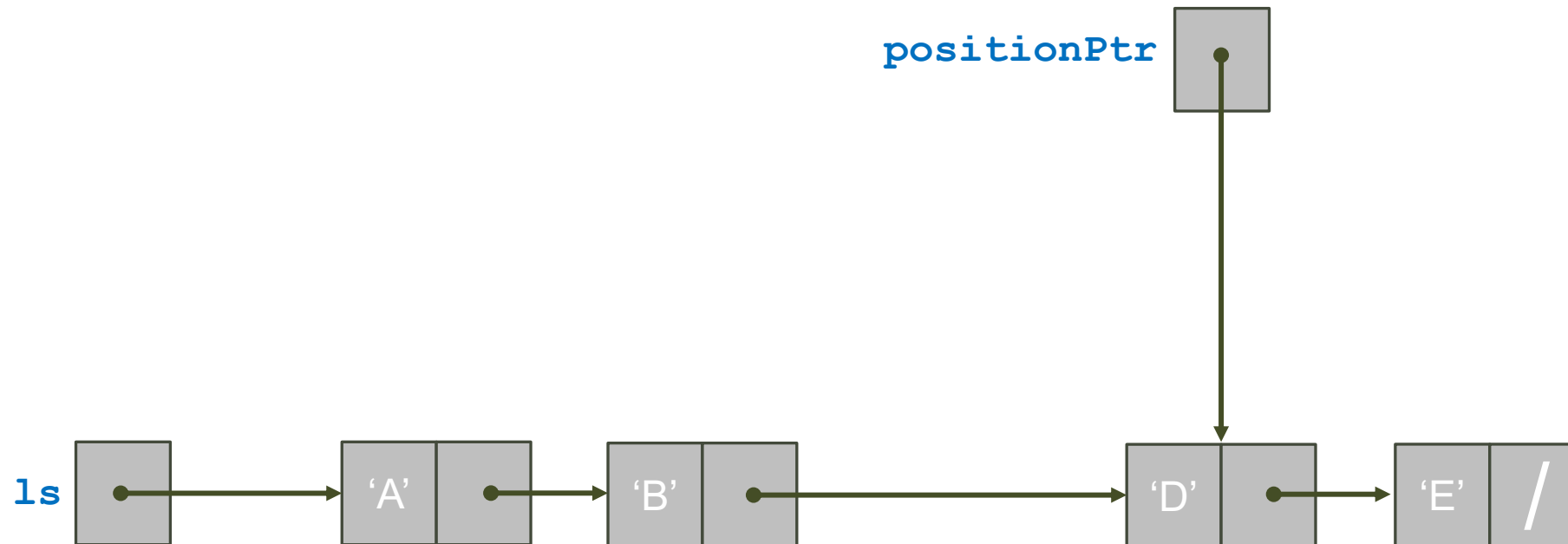
- Supponiamo di voler inserire 'C' in una lista di caratteri prima del nodo `positionPtr`
  - Se `positionPtr` è NULL: inserimento in ultima posizione.

•

1.

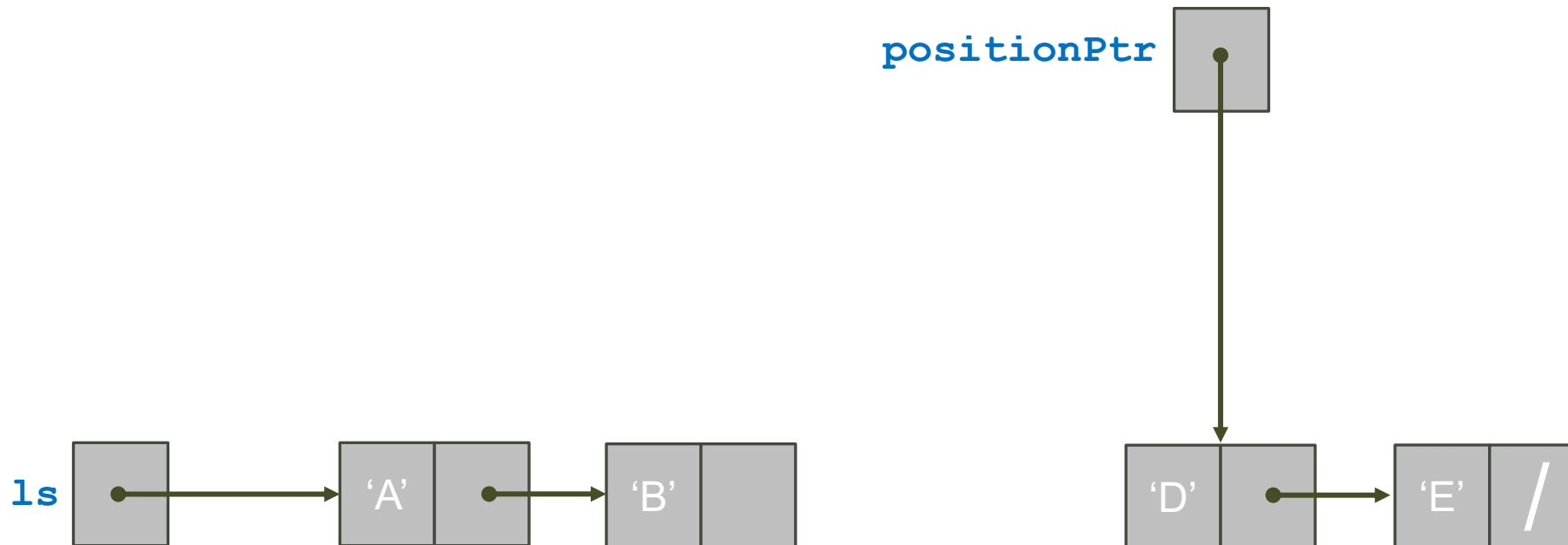
2.

3.



# Inserimento Prima di Nodo Dato I

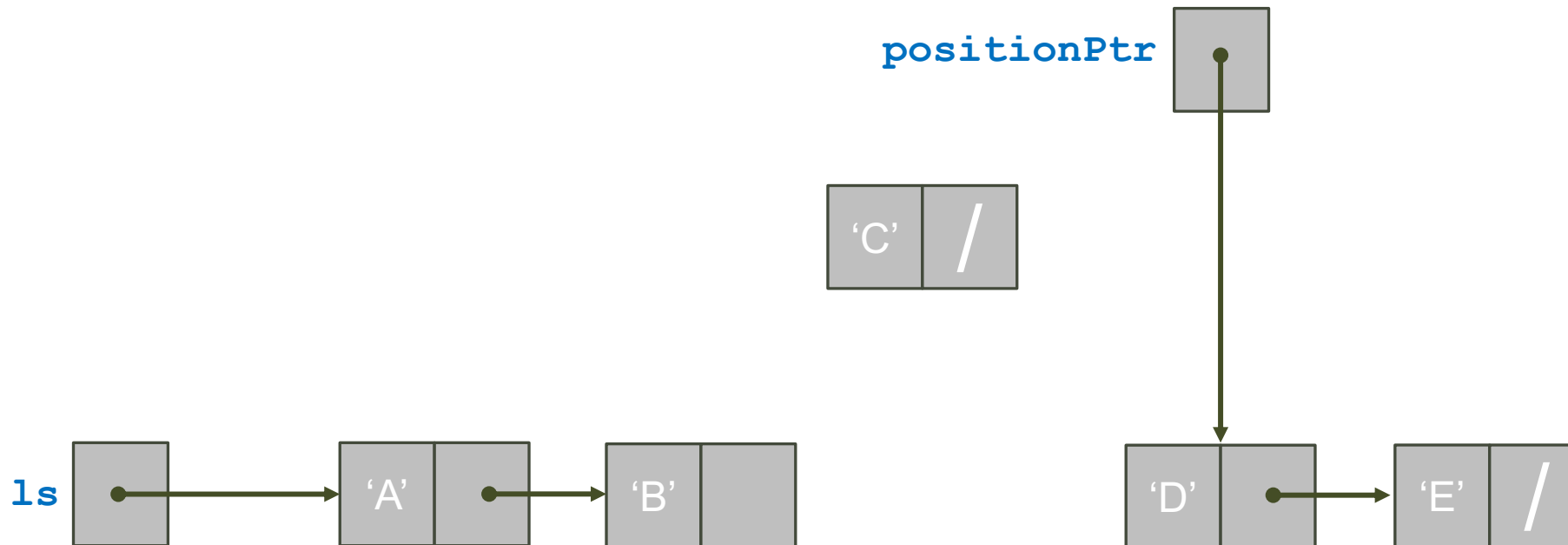
- Supponiamo di voler inserire 'C' in una lista di caratteri prima del nodo `positionPtr`
  - Se `positionPtr` è NULL: inserimento in ultima posizione.
- Cambia il passo 2:
  - 1.
  - 2.
  - 3.



# Inserimento Prima di Nodo Dato I



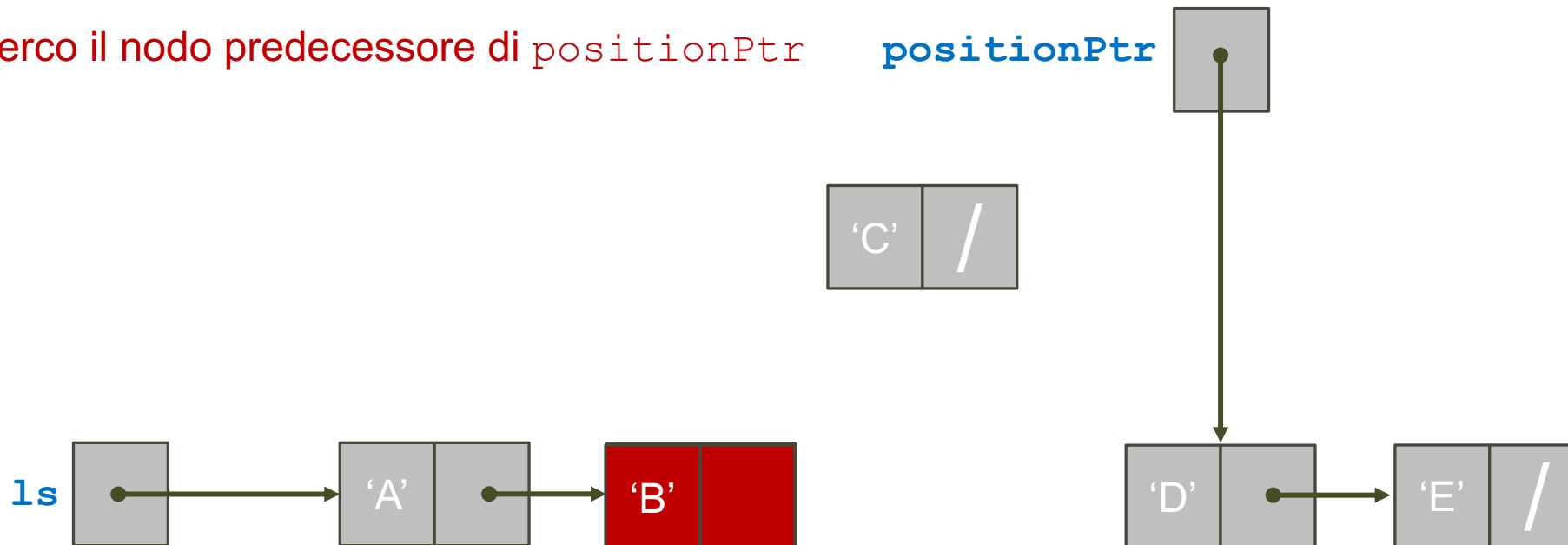
- Supponiamo di voler inserire 'C' in una lista di caratteri prima del nodo `positionPtr`
  - Se `positionPtr` è NULL: inserimento in ultima posizione.
- Cambia il passo 2:
  1. Creare un nuovo nodo per 'C'
  - 2.
  - 3.



# Inserimento Prima di Nodo Dato I

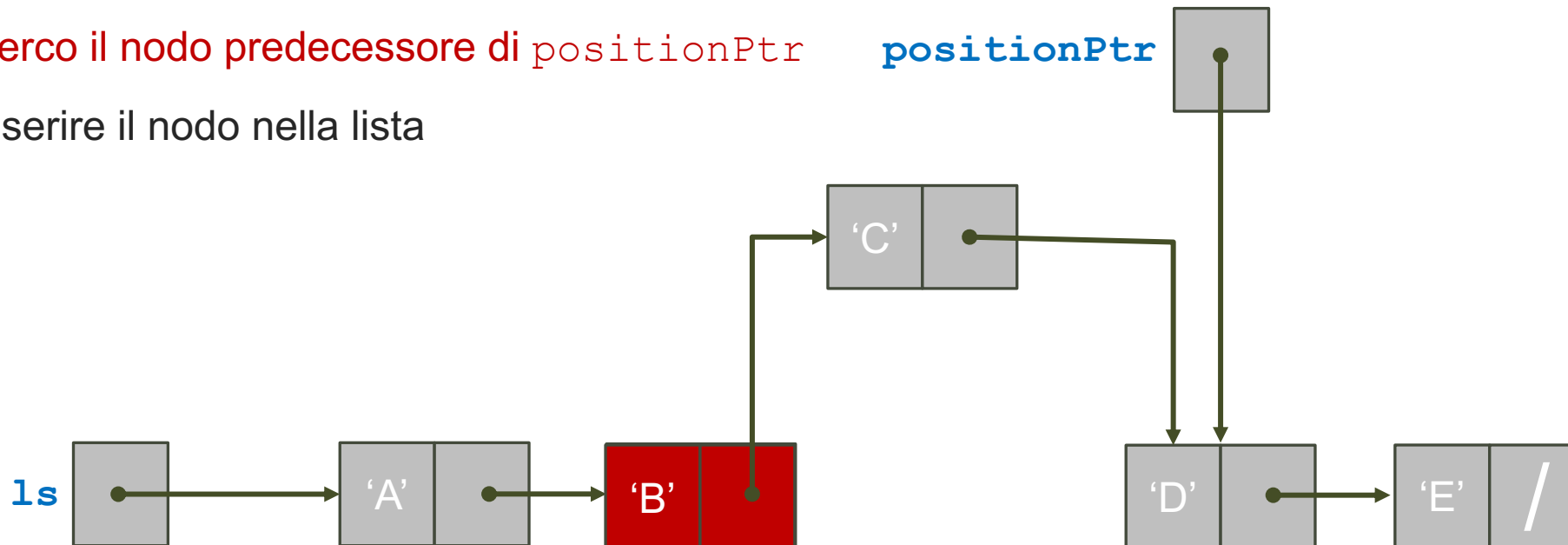


- Supponiamo di voler inserire 'C' in una lista di caratteri prima del nodo `positionPtr`
  - Se `positionPtr` è NULL: inserimento in ultima posizione.
- Cambia il passo 2:
  1. Creare un nuovo nodo per 'C'
  2. Cerco il nodo predecessore di `positionPtr`
  - 3.



# Inserimento Prima di Nodo Dato I

- Supponiamo di voler inserire 'C' in una lista di caratteri prima del nodo `positionPtr`
  - Se `positionPtr` è NULL: inserimento in ultima posizione.
- Cambia il passo 2:
  1. Creare un nuovo nodo per 'C'
  2. **Cerco il nodo predecessore di `positionPtr`**
  3. Inserire il nodo nella lista



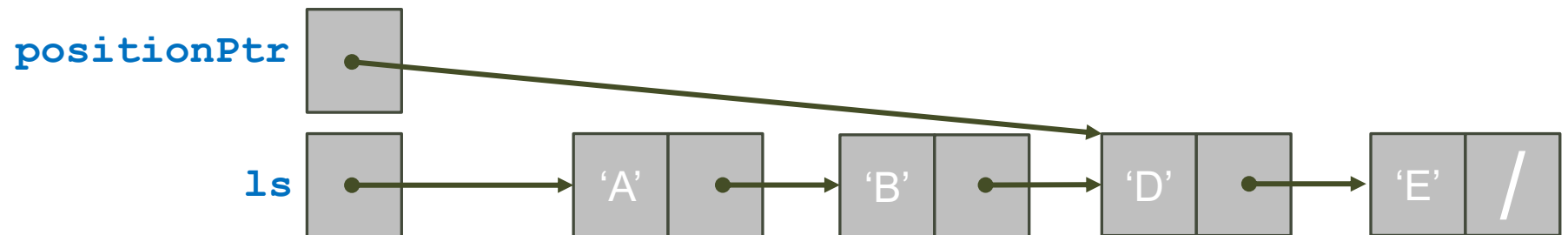
# Inserimento Prima di Nodo Dato II

## 2. Cercare la posizione di inserzione

- Scorriamo la lista per trovare l'elemento prima di `positionPtr`

```
_Bool insertBefore(List *lsPtr, T e, List positionPtr) {
    ...
    // Scorriamo la lista tenendo traccia del predecessore
```

...

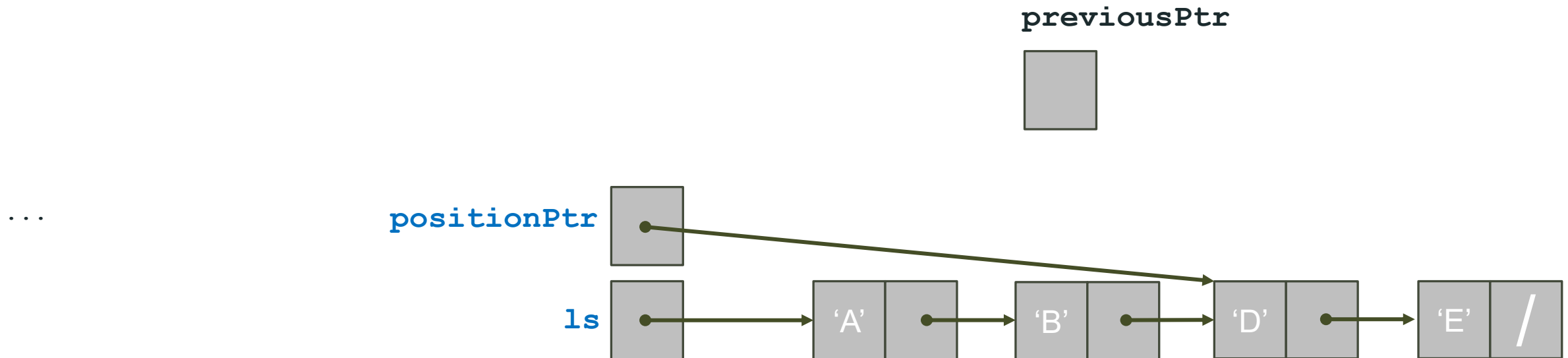


# Inserimento Prima di Nodo Dato II

## 2. Cercare la posizione di inserzione

- Scorriamo la lista per trovare l'elemento prima di `positionPtr`

```
_Bool insertBefore(List *lsPtr, T e, List positionPtr) {
    ...
    // Scorriamo la lista tenendo traccia del predecessore
    List previousPtr = NULL;
```

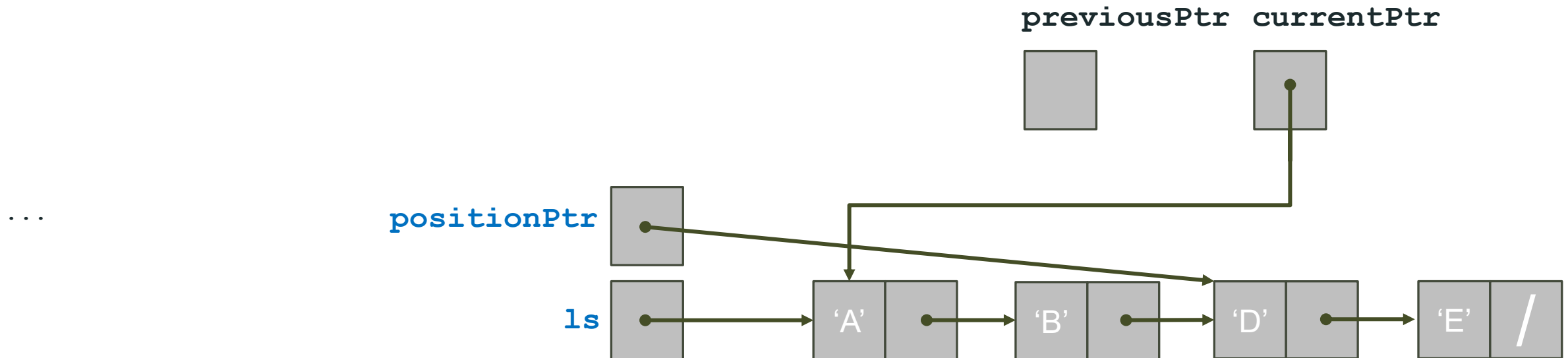


# Inserimento Prima di Nodo Dato II

## 2. Cercare la posizione di inserzione

- Scorriamo la lista per trovare l'elemento prima di `positionPtr`

```
_Bool insertBefore(List *lsPtr, T e, List positionPtr) {
    ...
    // Scorriamo la lista tenendo traccia del predecessore
    List previousPtr = NULL;
    List currentPtr = *lsPtr;
```





# Inserimento Prima di Nodo Dato II

## 2. Cercare la posizione di inserzione

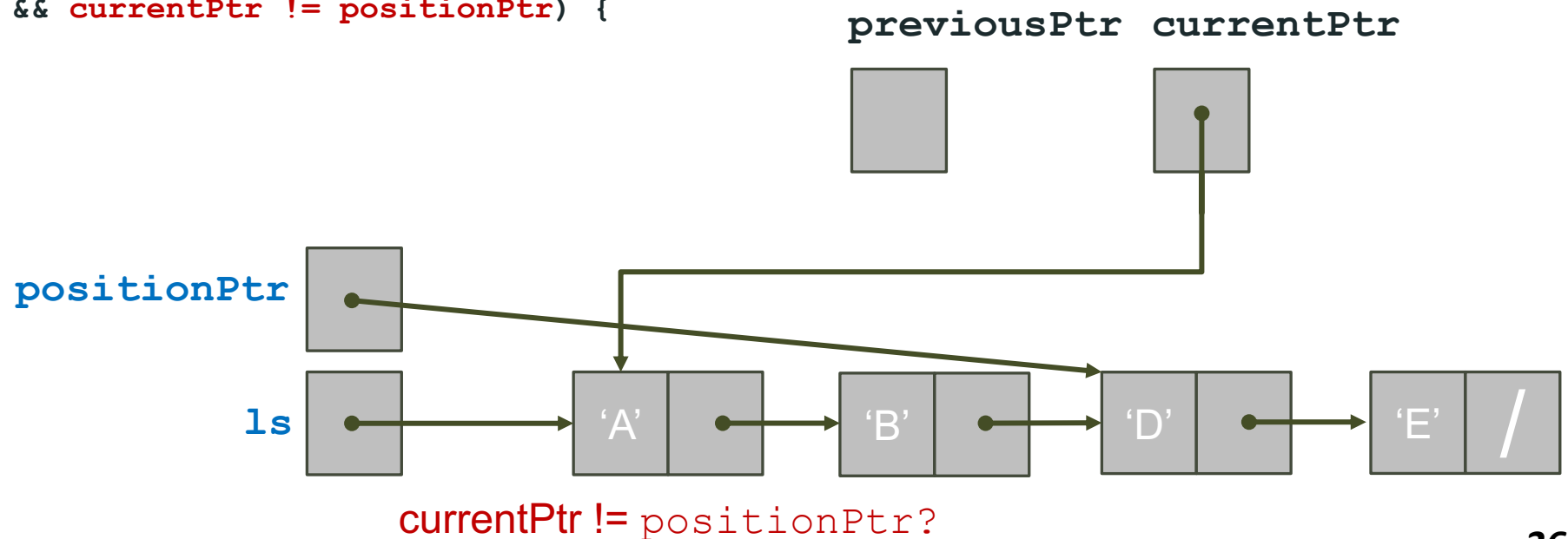
- Scorriamo la lista per trovare l'elemento prima di `positionPtr`

```
_Bool insertBefore(List *lsPtr, T e, List positionPtr) {
    ...
    // Scorriamo la lista tenendo traccia del predecessore
    List previousPtr = NULL;
    List currentPtr = *lsPtr;

    while (currentPtr != NULL && currentPtr != positionPtr) {

```

...



# Inserimento Prima di Nodo Dato II

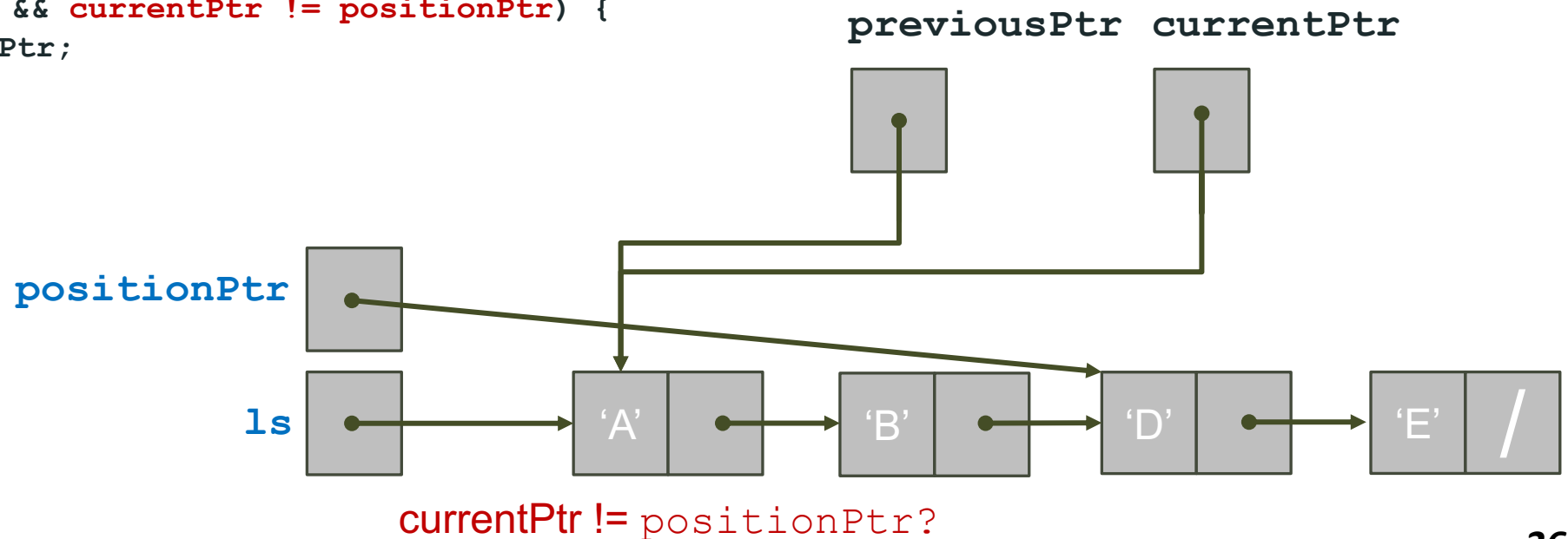
## 2. Cercare la posizione di inserzione

- Scorriamo la lista per trovare l'elemento prima di `positionPtr`

```
_Bool insertBefore(List *lsPtr, T e, List positionPtr) {
    ...
    // Scorriamo la lista tenendo traccia del predecessore
    List previousPtr = NULL;
    List currentPtr = *lsPtr;

    while (currentPtr != NULL && currentPtr != positionPtr) {
        previousPtr = currentPtr;
    }
}
```

...



# Inserimento Prima di Nodo Dato II

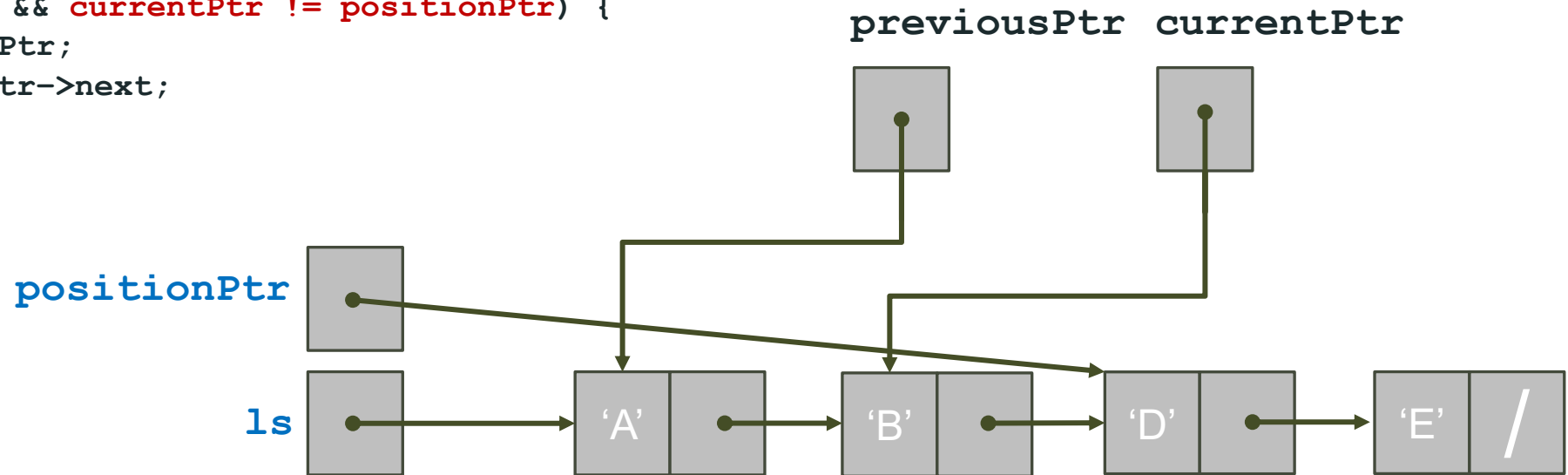
## 2. Cercare la posizione di inserzione

- Scorriamo la lista per trovare l'elemento prima di `positionPtr`

```
_Bool insertBefore(List *lsPtr, T e, List positionPtr) {
    ...
    // Scorriamo la lista tenendo traccia del predecessore
    List previousPtr = NULL;
    List currentPtr = *lsPtr;

    while (currentPtr != NULL && currentPtr != positionPtr) {
        previousPtr = currentPtr;
        currentPtr = currentPtr->next;
    }
}
```

...



# Inserimento Prima di Nodo Dato II

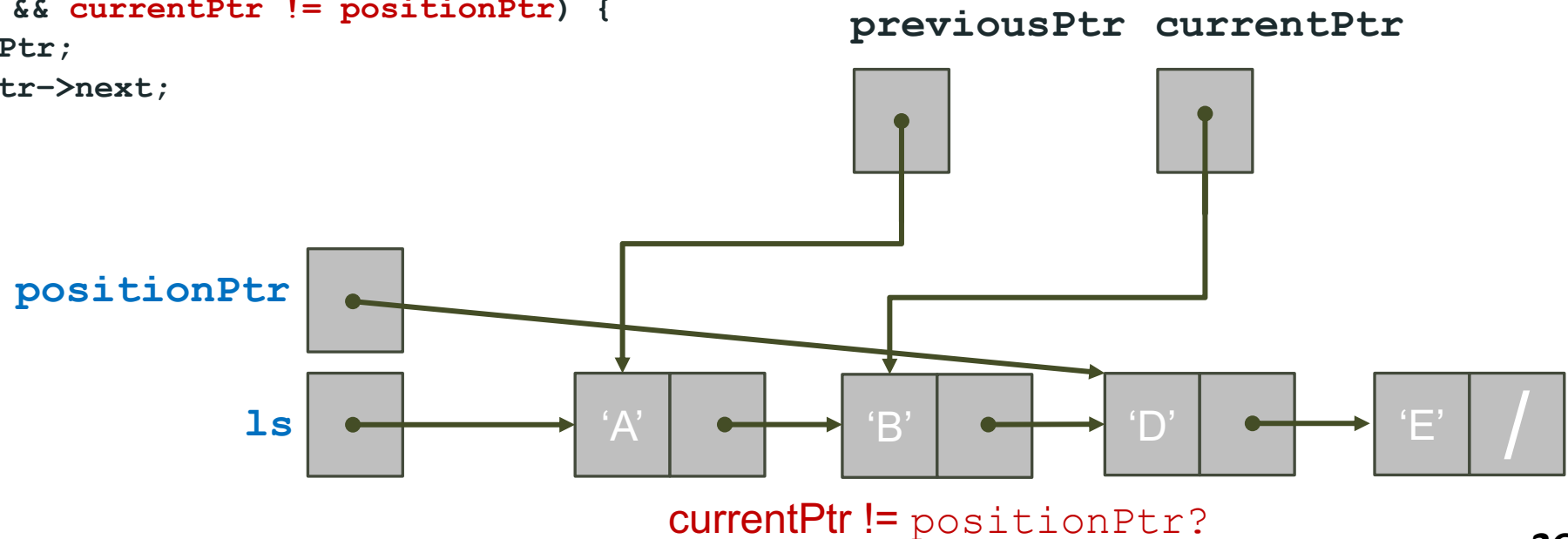
## 2. Cercare la posizione di inserzione

- Scorriamo la lista per trovare l'elemento prima di `positionPtr`

```
_Bool insertBefore(List *lsPtr, T e, List positionPtr) {
    ...
    // Scorriamo la lista tenendo traccia del predecessore
    List previousPtr = NULL;
    List currentPtr = *lsPtr;

    while (currentPtr != NULL && currentPtr != positionPtr) {
        previousPtr = currentPtr;
        currentPtr = currentPtr->next;
    }
}
```

...



# Inserimento Prima di Nodo Dato II

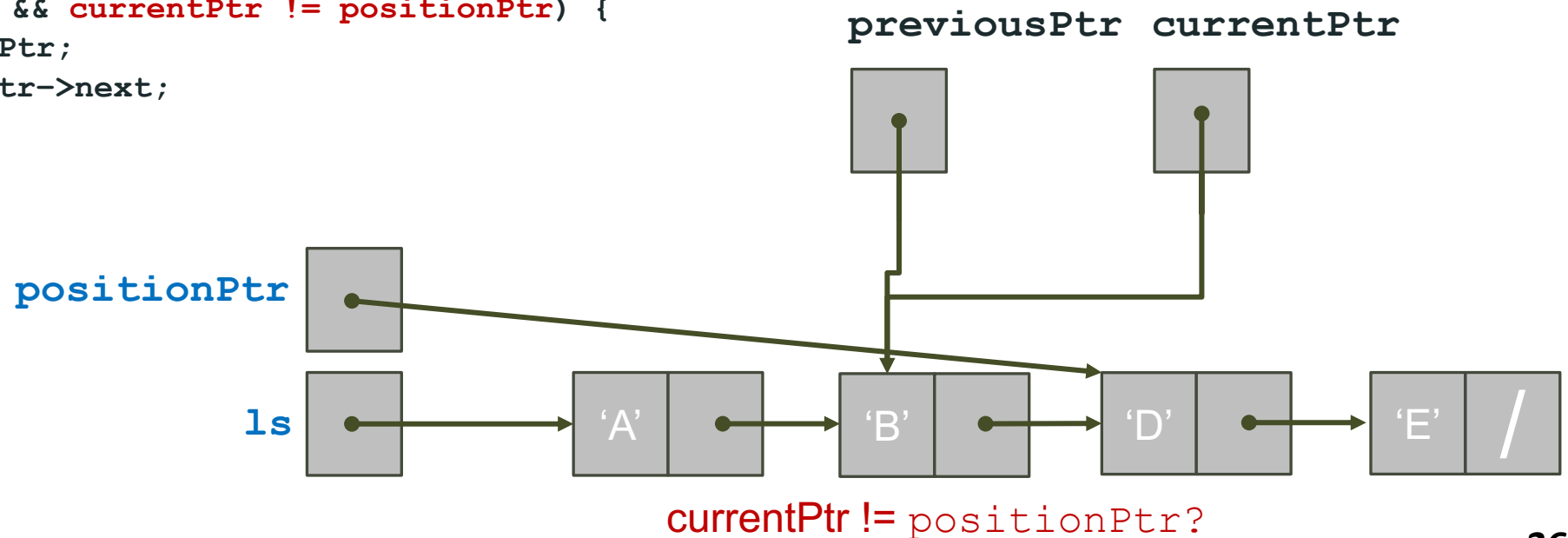
## 2. Cercare la posizione di inserzione

- Scorriamo la lista per trovare l'elemento prima di `positionPtr`

```
_Bool insertBefore(List *lsPtr, T e, List positionPtr) {
    ...
    // Scorriamo la lista tenendo traccia del predecessore
    List previousPtr = NULL;
    List currentPtr = *lsPtr;

    while (currentPtr != NULL && currentPtr != positionPtr) {
        previousPtr = currentPtr;
        currentPtr = currentPtr->next;
    }
}
```

...



# Inserimento Prima di Nodo Dato II

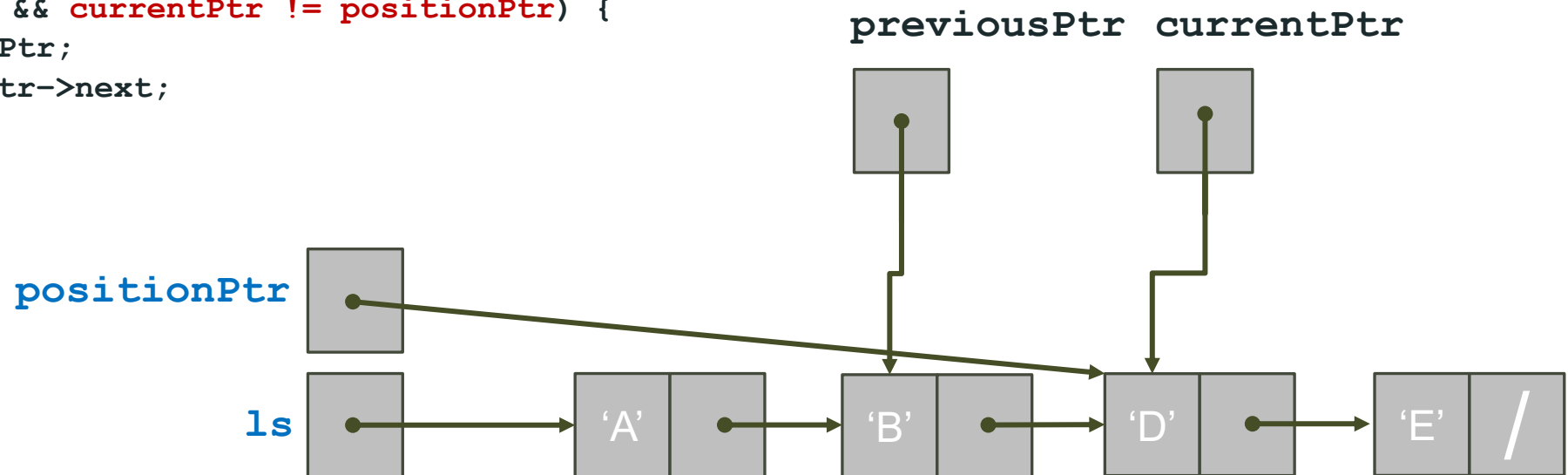
## 2. Cercare la posizione di inserzione

- Scorriamo la lista per trovare l'elemento prima di `positionPtr`

```
_Bool insertBefore(List *lsPtr, T e, List positionPtr) {
    ...
    // Scorriamo la lista tenendo traccia del predecessore
    List previousPtr = NULL;
    List currentPtr = *lsPtr;

    while (currentPtr != NULL && currentPtr != positionPtr) {
        previousPtr = currentPtr;
        currentPtr = currentPtr->next;
    }
}
```

...



# Inserimento Prima di Nodo Dato II

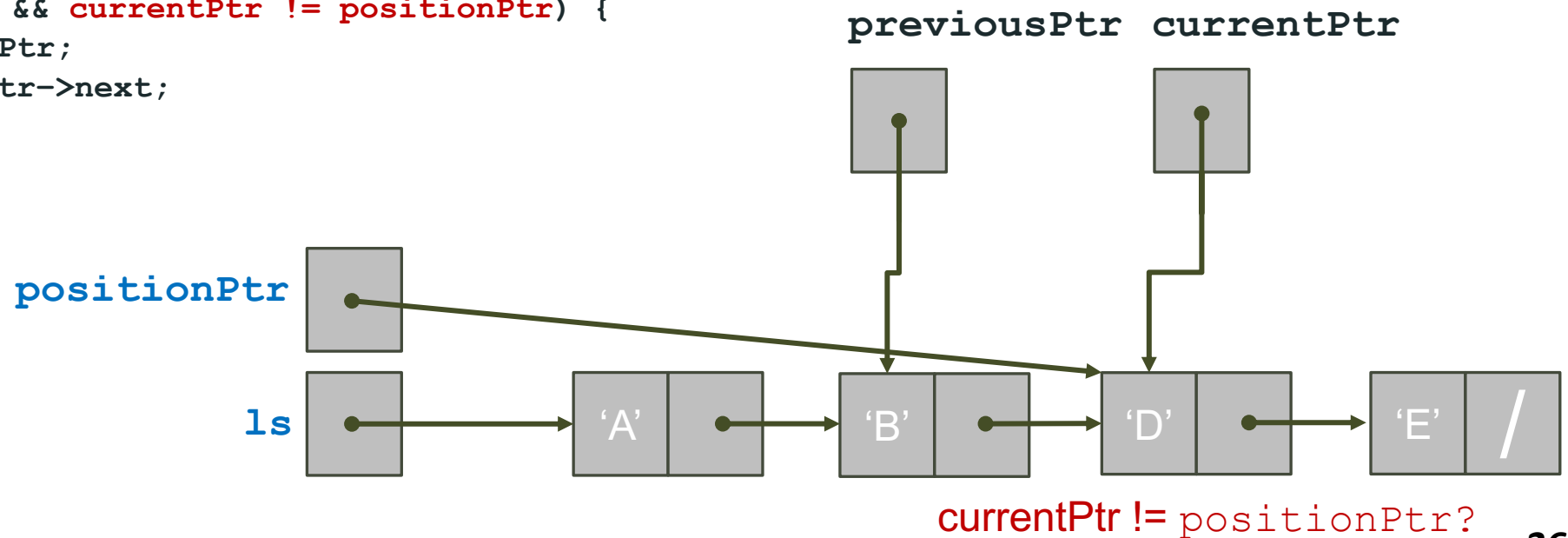
## 2. Cercare la posizione di inserzione

- Scorriamo la lista per trovare l'elemento prima di `positionPtr`

```
_Bool insertBefore(List *lsPtr, T e, List positionPtr) {
    ...
    // Scorriamo la lista tenendo traccia del predecessore
    List previousPtr = NULL;
    List currentPtr = *lsPtr;

    while (currentPtr != NULL && currentPtr != positionPtr) {
        previousPtr = currentPtr;
        currentPtr = currentPtr->next;
    }
}
```

...



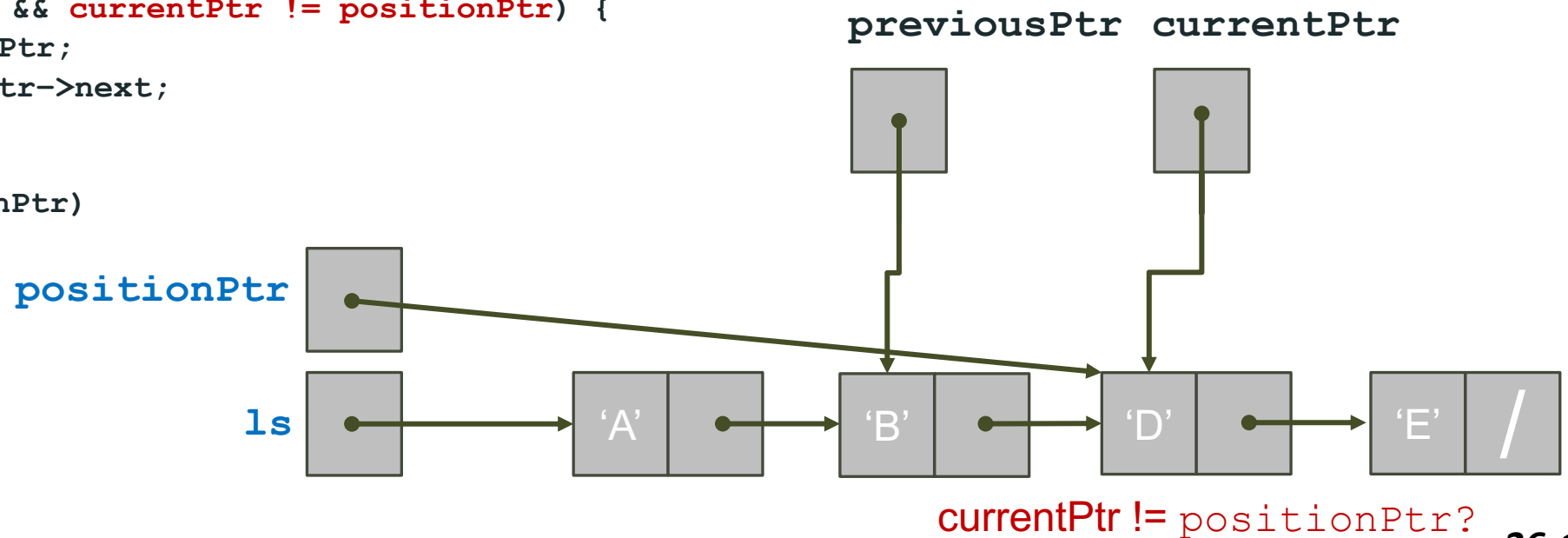
# Inserimento Prima di Nodo Dato II

## 2. Cercare la posizione di inserzione

- Scorriamo la lista per trovare l'elemento prima di `positionPtr`

```
_Bool insertBefore(List *lsPtr, T e, List positionPtr) {
    ...
    // Scorriamo la lista tenendo traccia del predecessore
    List previousPtr = NULL;
    List currentPtr = *lsPtr;

    while (currentPtr != NULL && currentPtr != positionPtr) {
        previousPtr = currentPtr;
        currentPtr = currentPtr->next;
    }
    // Controllo se valido
    if (currentPtr != positionPtr)
        return 0;
    ...
}
```





# Inserimento Dopo di Nodo Dato I

- Supponiamo di voler inserire 'C' in una lista di caratteri dopo del nodo `positionPtr`
  - Se `positionPtr` è NULL: inserimento in testa.

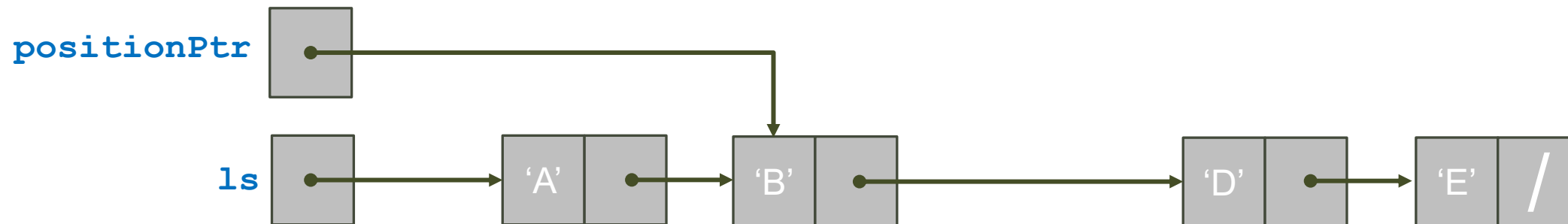
•

1.

2.

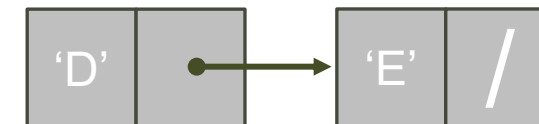
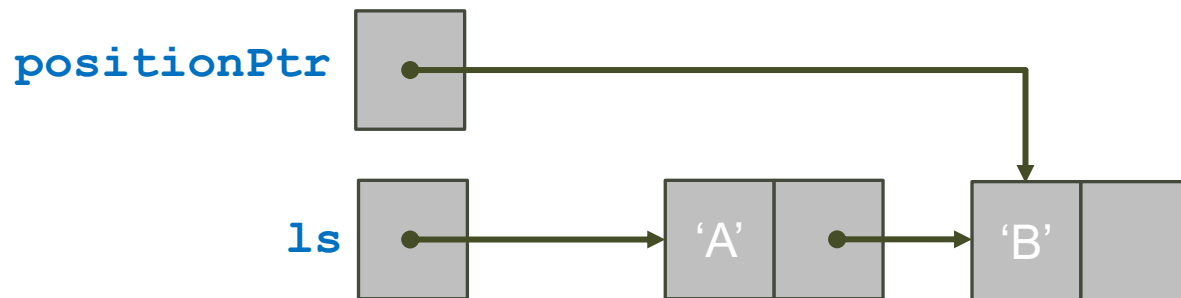
•

3.



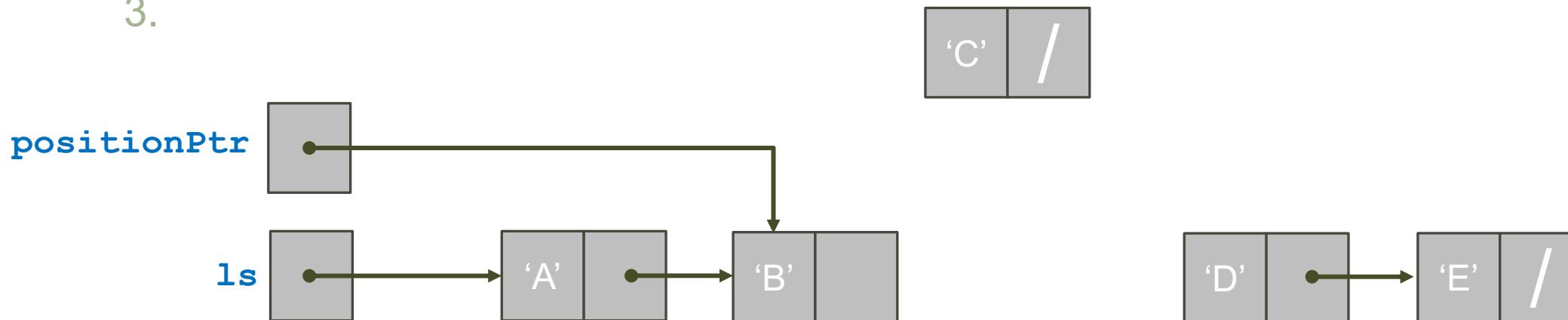
# Inserimento Dopo di Nodo Dato I

- Supponiamo di voler inserire 'C' in una lista di caratteri dopo del nodo `positionPtr`
  - Se `positionPtr` è `NULL`: inserimento in testa.
- Non serve passo 2, ma cambia 3:
  - 1.
  - 2.
  - 3.



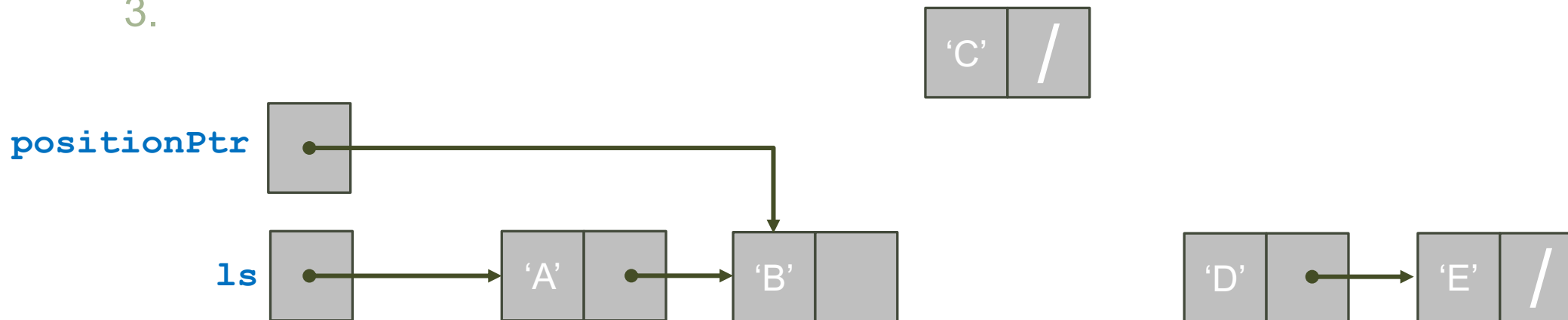
# Inserimento Dopo di Nodo Dato I

- Supponiamo di voler inserire 'C' in una lista di caratteri dopo del nodo `positionPtr`
  - Se `positionPtr` è `NULL`: inserimento in testa.
- Non serve passo 2, ma cambia 3:
  1. Creare un nuovo nodo per 'C'
  - 2.
  - 3.



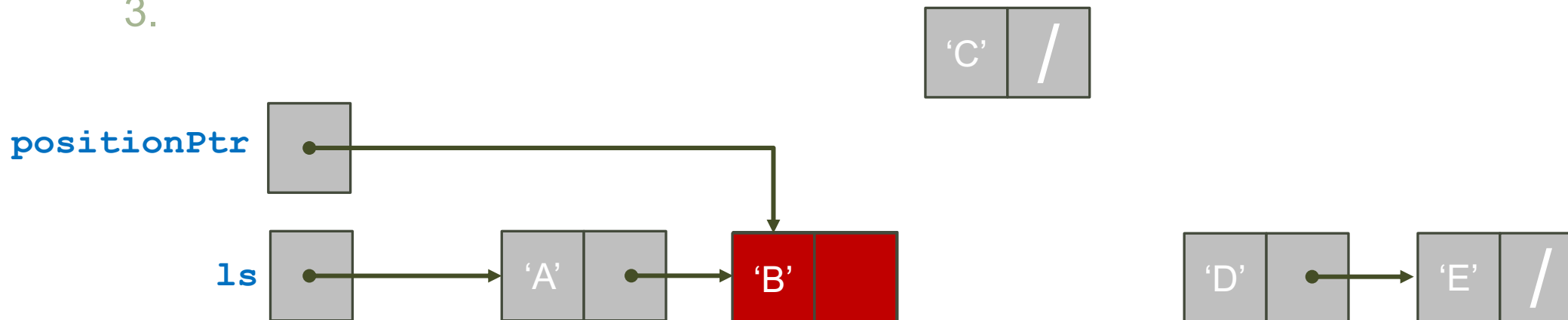
# Inserimento Dopo di Nodo Dato I

- Supponiamo di voler inserire 'C' in una lista di caratteri dopo del nodo `positionPtr`
  - Se `positionPtr` è NULL: inserimento in testa.
- Non serve passo 2, ma cambia 3:
  1. Creare un nuovo nodo per 'C'
  - ~~2. Cerco il nodo successore di `positionPtr`~~
  - 3.



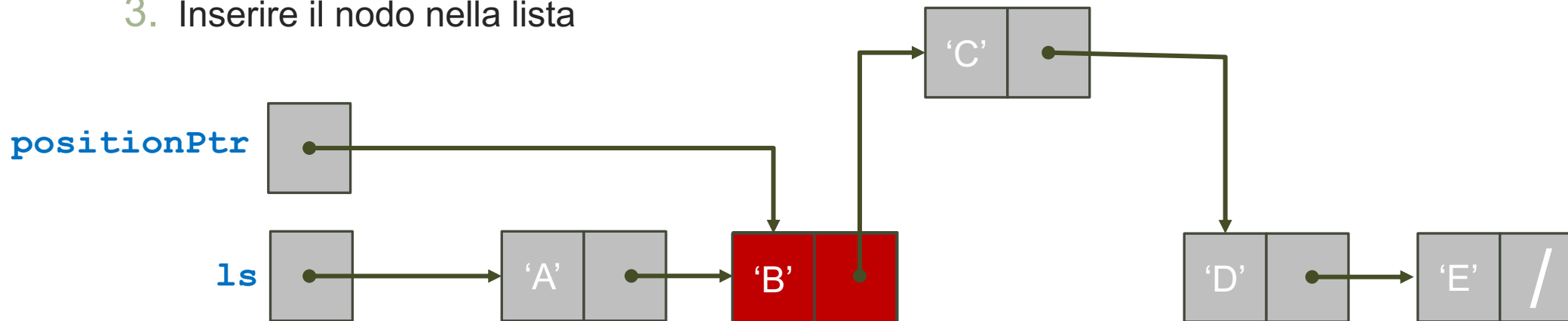
# Inserimento Dopo di Nodo Dato I

- Supponiamo di voler inserire 'C' in una lista di caratteri dopo del nodo `positionPtr`
  - Se `positionPtr` è NULL: inserimento in testa.
- Non serve passo 2, ma cambia 3:
  1. Creare un nuovo nodo per 'C'
  - ~~2. Cerco il nodo successore di `positionPtr`~~
    - Salvato in `next`
  - 3.



# Inserimento Dopo di Nodo Dato I

- Supponiamo di voler inserire 'C' in una lista di caratteri dopo del nodo `positionPtr`
  - Se `positionPtr` è NULL: inserimento in testa.
- Non serve passo 2, ma cambia 3:
  1. Creare un nuovo nodo per 'C'
  - ~~2. Cerco il nodo successore di `positionPtr`~~
    - Salvato in `next`
  3. Inserire il nodo nella lista



# Inserimento Dopo di Nodo Dato II

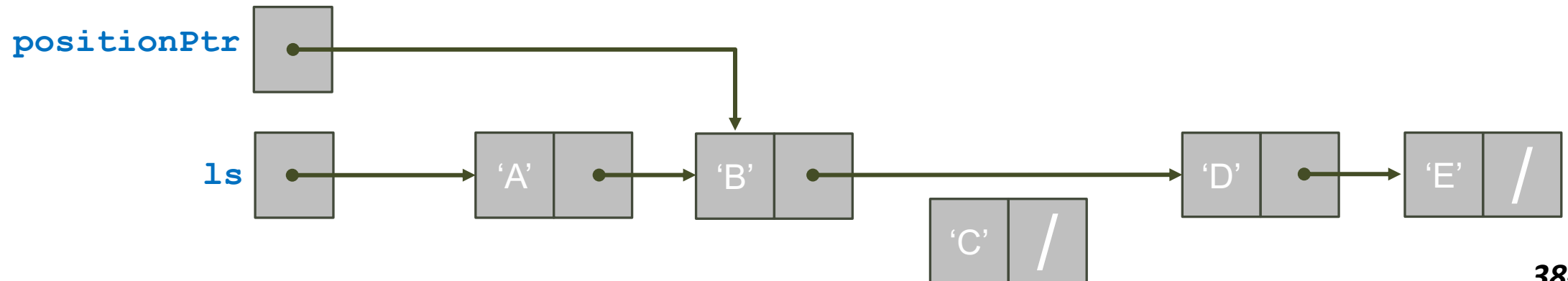


## 3. Inserire il nodo nella lista

- Dobbiamo inserire il nuovo nodo come successore di **previousPtr** e predecessore di **currentPtr**

```
_Bool insertAfter(List *lsPtr, T e, List positionPtr) {  
    ...  
    if (positionPtr != NULL) {  
  
    }  
}
```

...



# Inserimento Dopo di Nodo Dato II

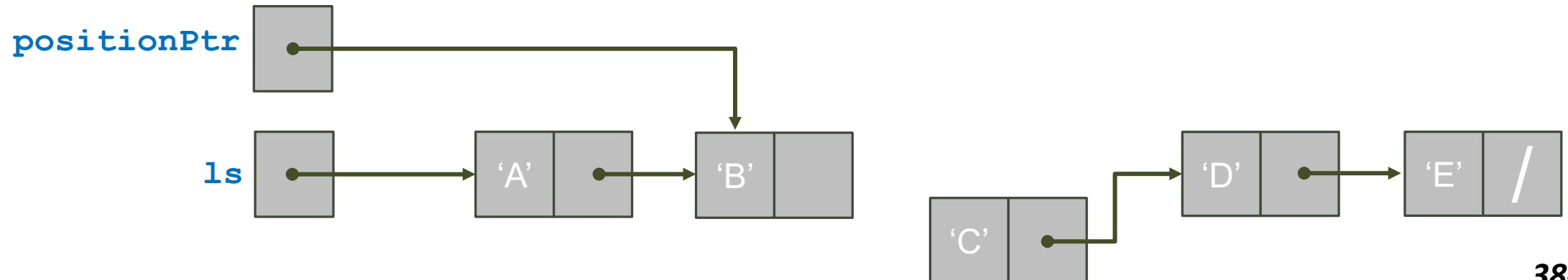


## 3. Inserire il nodo nella lista

- Dobbiamo inserire il nuovo nodo come successore di **previousPtr** e predecessore di **currentPtr**

```
_Bool insertAfter(List *lsPtr, T e, List positionPtr) {  
    ...  
    if (positionPtr != NULL) {  
        newPtr->next = positionPtr->next;  
    }  
}
```

...





# Inserimento Dopo di Nodo Dato II

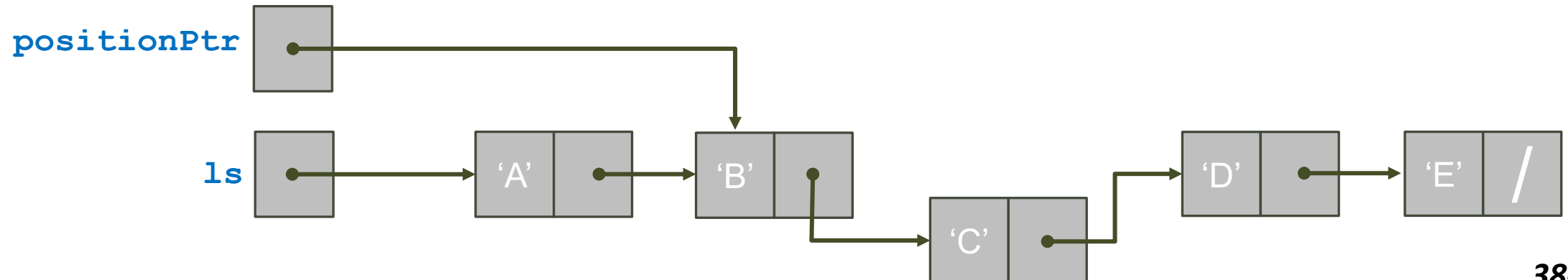


## 3. Inserire il nodo nella lista

- Dobbiamo inserire il nuovo nodo come successore di **previousPtr** e predecessore di **currentPtr**

```
_Bool insertAfter(List *lsPtr, T e, List positionPtr) {  
    ...  
    if (positionPtr != NULL) {  
        newPtr->next = positionPtr->next;  
        positionPtr->next = newPtr;  
    }  
}
```

...



# Inserimento Dopo di Nodo Dato II

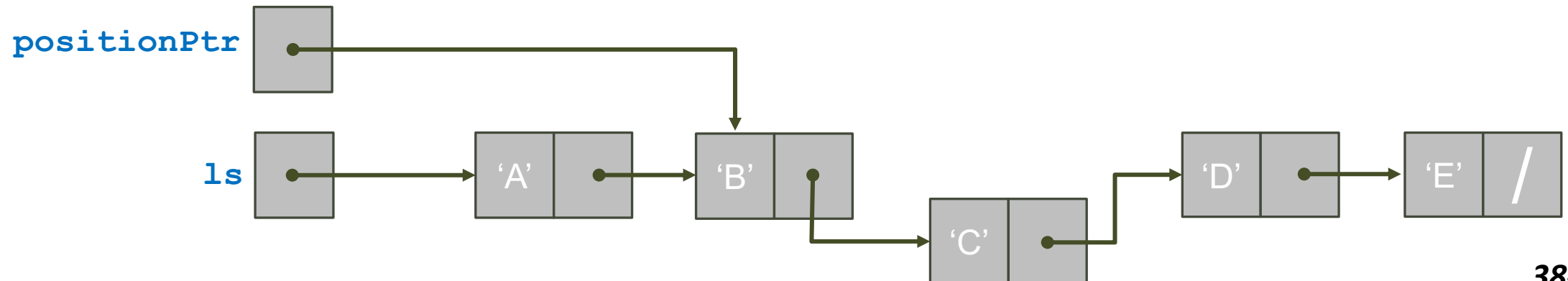
## 3. Inserire il nodo nella lista

- Dobbiamo inserire il nuovo nodo come successore di **previousPtr** e predecessore di **currentPtr**

```
_Bool insertAfter(List *lsPtr, T e, List positionPtr) {
```

```
    ...
    if (positionPtr != NULL) {
        newPtr->next = positionPtr->next;
        positionPtr->next = newPtr;
    }
```

```
    else {
        newPtr->next = *lsPtr;
        *lsPtr = newPtr;
    }
    ...
```

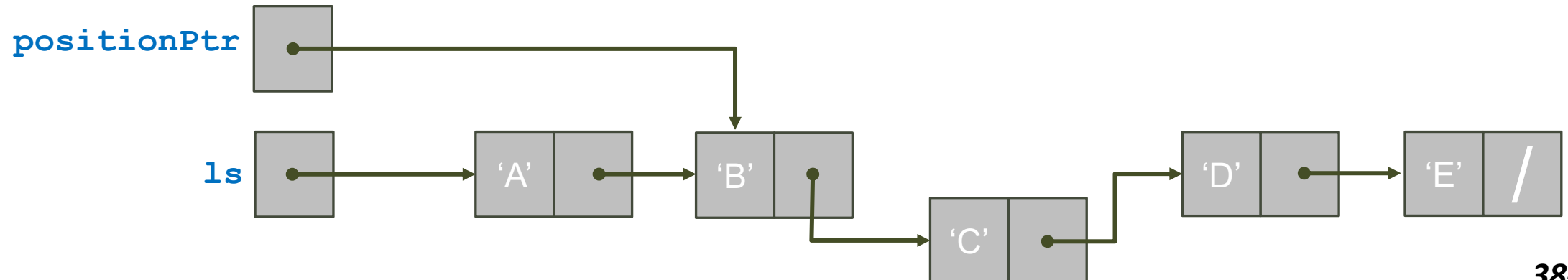


# Inserimento Dopo di Nodo Dato II

## 3. Inserire il nodo nella lista

- Dobbiamo inserire il nuovo nodo come successore di **previousPtr** e predecessore di **currentPtr**

```
_Bool insertAfter(List *lsPtr, T e, List positionPtr) {
    ...
    if (positionPtr != NULL) {
        newPtr->next = positionPtr->next;
        positionPtr->next = newPtr;
    }
    else {
        newPtr->next = *lsPtr;
        *lsPtr = newPtr;
    }
    ...
}
```



# Cancellazione da una Lista

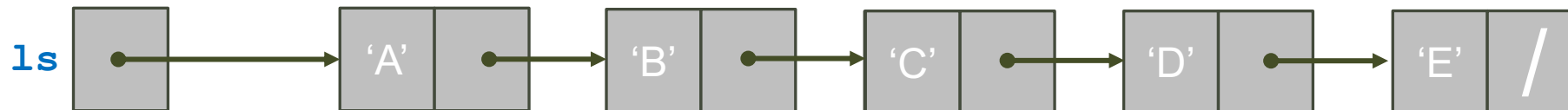
# Cancellazione



- Consideriamo tre diverse operazioni di cancellazione:
  - `_Bool deleteFromCharList(CharList *lsPtr, char value);`
    - Cancella dalla lista la prima occorrenza dell'elemento `value`
  - `_Bool delete(List *lsPtr, List positionPtr);`
    - Cancella il nodo puntato da `positionPtr`
  - `_Bool deleteAfter(List *lsPtr, List positionPtr);`
    - Cancella il nodo dopo il nodo puntato da `positionPtr` (se `positionPtr == NULL`, cancella il nodo di testa)
- Tutte e tre le operazioni restituiscono l'esito:
  - Quando l'inserimento può fallire?
    - Se l'elemento da cancellare non è presente.

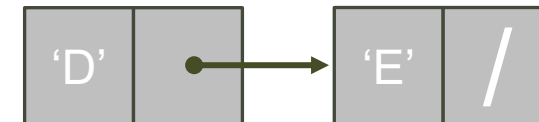
# Cancellazione Prima Occorrenza I

- Supponiamo di voler cancellare 'C' da una lista di caratteri `ls`
  - `ls` contiene ['A', 'B', 'C', 'D', 'E']
- - 1.
  - 2.
  - 3.



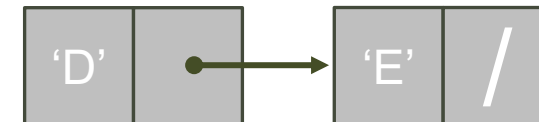
# Cancellazione Prima Occorrenza I

- Supponiamo di voler cancellare 'C' da una lista di caratteri `ls`
  - `ls` contiene ['A', 'B', 'C', 'D', 'E']
- La cancellazione consiste in 3 passi:
  - 1.
  - 2.
  - 3.



# Cancellazione Prima Occorrenza I

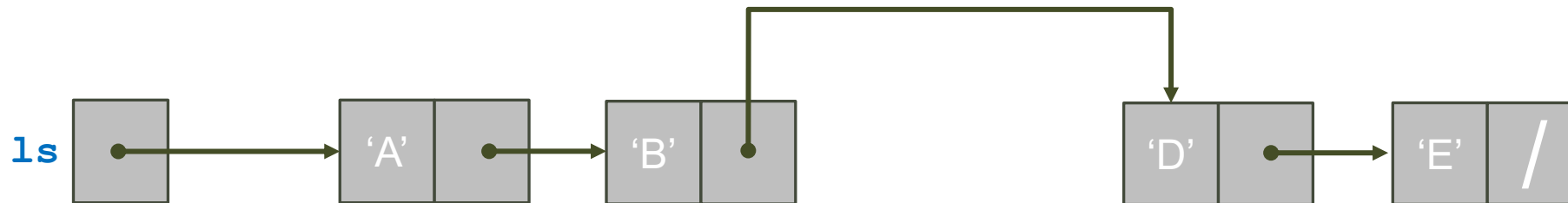
- Supponiamo di voler cancellare 'C' da una lista di caratteri `ls`
  - `ls` contiene ['A', 'B', 'C', 'D', 'E']
- La cancellazione consiste in 3 passi:
  1. Cercare l'elemento da cancellare
  - 2.
  - 3.





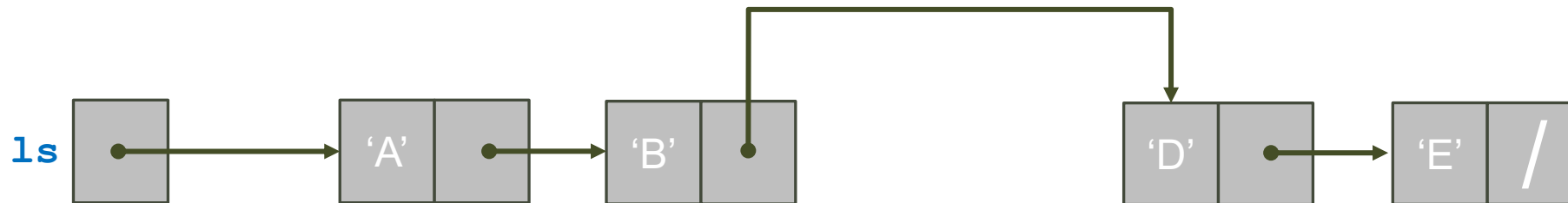
# Cancellazione Prima Occorrenza I

- Supponiamo di voler cancellare 'C' da una lista di caratteri `ls`
  - `ls` contiene ['A', 'B', 'C', 'D', 'E']
- La cancellazione consiste in 3 passi:
  1. Cercare l'elemento da cancellare
  2. Rimuovere il nodo dalla lista
  - 3.



# Cancellazione Prima Occorrenza I

- Supponiamo di voler cancellare 'C' da una lista di caratteri `ls`
  - `ls` contiene ['A', 'B', 'C', 'D', 'E']
- La cancellazione consiste in 3 passi:
  1. Cercare l'elemento da cancellare
  2. Rimuovere il nodo dalla lista
  3. Liberare la memoria



# Cancellazione Prima Occorrenza II



## 1. Cercare l'elemento da eliminare

- Scorriamo la lista per trovare il primo elemento con `data == value`

```
_Bool deleteFromCharList(CharList *lsPtr, char value) {  
    // Scorriamo la lista tenendo traccia del predecessore
```

...



# Cancellazione Prima Occorrenza II

## 1. Cercare l'elemento da eliminare

- Scorriamo la lista per trovare il primo elemento con `data == value`

```
_Bool deleteFromCharList(CharList *lsPtr, char value) {  
    // Scorriamo la lista tenendo traccia del predecessore  
    CharList previousPtr = NULL;
```

**previousPtr**



...



# Cancellazione Prima Occorrenza II

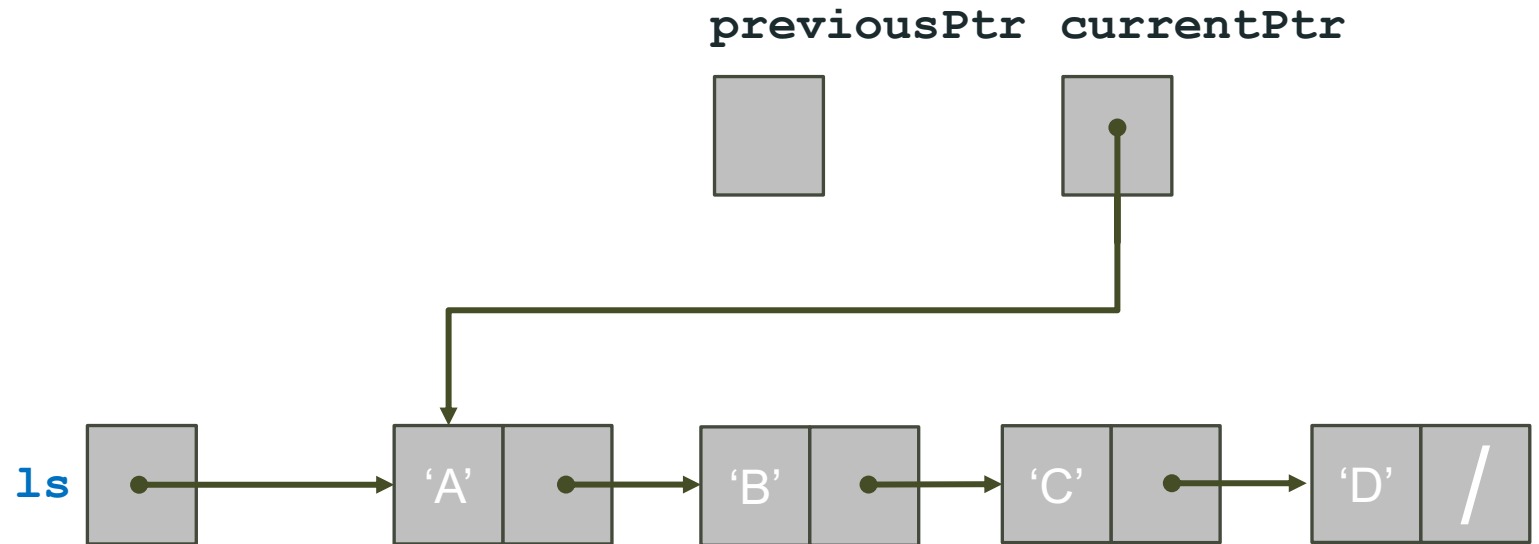


## 1. Cercare l'elemento da eliminare

- Scorriamo la lista per trovare il primo elemento con `data == value`

```
_Bool deleteFromCharList(CharList *lsPtr, char value) {  
    // Scorriamo la lista tenendo traccia del predecessore  
    CharList previousPtr = NULL;  
    CharList currentPtr = *lsPtr;
```

...



# Cancellazione Prima Occorrenza II

## 1. Cercare l'elemento da eliminare

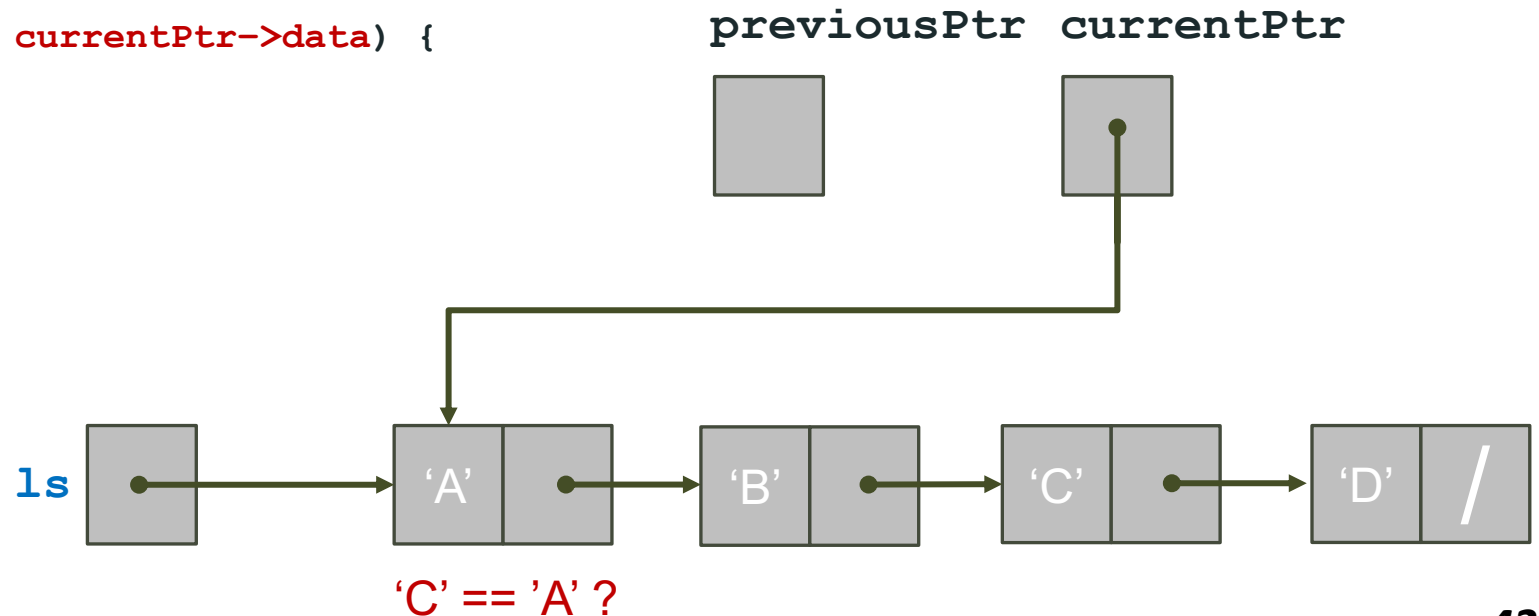
- Scorriamo la lista per trovare il primo elemento con `data == value`

```
_Bool deleteFromCharList(CharList *lsPtr, char value) {
    // Scorriamo la lista tenendo traccia del predecessore
    CharList previousPtr = NULL;
    CharList currentPtr = *lsPtr;

    while (currentPtr != NULL && value != currentPtr->data) {

```

...



# Cancellazione Prima Occorrenza II

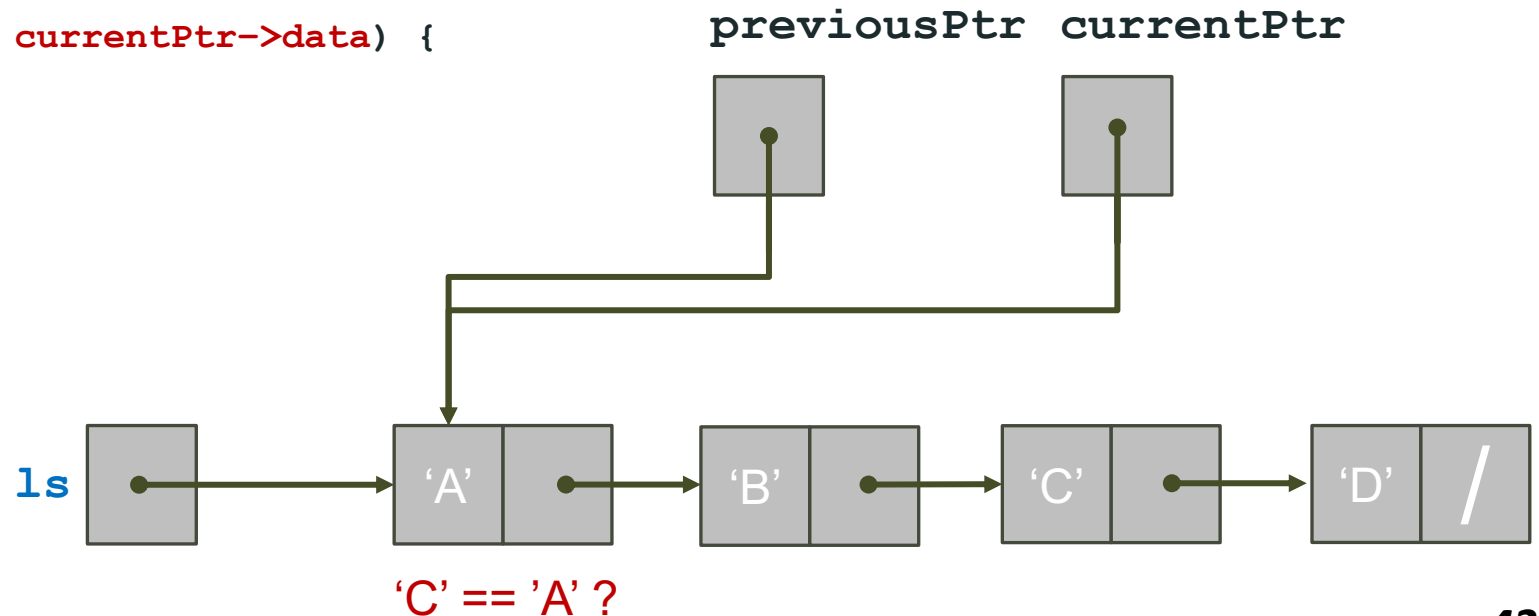


## 1. Cercare l'elemento da eliminare

- Scorriamo la lista per trovare il primo elemento con `data == value`

```
_Bool deleteFromCharList(CharList *lsPtr, char value) {  
    // Scorriamo la lista tenendo traccia del predecessore  
    CharList previousPtr = NULL;  
    CharList currentPtr = *lsPtr;  
  
    while (currentPtr != NULL && value != currentPtr->data) {  
        previousPtr = currentPtr;  
    }  
}
```

...



# Cancellazione Prima Occorrenza II

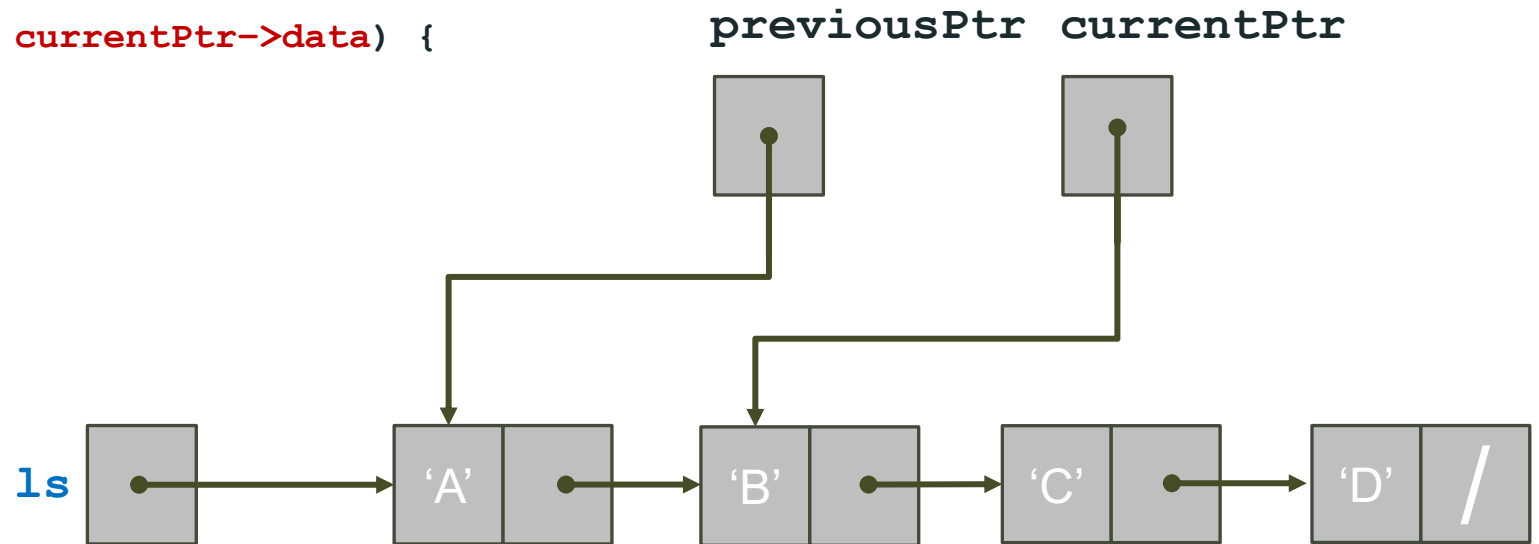
## 1. Cercare l'elemento da eliminare

- Scorriamo la lista per trovare il primo elemento con `data == value`

```
_Bool deleteFromCharList(CharList *lsPtr, char value) {
    // Scorriamo la lista tenendo traccia del predecessore
    CharList previousPtr = NULL;
    CharList currentPtr = *lsPtr;
```

```
while (currentPtr != NULL && value != currentPtr->data) {
    previousPtr = currentPtr;
    currentPtr = currentPtr->next;
}
```

• • •





# Cancellazione Prima Occorrenza II

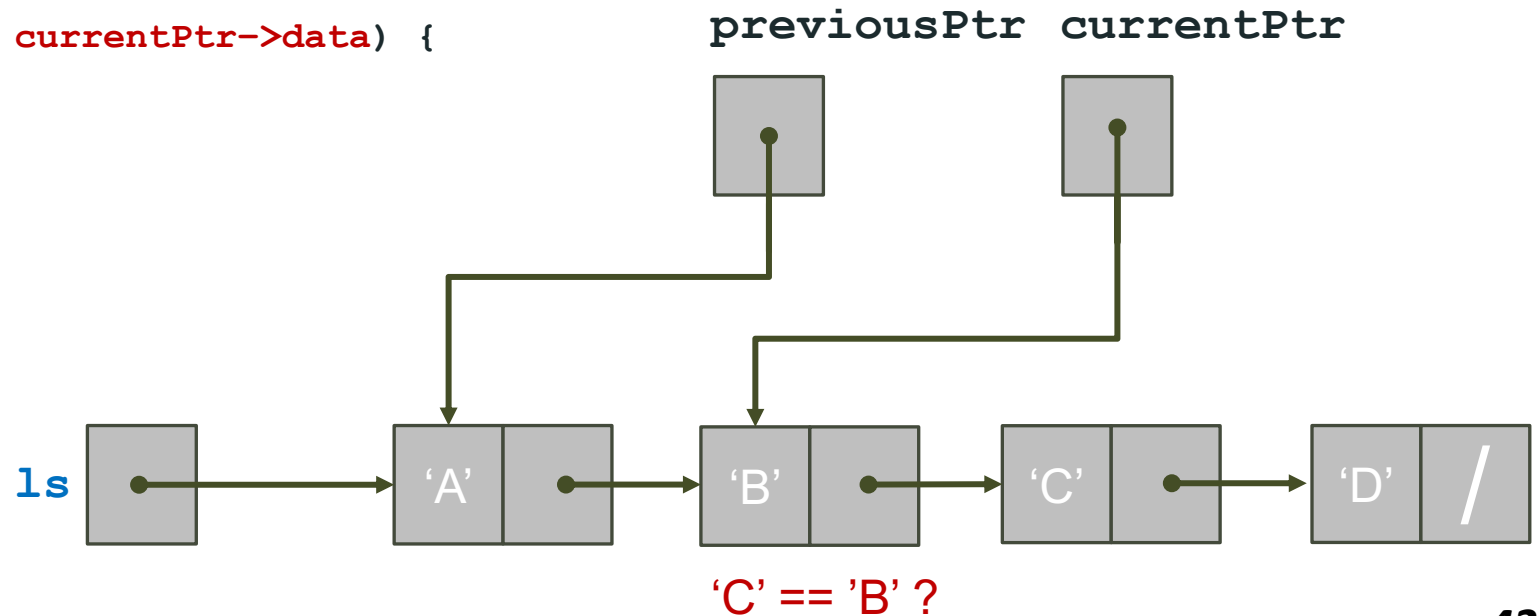


## 1. Cercare l'elemento da eliminare

- Scorriamo la lista per trovare il primo elemento con `data == value`

```
_Bool deleteFromCharList(CharList *lsPtr, char value) {  
    // Scorriamo la lista tenendo traccia del predecessore  
    CharList previousPtr = NULL;  
    CharList currentPtr = *lsPtr;  
  
    while (currentPtr != NULL && value != currentPtr->data) {  
        previousPtr = currentPtr;  
        currentPtr = currentPtr->next;  
    }  
}
```

...



# Cancellazione Prima Occorrenza II

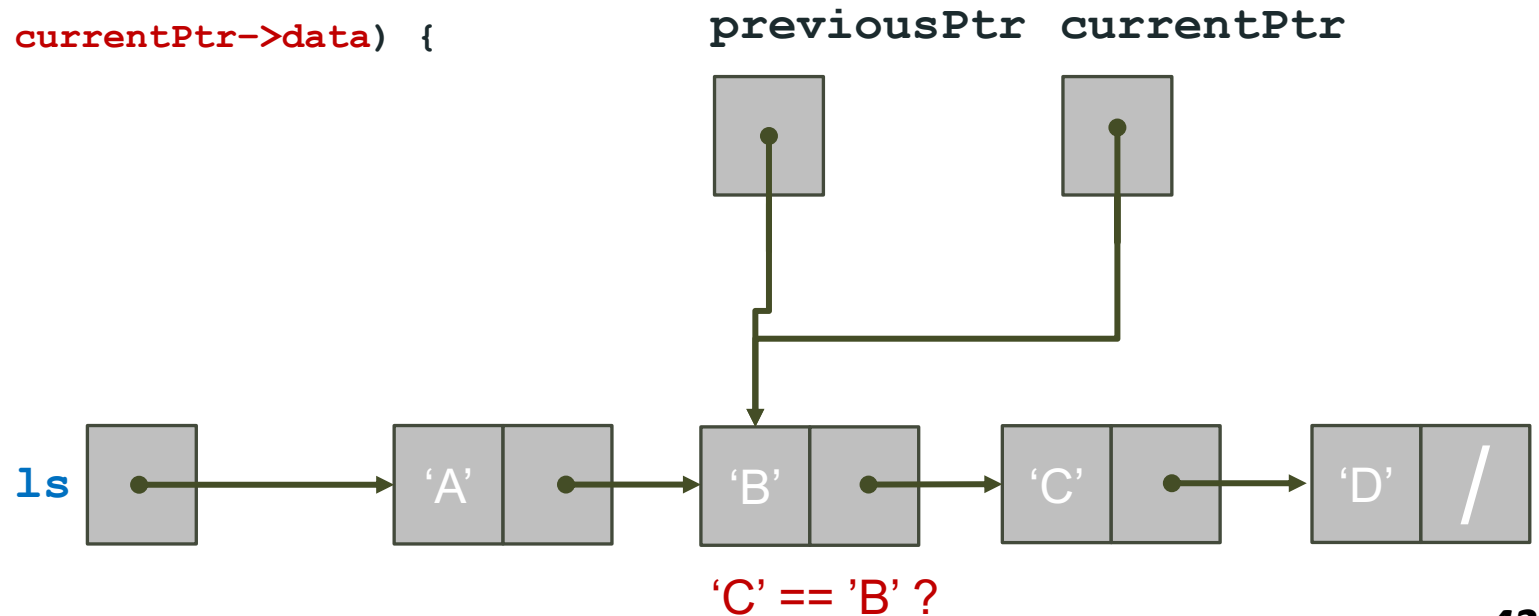


## 1. Cercare l'elemento da eliminare

- Scorriamo la lista per trovare il primo elemento con `data == value`

```
_Bool deleteFromCharList(CharList *lsPtr, char value) {  
    // Scorriamo la lista tenendo traccia del predecessore  
    CharList previousPtr = NULL;  
    CharList currentPtr = *lsPtr;  
  
    while (currentPtr != NULL && value != currentPtr->data) {  
        previousPtr = currentPtr;  
        currentPtr = currentPtr->next;  
    }  
}
```

...



# Cancellazione Prima Occorrenza II

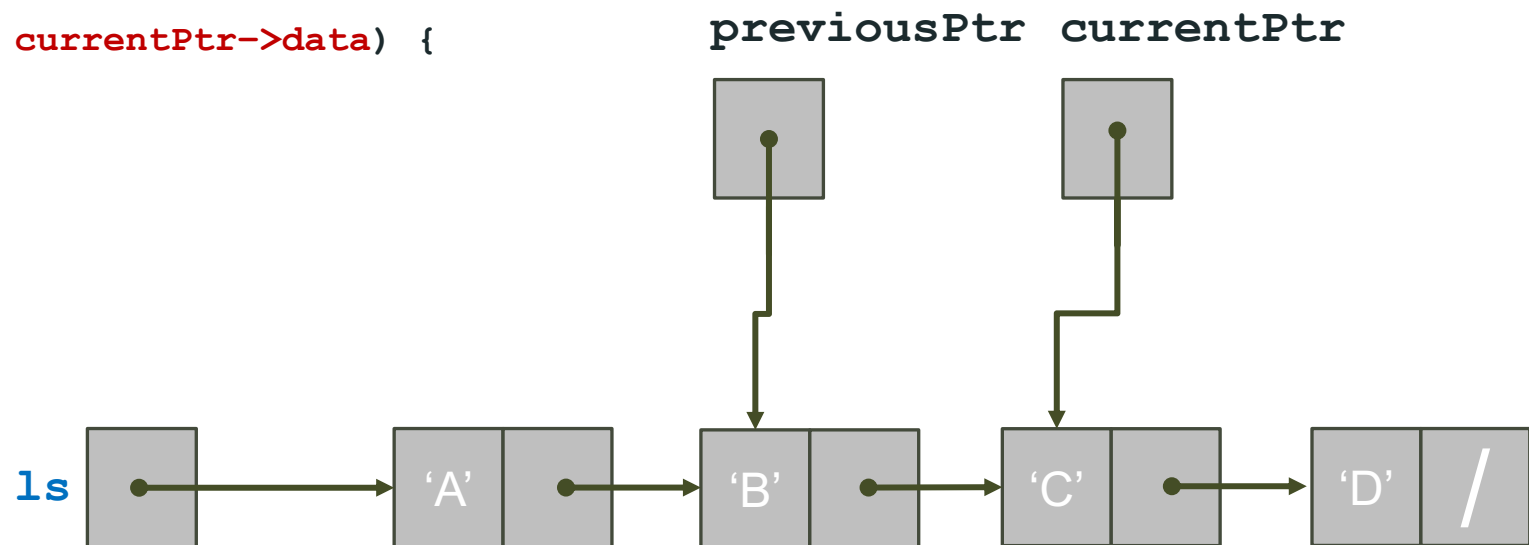
## 1. Cercare l'elemento da eliminare

- Scorriamo la lista per trovare il primo elemento con `data == value`

```
_Bool deleteFromCharList(CharList *lsPtr, char value) {
    // Scorriamo la lista tenendo traccia del predecessore
    CharList previousPtr = NULL;
    CharList currentPtr = *lsPtr;

    while (currentPtr != NULL && value != currentPtr->data) {
        previousPtr = currentPtr;
        currentPtr = currentPtr->next;
    }
}
```

...



# Cancellazione Prima Occorrenza II

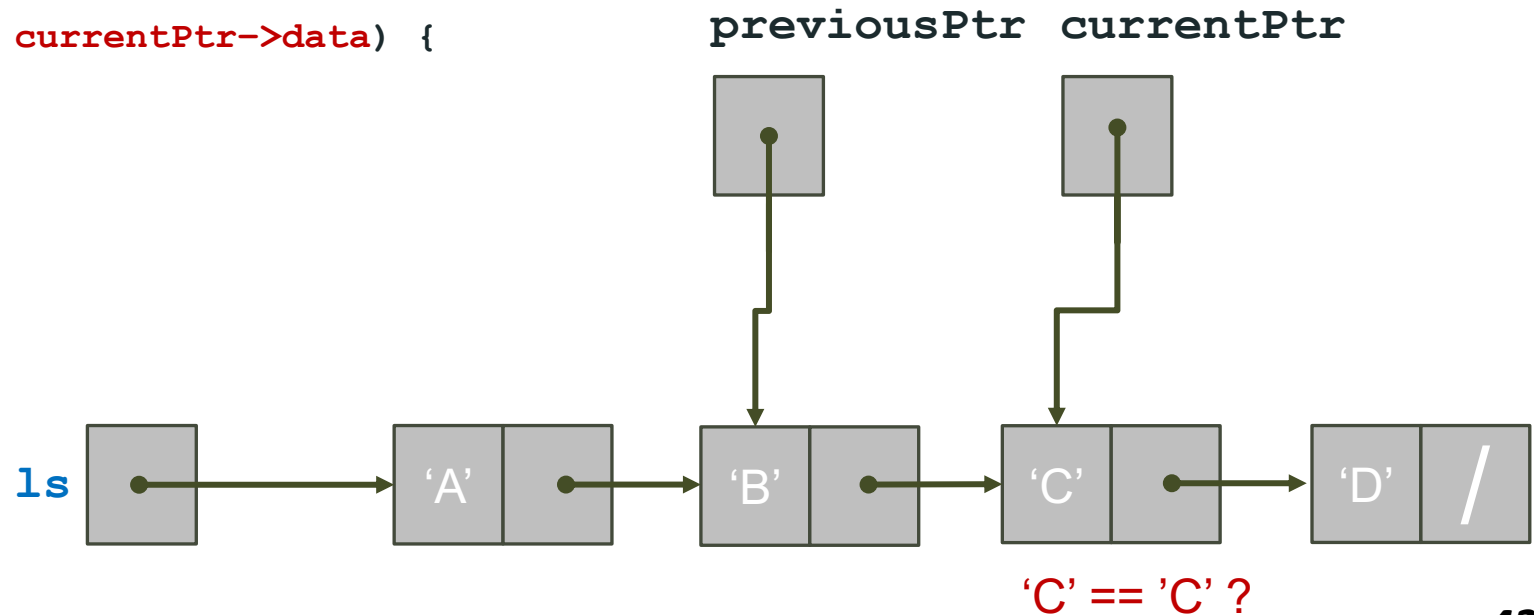


## 1. Cercare l'elemento da eliminare

- Scorriamo la lista per trovare il primo elemento con `data == value`

```
_Bool deleteFromCharList(CharList *lsPtr, char value) {  
    // Scorriamo la lista tenendo traccia del predecessore  
    CharList previousPtr = NULL;  
    CharList currentPtr = *lsPtr;  
  
    while (currentPtr != NULL && value != currentPtr->data) {  
        previousPtr = currentPtr;  
        currentPtr = currentPtr->next;  
    }  
}
```

...



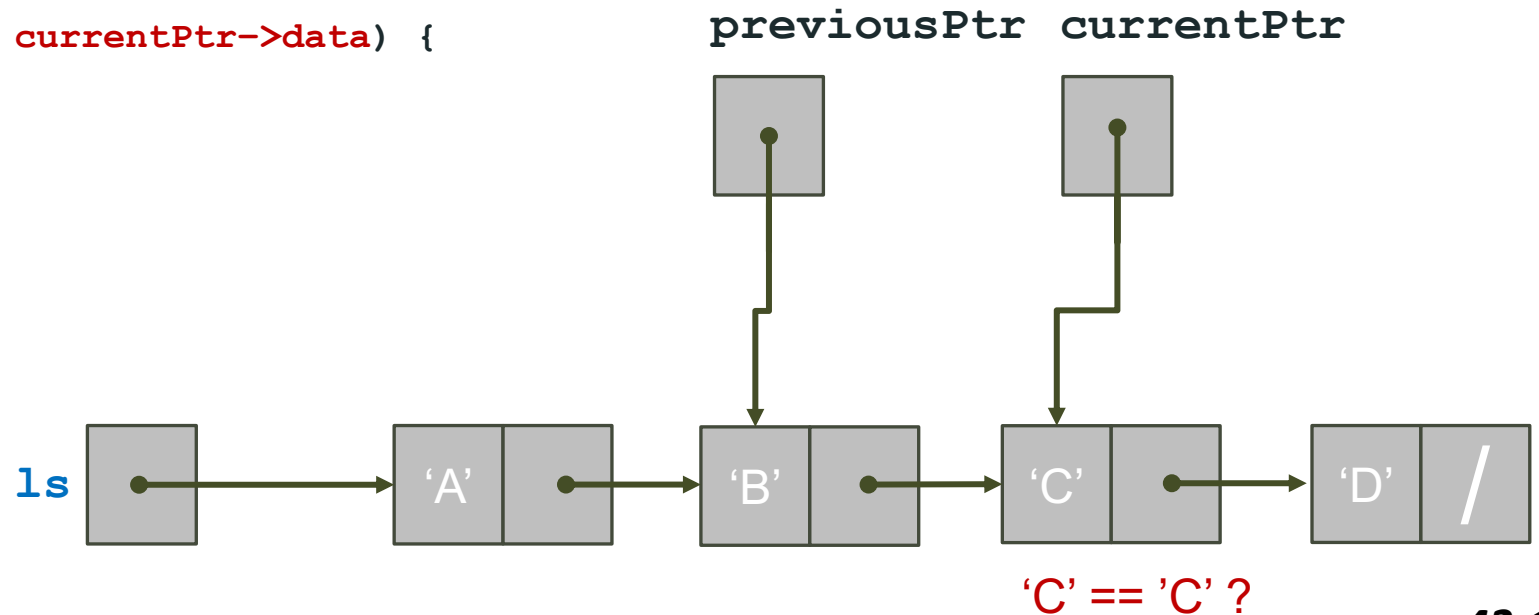
# Cancellazione Prima Occorrenza II



## 1. Cercare l'elemento da eliminare

- Scorriamo la lista per trovare il primo elemento con `data == value`

```
_Bool deleteFromCharList(CharList *lsPtr, char value) {  
    // Scorriamo la lista tenendo traccia del predecessore  
    CharList previousPtr = NULL;  
    CharList currentPtr = *lsPtr;  
  
    while (currentPtr != NULL && value != currentPtr->data) {  
        previousPtr = currentPtr;  
        currentPtr = currentPtr->next;  
    }  
  
    if (currentPtr == NULL)  
        return 0; // value non è presente  
    ...  
}
```



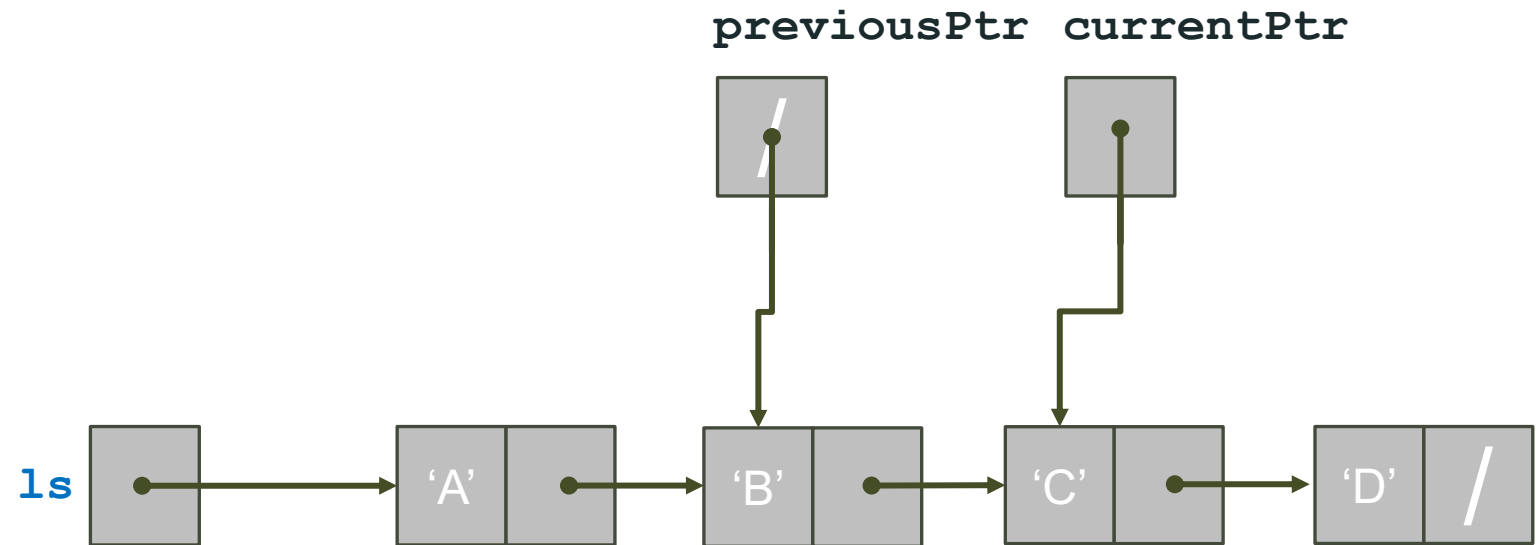
# Cancellazione Prima Occorrenza III



## 2. Rimuovere il nodo dalla lista

- Cancello il nodo come successore di `previousPtr`

...



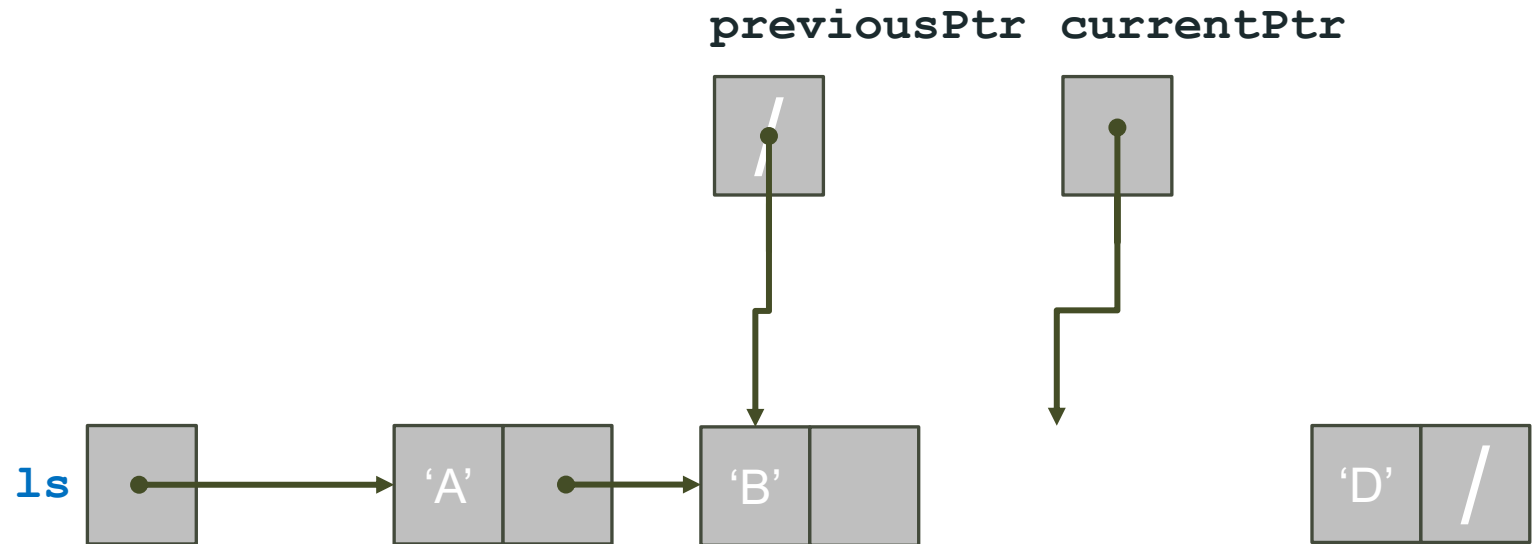
# Cancellazione Prima Occorrenza III



## 2. Rimuovere il nodo dalla lista

- Cancello il nodo come successore di **previousPtr**

```
...  
if (previousPtr != NULL){ // Il nodo da eliminare è currentPtr  
  
}
```

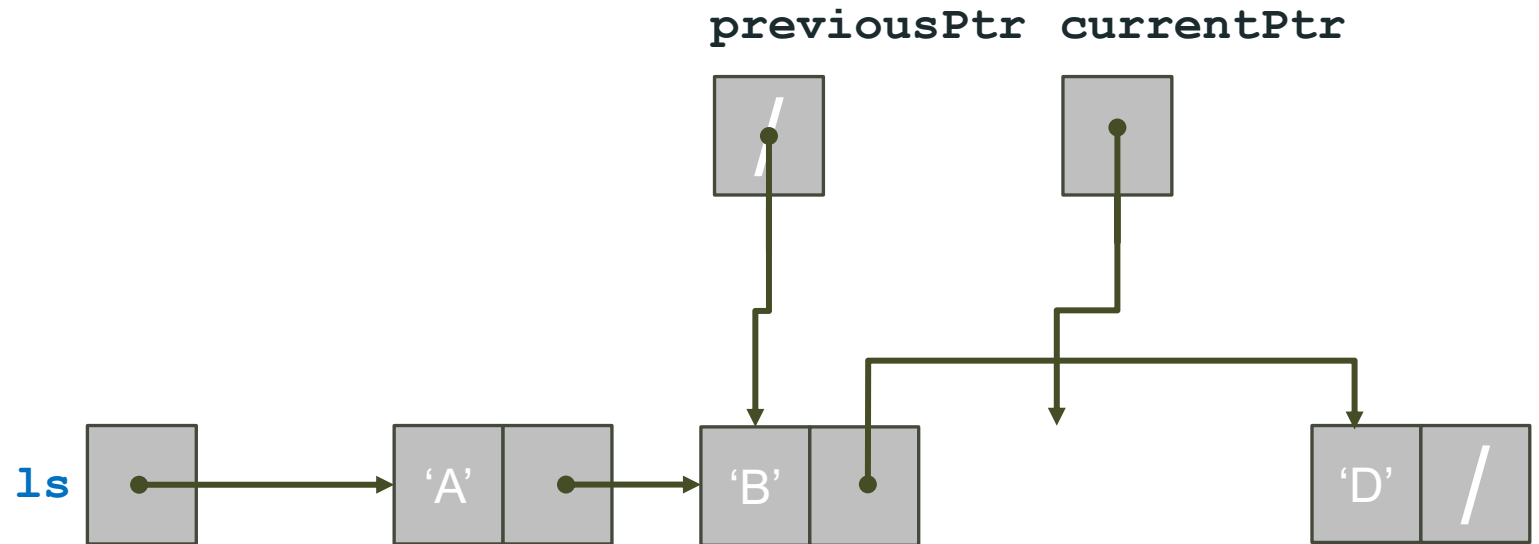


# Cancellazione Prima Occorrenza III

## 2. Rimuovere il nodo dalla lista

- Cancello il nodo come successore di **previousPtr**

```
...  
if (previousPtr != NULL){ // Il nodo da eliminare è currentPtr  
    previousPtr->next = currentPtr->next;  
}
```





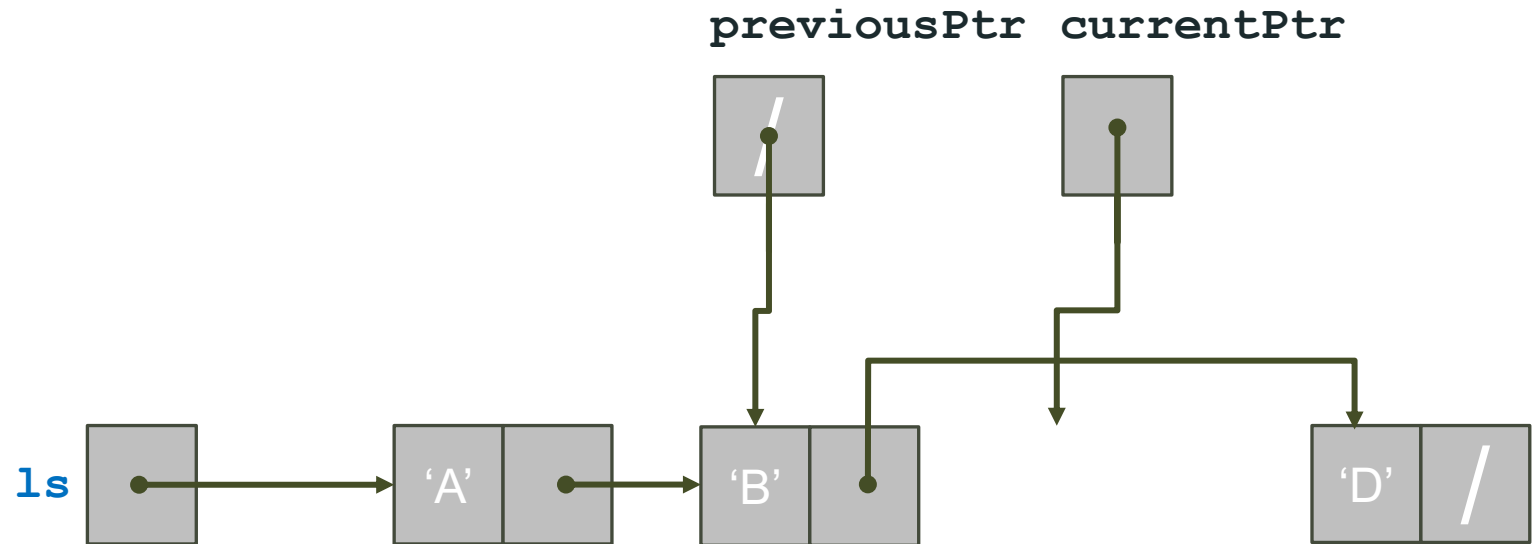
# Cancellazione Prima Occorrenza III

## 2. Rimuovere il nodo dalla lista

- Cancello il nodo come successore di **previousPtr**

```
...  
if (previousPtr != NULL){ // Il nodo da eliminare è currentPtr  
    previousPtr->next = currentPtr->next;  
}
```

```
free(currentPtr);
```

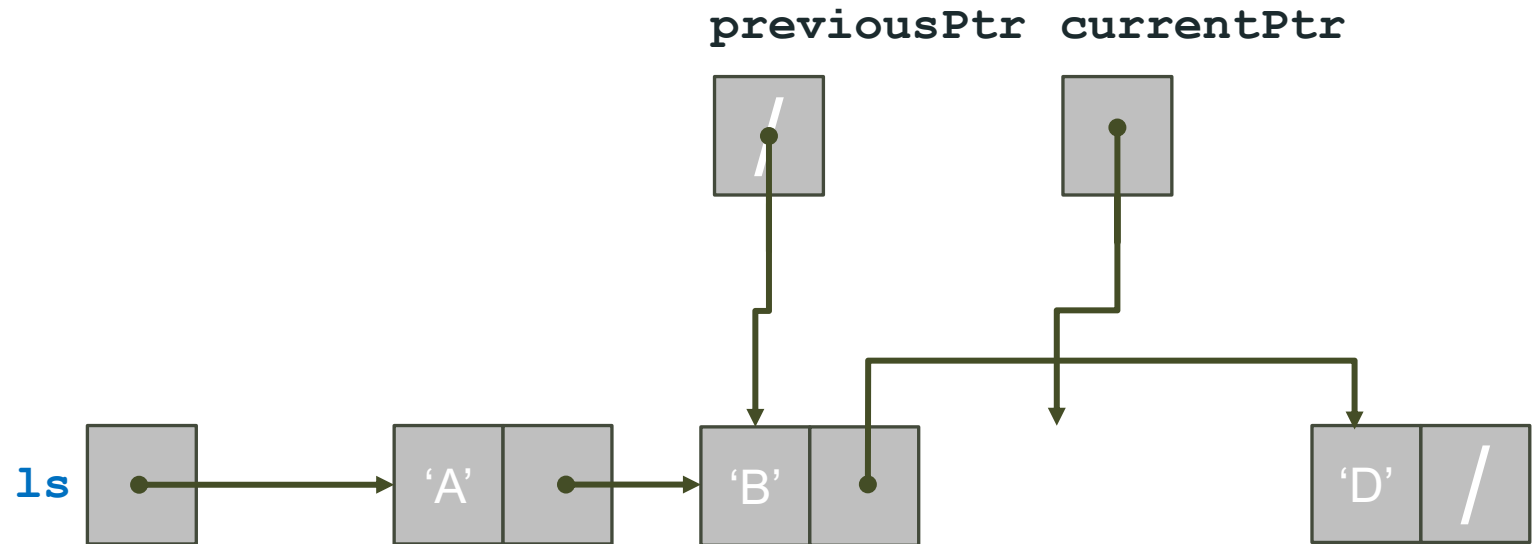


# Cancellazione Prima Occorrenza III

## 2. Rimuovere il nodo dalla lista

- Cancello il nodo come successore di **previousPtr**

```
...  
if (previousPtr != NULL){ // Il nodo da eliminare è currentPtr  
    previousPtr->next = currentPtr->next;  
}  
else { // Elimina nodo di testa  
    *lsPtr = (*lsPtr)->next;  
}  
free(currentPtr);  
return 1;  
}
```



# Esercizi

- Provate ad implementare:

1. `_Bool delete(List *lsPtr, List positionPtr);`

- Cancella il nodo puntato da `positionPtr`

2. `_Bool deleteAfter(List *lsPtr, List positionPtr);`

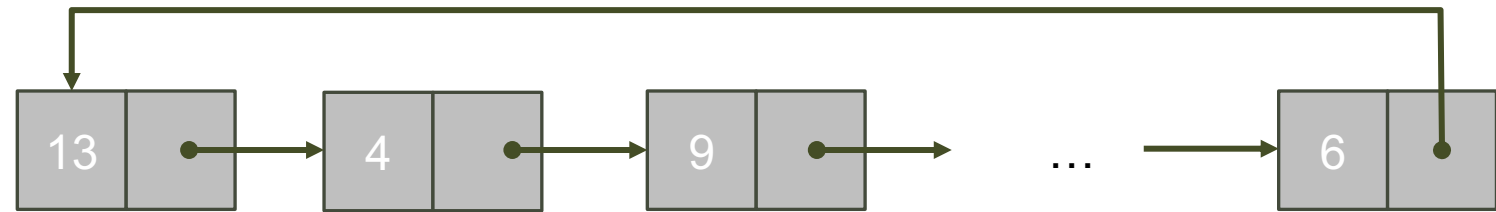
- Cancella il nodo dopo il nodo puntato da `positionPtr` (se `positionPtr == NULL`, cancella il nodo di testa)

# Tipi di Lista

# Tipi di Lista

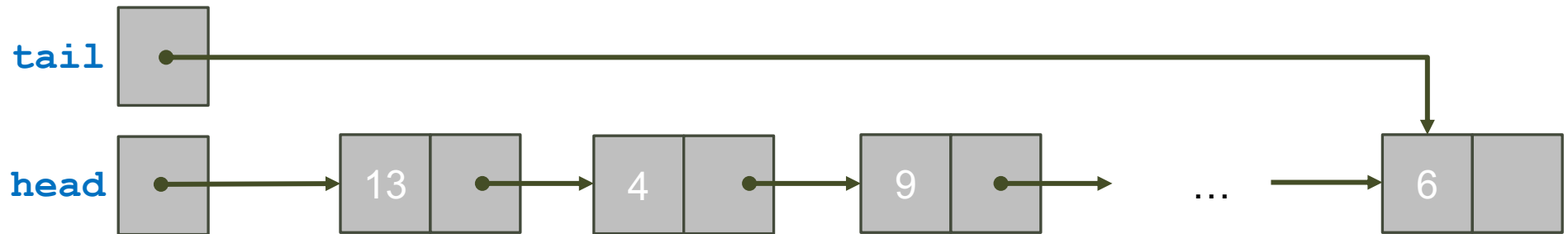
- La lista linkata è la base per realizzare liste di vari tipi:

- Lista circolare:



- Lista double-ended:

- Si può inserire/estrarre ad ambo le estremità



- Lista doppiamente linkata:

